

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Giao thức vay và cho vay phi tập trung trên nền tảng
Ethereum**

ĐINH TẠ HOÀNG DŨNG

Dung.DTH225290@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. Nguyễn Đức Toàn

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Giao thức vay và cho vay phi tập trung trên nền tảng
Ethereum

ĐINH TẠ HOÀNG DŨNG

Dung.DTH225290@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn:

TS. Nguyễn Đức Toàn

Chữ ký GVHD

Khoa:

Kỹ thuật máy tính

Trường:

Công nghệ Thông tin và Truyền thông

HÀ NỘI, 2026

LỜI CẢM ƠN

Tôi xin gửi lời cảm ơn chân thành đến TS. Nguyễn Đức Toàn — người đã định hướng đề tài, phản biện các quyết định kỹ thuật và nhắc tôi phân biệt giữa làm cho xong và làm cho đúng. Những buổi trao đổi thẳng thắn trong suốt quá trình thực hiện đồ án có giá trị hơn bất kỳ tài liệu nào tôi tự đọc được.

Cảm ơn gia đình đã luôn tạo điều kiện và động viên tôi trong suốt quá trình học tập và thực hiện đồ án.

Cuối cùng, tôi ghi nhận những nỗ lực của bản thân trong việc vượt qua các khó khăn và thử thách để hoàn thành đồ án này.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Hệ thống tài chính truyền thống phụ thuộc vào các tổ chức trung gian như ngân hàng để vận hành dịch vụ vay và cho vay, dẫn đến chi phí vận hành cao, thiếu minh bạch, và loại trừ một bộ phận người dân không đủ điều kiện tiếp cận tín dụng. Tài chính phi tập trung (DeFi) đã ra đời nhằm giải quyết các hạn chế này bằng cách thay thế trung gian bằng hợp đồng thông minh trên blockchain. Các giao thức cho vay phi tập trung tiêu biểu như Compound và Aave đã chứng minh tính khả thi của mô hình này với tổng giá trị tài sản khóa lên đến hàng chục tỷ USD. Tuy nhiên, kiến trúc của các giao thức này rất phức tạp, tài liệu kỹ thuật phân tán và chủ yếu hướng đến nhà phát triển chuyên nghiệp, tạo ra rào cản lớn cho việc nghiên cứu và học tập trong môi trường học thuật.

Đồ án lựa chọn hướng tiếp cận xây dựng một ứng dụng cho vay phi tập trung hoàn chỉnh trên nền tảng Ethereum, lấy cảm hứng từ kiến trúc của Compound v2 và Aave v2 nhưng tập trung vào việc thiết kế, phân tích và triển khai từ đầu nhằm làm rõ bản chất kỹ thuật của từng thành phần thay vì tái sử dụng mã nguồn có sẵn.

Hệ thống được thiết kế theo kiến trúc ba tầng. Tầng hợp đồng thông minh gồm bảy smart contract viết bằng Solidity, áp dụng cơ chế Lazy Accrual kết hợp Global Interest Index, kiến trúc bảo mật năm tầng phòng vệ theo chiều sâu, và mẫu UUPS Proxy. Tầng backend sử dụng kiến trúc đa tiến trình event-driven với năm worker độc lập và RabbitMQ đảm bảo không mất sự kiện. Tầng frontend xây dựng bằng Next.js, xác thực người dùng qua chuẩn EIP-4361, và cập nhật dữ liệu thanh lý theo thời gian thực qua WebSocket.

Đóng góp chính của đồ án là xây dựng thành công một hệ thống cho vay phi tập trung được triển khai trên mạng thử nghiệm Sepolia, đồng thời thể hiện rõ quy trình thiết kế và các quyết định kỹ thuật quan trọng trong từng tầng kiến trúc. Hệ thống đã được kiểm thử nghiêm ngặt với tổng cộng 198 test case tự động (127 test case smart contract và 71 test case backend, tỷ lệ pass 100%), đạt mức bao phủ mã nguồn hơn 93% trên tầng smart contract.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

Traditional financial systems rely on intermediary institutions such as banks to operate lending services, resulting in high operational costs, lack of transparency, and exclusion of a population segment from credit access. Decentralized Finance (DeFi) has emerged to address these limitations by replacing intermediaries with smart contracts on blockchain. Prominent decentralized lending protocols such as Compound and Aave have demonstrated the viability of this model with tens of billions of USD in total value locked. However, the architecture of these protocols is highly complex, with technical documentation scattered and primarily targeting professional developers, creating significant barriers for academic research and learning.

This thesis adopts the approach of building a complete decentralized lending application on the Ethereum platform, drawing inspiration from the architectures of Compound v2 and Aave v2 while focusing on designing, analyzing, and implementing each component from scratch to elucidate the technical essence of each element rather than reusing existing codebases.

The system is designed with a three-layer architecture. The smart contract layer comprises seven Solidity contracts implementing a Lazy Accrual mechanism combined with a Global Interest Index, a five-layer defense-in-depth security architecture, and the UUPS Proxy pattern. The backend layer employs a multi-process event-driven architecture with five independent workers and RabbitMQ ensuring at-least-once event delivery. The frontend layer is built with Next.js, authenticates users via the EIP-4361 standard (Sign-In with Ethereum), and delivers real-time liquidation data updates through WebSocket.

The primary contribution of this thesis is the successful development of a decentralized lending system deployed on the Sepolia testnet and clearly demonstrating the design process and critical technical decisions across each architectural layer. The system was rigorously validated with a total of 198 automated test cases (127 smart contract tests and 71 backend tests, 100% pass rate), achieving over 93% statement code coverage on the smart contract layer.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	3
CHƯƠNG 2. PHÂN TÍCH HỆ THỐNG	5
2.1 Khảo sát hiện trạng	5
2.1.1 Tổng quan các giao thức DeFi lending hiện có	5
2.1.2 Phân tích ưu và nhược điểm	6
2.1.3 Định vị của đồ án.....	7
2.2 Tổng quan chức năng	7
2.2.1 Biểu đồ usecase tổng quát.....	8
2.2.2 Biểu đồ usecase phân rã.....	9
2.3 Đặc tả chức năng	10
2.3.1 Kết nối ví	10
2.3.2 Đăng nhập (Xác thực địa chỉ ví)	12
2.3.3 Gửi tài sản.....	14
2.3.4 Thanh lý khoản vay	15
2.3.5 Đăng ký email	17
2.3.6 Hỗ trợ tài sản mới	19
2.4 Yêu cầu phi chức năng	21
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	23
3.1 Blockchain và Ethereum.....	23
3.1.1 Mô hình blockchain và cơ chế đồng thuận	23

3.1.2 Ethereum Virtual Machine (EVM) và Gas	24
3.1.3 Smart contract và tính bất biến	24
3.1.4 Chuẩn token ERC-20.....	25
3.2 DeFi Lending — Mô hình và cơ chế kinh tế	25
3.2.1 Mô hình pool thanh khoản chung.....	25
3.2.2 Overcollateralization và lý do tại sao.....	26
3.2.3 Health Factor và cơ chế thanh lý	26
3.2.4 Utilization Rate và cơ chế điều chỉnh lãi suất	27
3.3 Mô hình lãi suất Two-Slope (Kinked Rate Model)	27
3.3.1 Lý thuyết kinh tế của mô hình	27
3.3.2 Reserve Factor — Cơ chế doanh thu giao thức	28
3.4 UUPS Upgradeable Proxy Pattern	28
3.4.1 Vấn đề và nhu cầu proxy	28
3.4.2 Cơ chế delegatecall	28
3.4.3 So sánh ba loại proxy phổ biến	29
3.4.4 Storage Layout và rủi ro xung đột slot	29
3.5 Oracle và vấn đề giá tài sản on-chain	30
3.5.1 Oracle Problem trong blockchain.....	30
3.5.2 Chainlink Data Feed — Giải pháp oracle phi tập trung	30
3.5.3 Stale Price — Rủi ro giá cũ	30
3.6 Message Queue và kiến trúc Event-Driven	30
3.6.1 Message Broker và Producer-Consumer Pattern	30
3.6.2 Các đảm bảo delivery và trade-off	31
3.6.3 Idempotency — Xử lý hệ quả của at-least-once.....	31
3.7 Sign-In with Ethereum (EIP-4361)	31
3.7.1 Vấn đề xác thực trong ứng dụng phi tập trung	31

3.7.2 Chuẩn EIP-4361 và cơ chế hoạt động	32
3.8 Lý do lựa chọn công nghệ.....	32
CHƯƠNG 4. THIẾT KẾ HỆ THỐNG.....	34
4.1 Tổng quan kiến trúc hệ thống.....	34
4.2 Thiết kế hợp đồng thông minh (Smart contract).....	35
4.2.1 LendingPool / LendingPoolStorage.....	37
4.2.2 InterestRateModel.....	46
4.2.3 Liquidation.....	48
4.2.4 PriceRouter / PriceRouterStorage.....	50
4.2.5 Lớp quản trị: ProtocolController + ProtocolTimelock.....	52
4.3 Thiết kế cơ sở dữ liệu	54
4.3.1 Nhóm 1: Dữ liệu nghiệp vụ giao thức (Protocol Business Data).....	55
4.3.2 Nhóm 2: Dữ liệu log và snapshot (Historical & Time-series Data) ..	55
4.3.3 Nhóm 3: Dữ liệu quản trị (Governance Data).....	56
4.3.4 Nhóm 4: Dữ liệu hạ tầng hệ thống (Infrastructure State)	56
4.4 Thiết kế backend.....	58
4.4.1 Tổng quan kiến trúc — Mô hình đa tiến trình Event-Driven	58
4.4.2 Blockchain indexer (blc-indexer)	59
4.4.3 Cơ chế quét và phát hiện vị thế thanh lý	60
4.4.4 Tiến trình Blockchain Worker (blc-worker)	61
4.4.5 Tiến trình Cron (croner) và Snapshot hằng ngày.....	61
4.4.6 Tiến trình Notification Worker (noti-worker)	61
4.4.7 HTTP Server — REST API và WebSocket	62
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	64
5.1 Cơ chế tích lũy lãi Lazy Accrual + Global Interest Index	64
5.2 Kiến trúc bảo mật 5 tầng của LendingPool.....	65

5.3 Kiến trúc backend đa tiến trình event-driven	66
5.4 Thiết kế blockchain indexer: Polling thay vì Subscribe Event.....	67
5.5 Kiến trúc quản trị 2 tầng: Multisig và Timelock.....	69
CHƯƠNG 6. CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG	71
6.1 Hợp đồng thông minh	71
6.1.1 Công nghệ sử dụng.....	71
6.1.2 Cấu trúc codebase	71
6.1.3 Cài đặt và các quyết định kỹ thuật quan trọng	72
6.1.4 Quá trình triển khai	73
6.2 Backend.....	75
6.2.1 Công nghệ sử dụng.....	75
6.2.2 Cấu trúc codebase	76
6.2.3 Cài đặt và các quyết định kỹ thuật quan trọng	77
6.2.4 Quá trình triển khai	79
6.3 Frontend	79
6.3.1 Công nghệ sử dụng.....	79
6.3.2 Cấu trúc codebase	80
6.3.3 Cài đặt và các quyết định kỹ thuật quan trọng	81
6.3.4 Quá trình triển khai	82
6.3.5 Giao diện Demo.....	82
CHƯƠNG 7. KIỂM THỬ HỆ THỐNG.....	87
7.1 Kiểm thử smart contract	87
7.1.1 Công cụ và phương pháp kiểm thử.....	87
7.1.2 Phạm vi và cấu trúc kiểm thử	87
7.1.3 Chi tiết các nhóm kiểm thử	88
7.1.4 Kết quả Code Coverage	91

7.1.5 Kết quả Gas Report	92
7.2 Kiểm thử Backend	93
7.2.1 Công cụ và phương pháp kiểm thử.....	93
7.2.2 Phạm vi và cấu trúc kiểm thử	94
7.2.3 Chi tiết các nhóm kiểm thử quan trọng	95
7.2.4 Kết quả kiểm thử.....	96
7.3 Kiểm thử Frontend.....	96
CHƯƠNG 8. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	97
8.1 Kết luận	97
8.2 Hướng phát triển tiếp theo	100
TÀI LIỆU THAM KHẢO.....	102

DANH MỤC HÌNH VẼ

Hình 2.1	Sơ đồ usecase tổng quát	9
Hình 2.2	Sơ đồ usecase phân rã quản lý tài sản	10
Hình 2.3	Đặc tả usecase Kết nối ví	11
Hình 2.4	Sơ đồ tuần tự Kết nối ví	12
Hình 2.5	Đặc tả usecase Đăng nhập (Xác thực địa chỉ ví)	13
Hình 2.6	Sơ đồ tuần tự Đăng nhập (Xác thực địa chỉ ví)	13
Hình 2.7	Đặc tả usecase Gửi tài sản	14
Hình 2.8	Sơ đồ tuần tự Gửi tài sản	15
Hình 2.9	Đặc tả usecase Thanh lý khoản vay	16
Hình 2.10	Sơ đồ tuần tự Thanh lý khoản vay	17
Hình 2.11	Đặc tả usecase Đăng ký email	18
Hình 2.12	Sơ đồ tuần tự Đăng ký email	19
Hình 2.13	Đặc tả usecase Hỗ trợ tài sản mới	20
Hình 2.14	Sơ đồ tuần tự Hỗ trợ tài sản mới	21
Hình 4.1	Sơ đồ kiến trúc tổng quan hệ thống	35
Hình 4.2	Tổng quan kiến trúc smart contract	37
Hình 4.3	Sơ đồ luồng gửi tài sản	40
Hình 4.4	Sơ đồ luồng vay tài sản	41
Hình 4.5	Sơ đồ luồng trả nợ	41
Hình 4.6	Sơ đồ luồng rút tài sản	42
Hình 4.7	Sơ đồ luồng thanh lý	49
Hình 4.8	Sơ đồ ERD của hệ thống	57
Hình 4.9	Sơ đồ tổng quan hệ thống backend	59
Hình 6.1	Trang chủ	82
Hình 6.2	Pop up gửi tài sản	83
Hình 6.3	Pop up rút tài sản	83
Hình 6.4	Pop up vay tài sản	84
Hình 6.5	Pop up trả nợ tài sản	84
Hình 6.6	Trang dashboard cá nhân	84
Hình 6.7	Trang lịch sử giao dịch cá nhân	85
Hình 6.8	Trang admin panel	85
Hình 6.9	Admin tương tác với multisig	86
Hình 6.10	Hàng đợi timelock và danh sách đề nghị	86
Hình 6.11	Trang đề nghị thực thi	86

Hình 7.1	Kết quả chạy lệnh kiểm tra coverage	92
Hình 7.2	Kết quả chạy lệnh kiểm thử backend	94

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các giao thức tham chiếu và giao thức đề án	6
Bảng 3.1	So sánh 3 loại proxy phổ biến	29
Bảng 3.2	So sánh các đảm bảo delivery	31
Bảng 3.3	Các công nghệ được sử dụng	33
Bảng 4.1	Struct Market của LendingPool	38
Bảng 4.2	Struct Balance của LendingPool	38
Bảng 4.3	Cấu trúc dữ liệu tiêu biểu	38
Bảng 4.4	Các hàm nghiệp vụ chính	40
Bảng 4.5	Các modifier phân quyền truy cập	43
Bảng 4.6	Các hàm preview của giao thức	45
Bảng 4.7	Các tham số mô hình lãi suất	47
Bảng 4.8	Các tham số rủi ro	48
Bảng 4.9	Các hàm quản lý nguồn giá	52
Bảng 4.10	Các hàm quản trị admin	53
Bảng 4.11	Các nhóm endpoint của hệ thống	62
Bảng 5.1	So sánh hai hướng tiếp cận index blockchain	69
Bảng 6.1	Khởi động các tiến trình	79
Bảng 7.1	Kiểm thử tự động smart contract	88
Bảng 7.2	Coverage kiểm thử smart contract	91
Bảng 7.3	Gas report kiểm thử smart contract các hàm chính	93
Bảng 7.4	Gas report kiểm thử smart contract deploy	93
Bảng 7.5	Kiểm thử tự động backend	94
Bảng 8.1	Mục tiêu và kết quả của đề án	98
Bảng 8.2	Thông số kiểm thử smart contract	99

DANH MỤC THUẬT NGỮ

Thuật ngữ	Ý nghĩa
At-least-once delivery	Cơ chế đảm bảo message được chuyển tới ít nhất một lần, message có thể bị gửi lại, đòi hỏi consumer phải xử lý idempotent
aToken	Token đại diện cho vị thế gửi tiền trong Aave, số dư tăng trực tiếp theo thời gian khi lãi tích lũy
Block	Đơn vị dữ liệu cơ bản trên blockchain, chứa tập hợp các giao dịch được xác nhận
Blockchain	Chuỗi khối, cơ sở dữ liệu phân tán lưu trữ dữ liệu theo chuỗi các block liên kết bằng mật mã
Close factor	Tỷ lệ tối đa phần nợ của người vay có thể bị thanh lý trong một lần
Collateral	Tài sản thế chấp, tài sản người dùng khóa vào giao thức để bảo đảm cho khoản vay
cToken	Token đại diện cho vị thế gửi tiền trong Compound, tỷ giá quy đổi với tài sản gốc tăng dần theo lãi
Defense-in-depth	Kiến trúc bảo mật nhiều lớp, nếu một lớp bị phá vỡ thì các lớp còn lại vẫn bảo vệ hệ thống
DelegateCall	Cơ chế trong EVM cho phép contract A thực thi code của contract B nhưng trong context storage của A, nền tảng của Proxy Pattern
Event (Blockchain)	Bản ghi log được emit bởi smart contract sau khi thực thi một hàm, lưu trên blockchain nhưng không truy cập được từ contract khác

Thuật ngữ	Ý nghĩa
Flash loan	Cơ chế vay không cần thế chấp với điều kiện hoàn trả trong cùng một giao dịch
Gas	Đơn vị đo lường chi phí tính toán trong EVM, người dùng trả gas fee bằng ETH để thực hiện giao dịch
Global Interest Index	Chỉ số tích lũy lãi suất toàn cục, tăng theo thời gian, cho phép tính lãi bất kỳ tài khoản nào với chi phí $O(1)$
Health factor	Hệ số sức khỏe, tỷ lệ giữa giá trị thế chấp đã điều chỉnh và giá trị nợ; khi nhỏ hơn 1 thì vị thế có thể bị thanh lý
Idempotency	Tính lũy đẳng, một thao tác được thực hiện nhiều lần cho kết quả giống như thực hiện một lần
Implementation contract	Contract chứa logic nghiệp vụ trong mẫu Proxy, được gọi thông qua delegatecall từ proxy
Lazy Accrual	Cơ chế tích lũy lãi lười, không cập nhật lãi theo từng giây mà chỉ tính khi có tương tác với tài khoản, sử dụng Global Index để đảm bảo chính xác
Liquidation	Thanh lý, quá trình một bên thứ ba trả nợ thay cho người vay và nhận lại tài sản thế chấp cộng thêm phần thưởng
Liquidation threshold	Ngưỡng thanh lý, giá trị health factor mà dưới ngưỡng này vị thế có thể bị thanh lý
Liquidation incentive	Phần thưởng thanh lý, phần trăm thêm vào tài sản thế chấp mà liquidator nhận được, tạo động lực kinh tế
Mainnet	Mạng chính thức của Ethereum, nơi các giao dịch có giá trị thực

Thuật ngữ	Ý nghĩa
Multisig wallet	Ví đa chữ ký, ví yêu cầu chữ ký của nhiều thành viên để thực thi giao dịch
Nonce	Số ngẫu nhiên dùng một lần, trong xác thực SIWE để ngăn replay attack
Off-chain	Dữ liệu hoặc xử lý diễn ra bên ngoài blockchain
On-chain	Dữ liệu hoặc logic được lưu trực tiếp trên blockchain
Oracle	Hệ thống cung cấp dữ liệu thực tế vào trong smart contract trên blockchain, ví dụ giá tài sản hoặc tỷ giá
Overcollateralization	Thế chấp quá mức, người vay phải ký gửi tài sản có giá trị lớn hơn khoản vay muốn nhận
Proof of Stake (PoS)	Cơ chế đồng thuận dựa trên cổ phần, validator đặt cược ETH để tham gia xác nhận block
Proxy contract	Contract trung gian chuyển tiếp lời gọi đến implementation contract, lưu trữ toàn bộ dữ liệu
Pub/Sub	Publish-Subscribe, mô hình giao tiếp trong đó producer publish sự kiện và consumer đăng ký nhận
Reserve factor	Tỷ lệ dự trữ, phần trăm lãi suất vay được giữ lại trong treasury của giao thức
Sepolia	Mạng thử nghiệm testnet của Ethereum dành cho developer
SIWE	Sign-In with Ethereum (EIP-4361), chuẩn xác thực người dùng bằng chữ ký ví thay vì username/password

Thuật ngữ	Ý nghĩa
Smart contract	Hợp đồng thông minh, chương trình tự thực thi được lưu trên blockchain, không thể thay đổi sau khi triển khai trừ khi dùng proxy
Stablecoin	Tiền mã hóa ổn định giá, thường neo 1:1 với USD
Storage gap	Kỹ thuật dự trữ storage slot trong contract upgradeable để tránh xung đột slot khi thêm biến mới sau upgrade
Testnet	Mạng thử nghiệm của blockchain, dùng để phát triển và kiểm thử, token không có giá trị thực
Timelock	Hợp đồng buộc các hành động quản trị phải chờ một khoảng thời gian cố định trước khi thực thi
Transaction	Giao dịch, hành động trên blockchain do người dùng ký, thay đổi trạng thái ví dụ gửi ETH hoặc gọi hàm contract
Two-Slope Model	Mô hình lãi suất hai độ dốc, lãi suất tăng tuyến tính dưới ngưỡng utilization tối ưu và tăng dốc đứng hơn khi vượt ngưỡng
Utilization rate	Tỷ lệ sử dụng vốn = $\text{totalBorrows} / \text{totalDeposits}$, chỉ số quyết định lãi suất trong pool
UUPS Proxy	Loại proxy nâng cấp được trong đó logic upgrade nằm trong implementation contract, tiết kiệm gas hơn Transparent Proxy
View function	Hàm smart contract chỉ đọc dữ liệu, không ghi blockchain, không tốn gas khi gọi từ off-chain

Thuật ngữ	Ý nghĩa
Worker	Tiến trình nền độc lập, trong kiến trúc backend của đồ án, mỗi worker đảm nhận một nhiệm vụ cụ thể

DANH MỤC TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
ABI	Application Binary Interface — Giao diện nhị phân ứng dụng, định nghĩa cách gọi hàm và giải mã dữ liệu trả về của smart contract
API	Application Programming Interface — Giao diện lập trình ứng dụng
DAO	Decentralized Autonomous Organization — Tổ chức tự trị phi tập trung
DeFi	Decentralized Finance — Tài chính phi tập trung
EIP	Ethereum Improvement Proposal — Đề xuất cải tiến Ethereum
ERC	Ethereum Request for Comment — Đề xuất tiêu chuẩn kỹ thuật trên Ethereum
ERD	Entity-Relationship Diagram — Sơ đồ thực thể - quan hệ
EVM	Ethereum Virtual Machine — Máy ảo Ethereum, môi trường thực thi smart contract
JWT	JSON Web Token — Chuẩn token xác thực theo định dạng JSON
MQ	Message Queue — Hàng đợi tin nhắn
ORM	Object-Relational Mapping — Ánh xạ đối tượng - quan hệ
REST	Representational State Transfer — Kiến trúc API theo phong cách biểu diễn trạng thái
RPC	Remote Procedure Call — Giao thức gọi thủ tục từ xa, dùng để giao tiếp với Ethereum node

Thuật ngữ	Ý nghĩa
SMTP	Simple Mail Transfer Protocol — Giao thức truyền thư điện tử đơn giản
SSD	System Sequence Diagram — Sơ đồ tuần tự hệ thống
SSR	Server-Side Rendering — Kết xuất phía máy chủ
TTL	Time To Live — Thời gian sống của dữ liệu trong cache/Redis
TVL	Total Value Locked — Tổng giá trị tài sản bị khóa trong giao thức DeFi
TWAP	Time-Weighted Average Price — Giá trung bình theo thời gian
USDC	USD Coin — Stablecoin được neo giá 1:1 với USD
UUPS	Universal Upgradeable Proxy Standard — Chuẩn proxy có thể nâng cấp theo kiểu phổ quát, EIP-1822
WBTC	Wrapped Bitcoin — Bitcoin được bọc dưới dạng ERC20 token trên Ethereum
WebSocket	Giao thức kết nối hai chiều liên tục giữa client và server
WETH	Wrapped Ether — Ether được bọc dưới dạng ERC20 token

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong hệ thống tài chính truyền thống, các hoạt động vay và cho vay được thực hiện thông qua các tổ chức trung gian như ngân hàng thương mại hoặc công ty tài chính. Mô hình này đặt ra một số hạn chế cố hữu: người dùng phải tin tưởng và phụ thuộc hoàn toàn vào bên thứ ba trong việc quản lý tài sản; quy trình phê duyệt khoản vay đòi hỏi thủ tục phức tạp, tốn thời gian và thường yêu cầu hồ sơ tín dụng; dữ liệu giao dịch và chính sách lãi suất thiếu minh bạch, người dùng không có khả năng kiểm chứng độc lập; ngoài ra, một bộ phận lớn dân số toàn cầu không có khả năng tiếp cận dịch vụ ngân hàng do rào cản địa lý, pháp lý hoặc kinh tế.

Sự ra đời của công nghệ blockchain, đặc biệt là nền tảng Ethereum với khả năng thực thi hợp đồng thông minh (smart contract), đã mở ra một hướng tiếp cận mới nhằm giải quyết những hạn chế trên [1], [2]. Tài chính phi tập trung (Decentralized Finance — DeFi) là hệ sinh thái các ứng dụng tài chính được xây dựng trên nền tảng blockchain, cho phép các giao dịch tài chính diễn ra trực tiếp giữa các bên mà không cần trung gian, với toàn bộ logic nghiệp vụ được mã hóa và thực thi tự động trên chuỗi [3]. Các nghiên cứu tổng quan ghi nhận TVL của các giao thức DeFi tăng nhanh trong giai đoạn bùng nổ, phản ánh nhu cầu thực tế và quy mô đáng kể của lĩnh vực này [3].

Trong hệ sinh thái DeFi, giao thức vay và cho vay phi tập trung (decentralized lending protocol) đóng vai trò nền tảng. Các giao thức tiêu biểu như Aave và Compound đã chứng minh tính khả thi của mô hình [4]–[6]: người dùng có thể gửi tài sản kỹ thuật số để nhận lãi suất, hoặc vay tài sản dựa trên cơ chế thế chấp quá mức (overcollateralization) mà không cần thông qua bất kỳ tổ chức nào. Tuy nhiên, các giao thức thực tế này có độ phức tạp rất cao và tài liệu kỹ thuật đầy đủ về cách xây dựng một giao thức tương tự từ đầu còn hạn chế, tạo ra khoảng cách lớn giữa người học và thực tiễn ngành.

Xuất phát từ bối cảnh đó, đề án này đặt ra bài toán thiết kế, xây dựng và triển khai một ứng dụng vay và cho vay phi tập trung hoàn chỉnh trên nền tảng Ethereum. Bài toán không chỉ dừng lại ở việc xây dựng smart contract mà còn bao gồm toàn bộ hệ thống xung quanh — từ cơ chế tích lũy lãi suất trên chuỗi, quản lý rủi ro và thanh lý vị thế, đến hệ thống backend index và lưu trữ event on-chain thành dạng có thể query được, tổng hợp dữ liệu người dùng và hỗ trợ xử lý dữ liệu off-chain kết hợp giao diện người dùng tương tác thời gian thực.

1.2 Mục tiêu và phạm vi đề tài

Các giao thức DeFi lending thực tế hiện nay có thể chia thành hai thế hệ. Thế hệ đầu, tiêu biểu là Compound v2 và Aave v2, sử dụng mô hình pool thanh khoản chung và hỗ trợ nhiều loại tài sản. Thế hệ thứ hai như Aave v3 và Compound v3 bổ sung các tính năng nâng cao hơn như cô lập rủi ro theo tài sản, tối ưu hóa vốn đa chuỗi và cơ chế quản trị DAO [4], [5], [7], [8]. Cả hai thế hệ đều đã được kiểm toán bảo mật nghiêm ngặt và triển khai với hàng tỷ USD tài sản. Tuy nhiên, do độ phức tạp cao và quy mô lớn, các hệ thống này không phù hợp làm điểm xuất phát để nghiên cứu và nắm bắt toàn bộ từ đầu.

Dựa trên phân tích trên, đề án này hướng tới xây dựng một giao thức lending phi tập trung ở quy mô đủ để hiểu và triển khai hoàn chỉnh, đồng thời áp dụng đúng các nguyên tắc thiết kế của thế hệ đầu tiên. Mục tiêu cụ thể bao gồm bốn nhóm chính.

Nhóm thứ nhất là xây dựng lớp smart contract cốt lõi, bao gồm các chức năng gửi tài sản để nhận lãi, vay tài sản dựa trên thế chấp, trả nợ và rút tài sản; cơ chế tích lũy lãi suất theo mô hình Two-Slope (Kinked Rate Model); cơ chế thanh lý vị thế khi hệ số sức khỏe giảm xuống dưới ngưỡng an toàn; và lớp quản trị bao gồm ví đa chữ ký kết hợp với timelock để đảm bảo tính minh bạch, cùng với đó là cơ chế quản lý ngân quỹ (treasury) để trích xuất lợi nhuận giao thức và hợp đồng thông minh có khả năng nâng cấp toàn diện.

Nhóm thứ hai là xây dựng hệ thống backend hỗ trợ theo kiến trúc event-driven, bao gồm lắng nghe blockchain để lưu trữ lịch sử giao dịch của người dùng và lịch sử quản trị của giao thức, tổng hợp dữ liệu tài sản vay, gửi, trả, rút của người dùng, giúp người dùng theo dõi được quá trình tương tác với giao thức, snapshot dữ liệu định kỳ phục vụ hiển thị lịch sử và hệ thống thông báo cho người dùng.

Nhóm thứ ba là xây dựng giao diện người dùng cho phép kết nối ví, thực hiện các giao dịch tài chính, theo dõi vị thế cá nhân theo thời gian thực và xem lịch sử hoạt động trong giao thức.

Nhóm thứ tư là kiểm thử và triển khai hệ thống trên mạng thử nghiệm Sepolia của Ethereum.

Về phạm vi, đề án tập trung vào các nội dung trên và xác định rõ các tính năng nằm ngoài phạm vi thực hiện, bao gồm: hỗ trợ đa chuỗi, cơ chế quản trị DAO phi tập trung, flash loan, tối ưu chi phí gas toàn giao thức và kiểm toán bảo mật chuyên sâu. Những tính năng này được ghi nhận là hướng phát triển tiếp theo trong Chương 8.

1.3 Định hướng giải pháp

Từ các yêu cầu đã xác định ở mục 1.2, đề án lựa chọn hướng tiếp cận và công nghệ theo ba tầng kiến trúc tách biệt.

Tầng smart contract triển khai trên Ethereum — nền tảng blockchain có hệ sinh thái DeFi trưởng thành nhất, cho phép tham chiếu trực tiếp từ các giao thức đã được kiểm chứng thực tế như Aave và Compound. Để giải quyết bài toán nâng cấp smart contract mà không làm gián đoạn hoạt động hay mất dữ liệu người dùng, các smart contract cốt lõi áp dụng mẫu UUPS Proxy [9], [10].

Tầng backend sử dụng kiến trúc đa tiến trình hướng sự kiện thay vì monolith. Lựa chọn này xuất phát từ bản chất của dữ liệu blockchain: mọi thay đổi trạng thái đều được biểu diễn dưới dạng event liên tục, đòi hỏi từng thành phần phải xử lý và có thể mở rộng độc lập. Hệ thống lựa chọn tự xây blockchain indexer thay vì dùng The Graph hay Moralis, để giữ toàn quyền kiểm soát dữ liệu, tránh phụ thuộc vào bên thứ ba, và đơn giản hóa stack trong phạm vi đề án.

Tầng frontend sử dụng Next.js kết hợp ethers.js v6 và chuẩn EIP-4361 để xác thực người dùng trực tiếp qua ví Ethereum, không cần hệ thống tài khoản truyền thống [11], [12].

1.4 Bố cục đề án

Phần còn lại của báo cáo được tổ chức như sau.

Chương 2 trình bày quá trình khảo sát và phân tích yêu cầu hệ thống. Chương này bắt đầu bằng việc so sánh các giao thức DeFi lending hiện có để xác định vị trí của đề án trong bức tranh tổng thể, sau đó đi vào phân tích các yêu cầu chức năng thông qua các biểu đồ use case và sơ đồ SSD, cuối cùng là trình bày các yêu cầu phi chức năng của hệ thống.

Chương 3 giới thiệu nền tảng lý thuyết và công nghệ sử dụng trong đề án, bao gồm các khái niệm cốt lõi của DeFi, cơ chế hoạt động của Ethereum và smart contract, mô hình lãi suất, cũng như các công nghệ backend và frontend được áp dụng. Với mỗi công nghệ được trình bày, chương này phân tích lý do lựa chọn so với các giải pháp thay thế.

Chương 4 đi vào trình bày thiết kế chi tiết tầng smart contract và backend. Phần smart contract mô tả kiến trúc và logic nghiệp vụ của từng smart contract. Phần backend trình bày thiết kế cơ sở dữ liệu, kiến trúc đa tiến trình và các cơ chế xử lý đặc thù.

Chương 5 tập trung trình bày các giải pháp và đóng góp nổi bật của đề án, bao gồm cơ chế tích lũy lãi suất Lazy Accrual với Global Interest Index, thiết kế bảo

mật theo chiều sâu cho smart contract, các giải pháp đảm bảo tính nhất quán dữ liệu trong hệ thống phân tán và các chiến lược lắng nghe blockchain đảm bảo toàn vẹn dữ liệu cùng với giải pháp quản trị qua hai tầng tách biệt.

Chương 6 tập trung vào quá trình cài đặt, triển khai hệ thống và các quyết định kỹ thuật quan trọng.

Chương 7 trình bày kết quả kiểm thử hệ thống, bao gồm kiểm thử smart contract, backend và frontend, kèm theo số liệu và đánh giá cụ thể.

Chương 8 đưa ra kết luận về kết quả đạt được, so sánh với các giao thức tham chiếu, và trình bày các hướng phát triển tiếp theo.

CHƯƠNG 2. PHÂN TÍCH HỆ THỐNG

2.1 Khảo sát hiện trạng

2.1.1 Tổng quan các giao thức DeFi lending hiện có

Trong hệ sinh thái DeFi, hai giao thức vay và cho vay phi tập trung có ảnh hưởng lớn và được nghiên cứu rộng rãi nhất là Compound và Aave. Cả hai đều sử dụng mô hình pool thanh khoản chung, trong đó người dùng gửi tài sản vào một pool được quản lý bởi smart contract để nhận lãi suất, và người vay có thể rút tài sản từ pool với điều kiện thế chấp tài sản khác có giá trị vượt mức khoản vay. Lãi suất trong các giao thức này được điều chỉnh tự động dựa trên tỷ lệ sử dụng vốn của từng pool: khi tỷ lệ sử dụng cao, lãi suất tăng để thu hút thêm người gửi và khuyến khích người vay trả nợ; khi tỷ lệ sử dụng thấp, lãi suất giảm để kích thích vay.

Compound ra mắt năm 2018 và là giao thức đặt nền móng cho toàn bộ lĩnh vực DeFi lending. Compound giới thiệu khái niệm lãi suất thuật toán và vị thế gửi tiền được token hóa (cToken), trở thành chuẩn mực mà Aave và các giao thức khác xây dựng theo. Về mặt kỹ thuật, khi người dùng gửi tài sản, họ nhận lại cToken tương ứng (ví dụ cUSDC). Số dư cToken không thay đổi theo thời gian, thay vào đó tỷ giá chuyển đổi giữa cToken và tài sản gốc tăng dần khi lãi tích lũy. Cơ chế kiểm soát rủi ro tập trung vào smart contract Comptroller — đây là bộ máy quản lý rủi ro trung tâm, mọi hành động gửi, vay, rút đều phải thông qua Comptroller để kiểm tra mức thế chấp và tình trạng nợ. Compound v2 sử dụng Comptroller chung để quản lý rủi ro cho tất cả tài sản, dẫn đến rủi ro lan truyền ở tầng quản trị thế chấp: nếu một tài sản rủi ro cao bị định giá sai hoặc bị khai thác, toàn bộ hệ thống có thể bị ảnh hưởng. Compound v3 (Comet) giải quyết vấn đề này bằng cách tách thành các market độc lập, mỗi market chỉ cho phép vay một loại tài sản duy nhất trong khi vẫn chấp nhận nhiều loại collateral khác nhau, đánh đổi tính linh hoạt lấy sự an toàn và cô lập rủi ro. [5], [6], [8].

Aave ra mắt công khai năm 2020 và phát triển qua hai thế hệ chính. Aave v2 tập trung tối ưu hóa mô hình thế chấp và lãi suất ổn định. Aave v3 bổ sung các tính năng nâng cao về hiệu quả vốn và khả năng đa chuỗi: Isolation Mode giới hạn các tài sản rủi ro cao vào các pool độc lập để ngăn ảnh hưởng hệ thống; E-Mode cho phép hệ số thế chấp rất cao với các tài sản tương quan chặt; Portal cho phép dịch chuyển thanh khoản giữa các chuỗi. Về cơ chế ghi nhận vị thế, Aave sử dụng aToken — token mang lãi suất được mint 1:1 với tài sản gửi và số dư aToken tăng trực tiếp theo thời gian khi lãi tích lũy, khác với cToken của Compound dùng tỷ giá quy đổi. Aave còn tiên phong với Flash Loan — cơ chế vay không cần thế chấp với

điều kiện hoàn trả trong cùng một giao dịch, mở ra khả năng arbitrage, hoán đổi thể chấp và thanh lý phức tạp mà Compound không hỗ trợ [4], [7].

Về mô hình lãi suất, cả Aave và Compound v2 đều áp dụng mô hình Two-Slope (còn gọi là Kinked Rate Model): lãi suất vay tăng tuyến tính theo tỷ lệ sử dụng vốn khi dưới ngưỡng tối ưu (U_{optimal}), và tăng với độ dốc lớn hơn nhiều khi vượt ngưỡng này. Hai giao thức có cùng cơ chế kinh tế nhưng khác nhau về chi tiết triển khai và các tham số cấu hình [4], [5].

2.1.2 Phân tích ưu và nhược điểm

Bảng 2.1 dưới đây tổng hợp so sánh các đặc điểm kỹ thuật chính của hai giao thức tham chiếu và giao thức được xây dựng trong đề án này:

Tiêu chí	Compound v2	Aave v2/v3	Giao thức đề án
Mô hình pool	Pool chung đa tài sản	Pool chung đa tài sản (v2), Isolated (v3)	Pool chung đa tài sản
Ghi nhận vị thế gửi	cToken (tỷ giá quy đổi tăng dần)	aToken (số dư tăng trực tiếp)	Internal (ghi nhận bên trong smart contract)
Mô hình lãi suất	Two-Slope (JumpRateModel)	Two-Slope (Default-Reserve InterestRateStrategy)	Two-Slope
Cơ chế thanh lý	Close factor + liquidation incentive	Close factor + liquidation bonus	Close factor + liquidation incentive
Oracle giá	Chainlink + Uniswap TWAP	Chainlink + dự phòng	Chainlink + MyOracle (môi trường kiểm thử)
Nâng cấp smart contract	Proxy pattern	UUPS/Transparent Proxy	UUPS
Quản trị	COMP token + Timelock	AAVE token + Timelock	Multisig + ProtocolTimelock
Flash Loan	Không hỗ trợ	Hỗ trợ	Ngoài phạm vi
Đa chuỗi	Hạn chế	Hỗ trợ 14+ chuỗi	Ngoài phạm vi
Hệ thống backend	Outsource indexer (The Graph) + API server	Outsource indexer (The Graph) + API server	Tự xây indexer, event-driven đa tiến trình

Bảng 2.1: So sánh các giao thức tham chiếu và giao thức đề án

Nhận xét về ưu điểm của các giao thức tham chiếu: Compound v2 nổi bật bởi sự đơn giản và rõ ràng trong thiết kế — mô hình cToken dễ hiểu, kiến trúc Comptroller tập trung hóa quản lý rủi ro, và toàn bộ codebase đã được kiểm toán và kiểm chứng qua nhiều năm vận hành thực tế. Aave v2 cân bằng tốt giữa tính năng phong phú và độ phức tạp có kiểm soát, với hệ thống sự kiện (event) được thiết kế chi tiết cho phép tái tạo trạng thái từ lịch sử. Aave v3 đẩy giới hạn về hiệu quả vốn và khả năng mở rộng đa chuỗi, nhưng đánh đổi bằng độ phức tạp cài đặt tăng đáng kể.

Nhận xét về hạn chế liên quan đến mục tiêu đề án: Trong thực tế, cả Aave và

Compound đều phải dựa vào dịch vụ indexer bên ngoài như The Graph để phục vụ frontend với dữ liệu lịch sử và tổng hợp, tạo ra một điểm phụ thuộc vào bên thứ ba nằm ngoài kiến trúc on-chain. Đây là điểm mà đề án này tiếp cận khác biệt: thay vì outsource indexing layer, hệ thống tự xây toàn bộ backend event-driven để chủ động kiểm soát dữ liệu và không phụ thuộc vào hạ tầng bên ngoài. Ngoài ra, tài liệu hướng dẫn xây dựng một giao thức tương tự từ đầu vẫn còn rời rạc, chưa có nguồn tham chiếu toàn diện nào trình bày đầy đủ cả ba tầng smart contract, backend và frontend trong một hệ thống thống nhất.

2.1.3 Định vị của đề án

Từ quá trình khảo sát trên, đề án này xác định hướng xây dựng một giao thức lending phi tập trung ở quy mô phù hợp để có thể hiểu và triển khai hoàn chỉnh, lấy thiết kế của Compound v2 và Aave v2 làm tham chiếu cho lớp smart contract [4], [5], tự xây toàn bộ indexing layer thay vì outsource ra The Graph, đồng thời tích hợp luôn notification và snapshot vào cùng một hệ thống backend thống nhất. Về cơ chế ghi nhận vị thế, đề án áp dụng Global Interest Index tương tự Compound v2 nhưng lưu trực tiếp trong storage của LendingPool thay vì thông qua cơ chế token trung gian như cToken hay aToken, đơn giản hóa kiến trúc trong khi vẫn đảm bảo tính chính xác về mặt toán học. Về quản trị, đề án áp dụng mô hình Multisig Wallet kết hợp Timelock ở quy mô nhỏ phù hợp với giai đoạn phát triển ban đầu của giao thức. Chi tiết về các quyết định thiết kế này sẽ được trình bày trong Chương 4 và Chương 5.

Giao thức đề án không hướng đến cạnh tranh về quy mô hay tính năng với Compound và Aave. Giá trị khác biệt nằm ở việc xây dựng và phân tích từng thành phần kỹ thuật từ đầu, bao gồm cả tầng backend thường bị bỏ qua trong các tài liệu học thuật về DeFi.

2.2 Tổng quan chức năng

Hệ thống vay và cho vay phi tập trung được thiết kế xoay quanh các tác nhân chính sau:

Người dùng (User) là tác nhân trung tâm của hệ thống. Người dùng tương tác với giao thức thông qua ví điện tử (MetaMask hoặc các ví tương thích EVM), không cần đăng ký tài khoản truyền thống. Các chức năng chính của User bao gồm: (i) Kết nối và ngắt kết nối ví điện tử, (ii) Xác thực địa chỉ và đăng nhập bằng chữ ký điện tử, (iii) Gửi tài sản vào giao thức để nhận lãi, (iv) Vay tài sản dựa trên tài sản thế chấp, (v) Trả nợ vay và rút tài sản đã gửi, (vi) Tham gia thanh lý các vị thế không đảm bảo, (vii) Theo dõi trạng thái vị thế (collateral, debt, health factor), (viii) Theo dõi các thông số tài sản đang có trong giao thức, (ix) Xem lịch sử giao

dịch và các sự kiện liên quan đến tài khoản, (x) Xem các hành động của admin đối với giao thức và (xi) Đăng ký email để nhận thông báo về các thay đổi quan trọng của giao thức.

Điểm quan trọng khác với mô hình tiết kiệm ngân hàng truyền thống, giao thức không ràng buộc người dùng vào kỳ hạn cố định. Người dùng có thể gửi tài sản và rút ra bất kỳ lúc nào, đồng thời vẫn nhận đủ lãi suất tương ứng với thời gian thực tế đã gửi tính đến giây. Tương tự, lãi suất vay và gửi không cố định mà thay đổi liên tục theo tỷ lệ sử dụng vốn của từng tài sản trong giao thức. Hai tính chất này — không kỳ hạn và lãi suất thả nổi — là yêu cầu cốt lõi quyết định toàn bộ cách thiết kế cơ chế tích lũy lãi của hệ thống.

Admin là tác nhân có quyền quản trị hệ thống, kế thừa toàn bộ chức năng của User. Ngoài ra, Admin còn có các quyền đặc biệt nhằm đảm bảo hoạt động ổn định và an toàn của giao thức (tất cả các hành động của admin đều được thực hiện thông qua ví đa chữ ký và timelock), bao gồm: (i) Quản lý danh sách tài sản được hỗ trợ, (ii) Cấu hình các tham số của giao thức (lãi suất, hệ số thể chấp, ngưỡng thanh lý,...), (iii) Tạm dừng hoặc kích hoạt lại giao thức trong trường hợp khẩn cấp, (iv) Quản lý các hợp đồng thông minh liên quan đến giao thức (update địa chỉ, logic (với những upgradeable contract)), (v) Quản lý ngân quỹ và các hoạt động tài chính của giao thức.

Oracle là tác nhân hệ thống bên ngoài, có nhiệm vụ cung cấp dữ liệu giá tài sản on-chain cho giao thức. Trong đề án, Chainlink Data Feed được sử dụng nhằm đảm bảo dữ liệu giá có độ tin cậy cao, chống thao túng và phù hợp với yêu cầu của các ứng dụng DeFi [13].

2.2.1 Biểu đồ usecase tổng quát

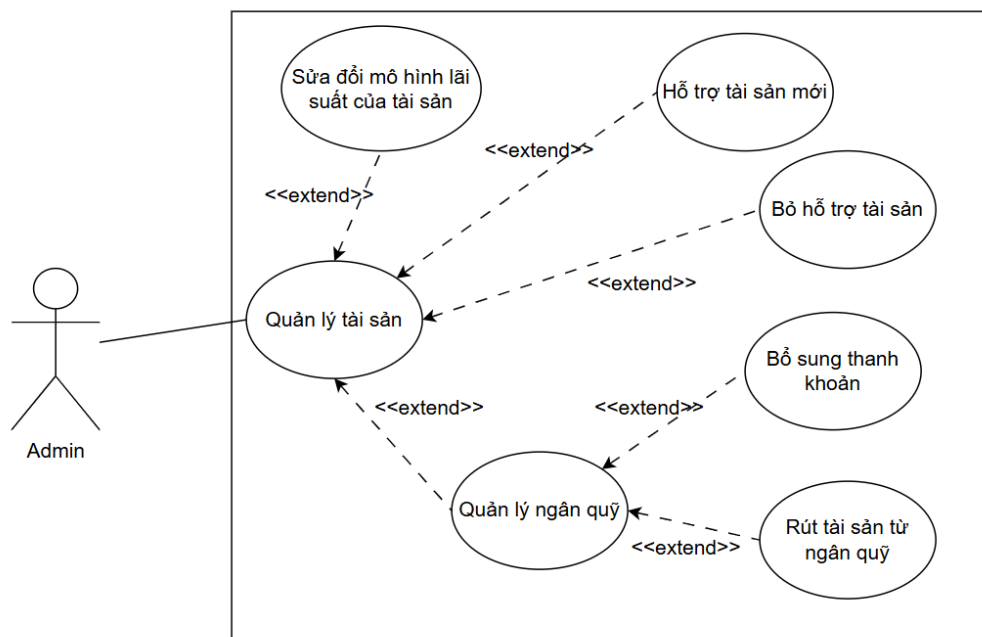
Sơ đồ ca sử dụng tổng quát của hệ thống được thể hiện bởi hình 2.1 dưới đây



Hình 2.1: Sơ đồ usecase tổng quát

2.2.2 Biểu đồ usecase phân rã

Sơ đồ ca sử dụng phân rã quản lý tài sản được thể hiện bằng hình 2.2 dưới đây



Hình 2.2: Sơ đồ usecase phân rã quản lý tài sản

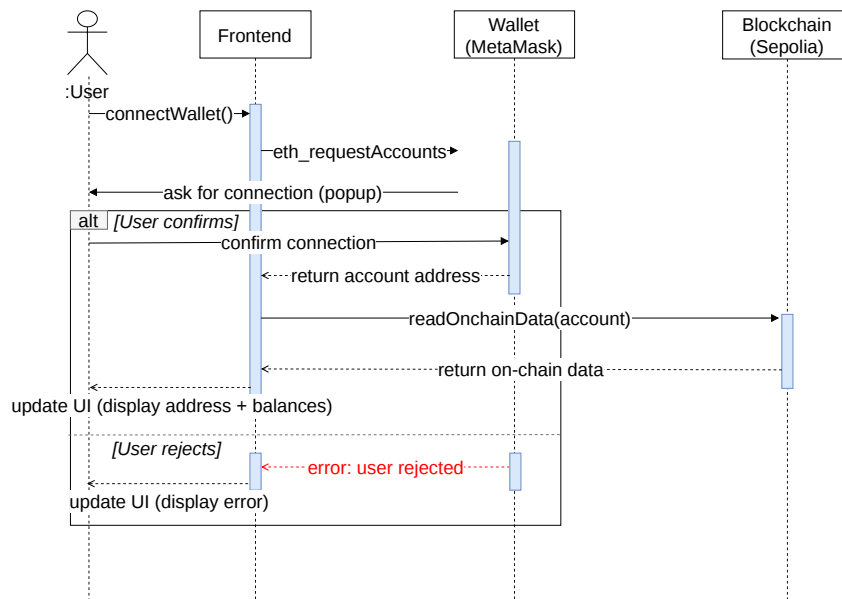
2.3 Đặc tả chức năng

2.3.1 Kết nối ví

Đặc tả usecase kết nối ví được thể hiện bằng hình 2.3, sơ đồ tuần tự được thể hiện trong hình 2.4

Tên ca sử dụng: Kết nối ví		ID: 1
Tác nhân chính	Người dùng	
Tác nhân phụ	Không có	
Mô tả ngắn gọn	Ca sử dụng này miêu tả quá trình người dùng kết nối với ví tiền điện tử	
Tiền điều kiện	Người dùng chưa kết nối ví	
Luồng sự kiện thông thường	1. Người dùng nhấn nút chọn kết nối với ví. 2. Cửa sổ chọn ví hiện lên. 3. Người dùng chọn ví muốn kết nối (VD: MetaMask). 4. Cửa sổ MetaMask xuất hiện xác nhận kết nối. 5. Người dùng xác nhận kết nối trên ví. 6. Hệ thống hiển thị địa chỉ ví trên giao diện.	
Luồng sự kiện ngoại lệ	4a. Người dùng không có ví tiền điện tử, hệ thống hiển thị thông báo hướng dẫn cài đặt ví. 5a. Người dùng từ chối xác nhận, hệ thống hiển thị thông báo từ chối xác nhận.	
Hậu điều kiện	Địa chỉ ví được hiển thị trên giao diện. Người dùng có thể thực hiện các thao tác yêu cầu xác thực.	

Hình 2.3: Đặc tả usecase Kết nối ví



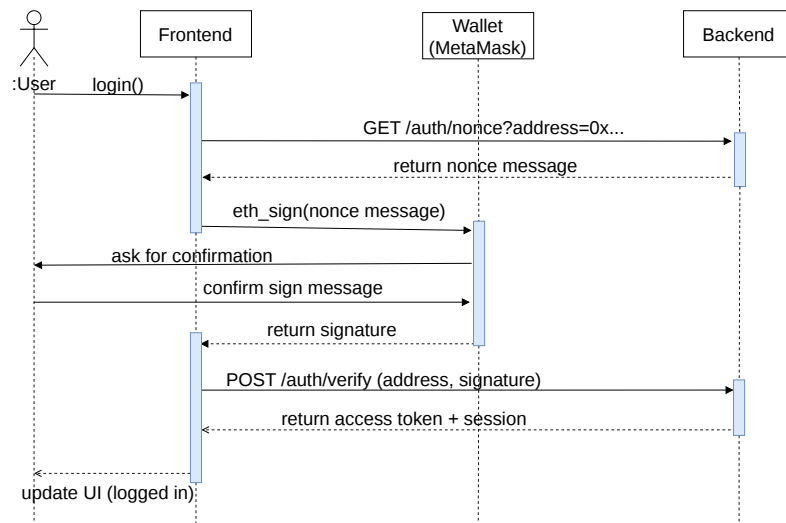
Hình 2.4: Sơ đồ tuần tự Kết nối ví

2.3.2 Đăng nhập (Xác thực địa chỉ ví)

Đặc tả usecase đăng nhập (Xác thực địa chỉ ví) được thể hiện bằng hình 2.5, sơ đồ tuần tự được thể hiện trong hình 2.6 dưới đây

Tên ca sử dụng: Đăng nhập (Xác thực địa chỉ ví)	ID: 3
Tác nhân chính	Người dùng
Tác nhân phụ	Không có
Mô tả ngắn gọn	Ca sử dụng này miêu tả quá trình người dùng xác thực địa chỉ ví của mình để xem những thông tin cá nhân
Tiền điều kiện	Người dùng đã kết nối ví
Luồng sự kiện thông thường	1. Người dùng chọn xác thực tài khoản 2. Hệ thống tạo một tin nhắn cho người dùng 3. Người dùng ký tin nhắn trên ví điện tử 4. Hệ thống xác nhận địa chỉ của người dùng và cung cấp token xác thực
Luồng sự kiện ngoại lệ	3a. Người dùng từ chối ký xác nhận, hệ thống hiển thị thông báo từ chối xác nhận
Hậu điều kiện	Người dùng có thể truy cập vào thông tin cá nhân của mình

Hình 2.5: Đặc tả usecase Đăng nhập (Xác thực địa chỉ ví)



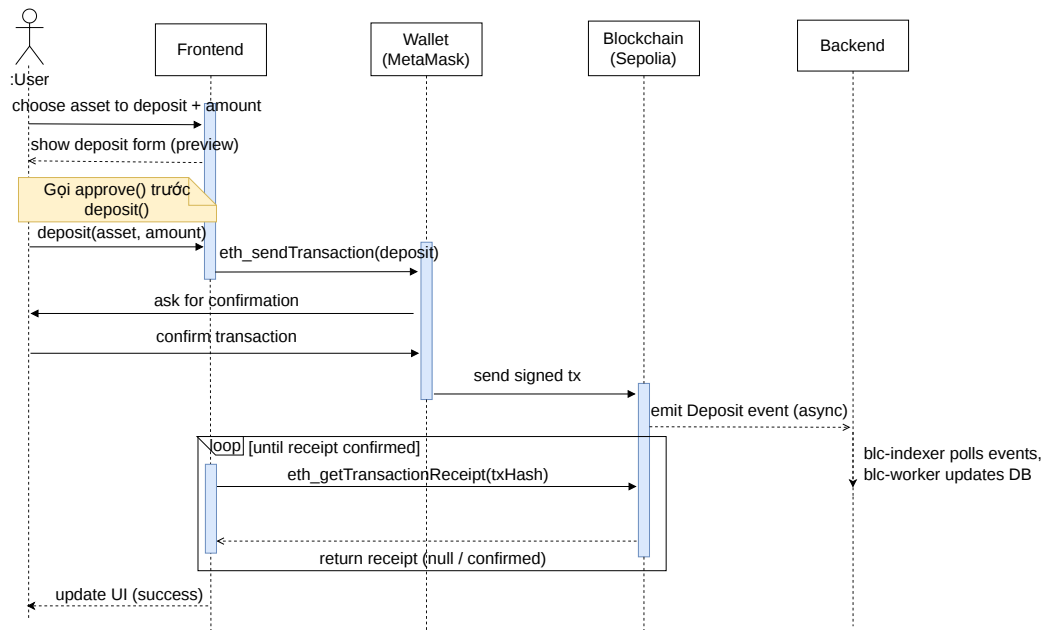
Hình 2.6: Sơ đồ tuần tự Đăng nhập (Xác thực địa chỉ ví)

2.3.3 Gửi tài sản

Đặc tả usecase Gửi tài sản được thể hiện bằng hình 2.7, cùng với sơ đồ tuần tự trong hình 2.8 dưới đây

Tên ca sử dụng: Gửi tài sản		ID: 4
Tác nhân chính	Người dùng	
Tác nhân phụ	Không có	
Mô tả ngắn gọn	Ca sử dụng này miêu tả quá trình người dùng gửi tài sản vào giao thức.	
Tiền điều kiện	Người dùng đã kết nối ví Ví người dùng có tài sản mà giao thức hỗ trợ	
Luồng sự kiện thông thường	1. Người dùng chọn trang gửi tài sản 2. Hệ thống hiển thị những tài sản đang có trong ví người dùng để có thể gửi 3. Người dùng nhấn nút gửi tài sản 4. Hệ thống hiện form để người dùng điền thông tin 5. Người dùng điền thông tin vào form. 6. Người dùng xác nhận trên ví tiền điện tử 7. Giao thức nhận tài sản và thông báo thành công	
Luồng sự kiện ngoại lệ	5a. Nếu nhập dữ liệu sai, hiển thị lỗi trên form 6a. Người dùng từ chối ký xác nhận, hệ thống hiển thị thông báo từ chối xác nhận	
Hậu điều kiện	Số dư tài sản trong giao thức của người dùng tăng lên, số dư trong ví giảm đi	

Hình 2.7: Đặc tả usecase Gửi tài sản



Hình 2.8: Sơ đồ tuần tự Gửi tài sản

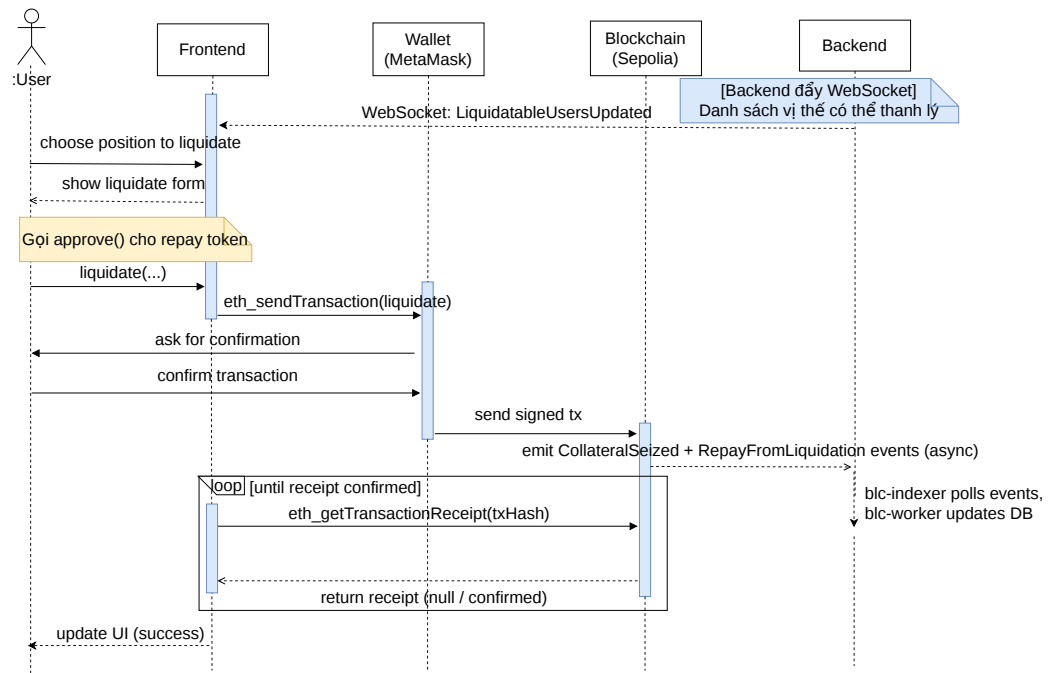
Đối với các ca sử dụng Vay tài sản, Trả nợ và Rút tài sản, ba chức năng còn lại của nghiệp vụ cốt lõi (Vay — Borrow, Trả nợ — Repay, Rút — Withdraw) có chung mô hình tương tác với Gửi tài sản: người dùng nhập số tiền trên giao diện → frontend gọi MetaMask để ký giao dịch (bao gồm bước approve ERC-20 nếu cần) → MetaMask broadcast giao dịch lên blockchain → smart contract thực thi nghiệp vụ và phát event → backend indexer bắt event, lưu database → frontend cập nhật giao diện. Sự khác biệt giữa bốn chức năng nằm hoàn toàn ở tầng business logic guard bên trong smart contract (kiểm tra mức thể chấp với Borrow, kiểm tra thanh khoản pool với Withdraw, cập nhật chỉ số nợ với Repay), không ảnh hưởng đến luồng tương tác giữa các tác nhân. Do đó, đồ án chỉ trình bày đặc tả và sơ đồ tuần tự chi tiết cho Gửi tài sản làm đại diện, còn luồng nghiệp vụ cụ thể của từng hàm được mô tả bằng sơ đồ luồng (flowchart) tại Chương 4 (mục 4.2.1.b, các hình 4.3, 4.4, 4.5 và 4.6).

2.3.4 Thanh lý khoản vay

Đặc tả usecase thanh lý khoản vay được thể hiện bằng hình 2.9, cùng với sơ đồ tuần tự ở hình 2.10

Tên ca sử dụng: Thanh lý khoản vay		ID: 8
Tác nhân chính	Người dùng	
Tác nhân phụ	Không có	
Mô tả ngắn gọn	Ca sử dụng này miêu tả quá trình người dùng thanh lý khoản vay của người khác	
Tiền điều kiện	Người dùng đã kết nối ví Tồn tại khoản vay của người khác có health factor dưới ngưỡng thanh lý, người dùng có đủ tài sản để trả nợ thay	
Luồng sự kiện thông thường	1. Trong trang thanh lý khoản vay, hệ thống hiển thị thông tin các khoản vay có thể thanh lý 2. Người dùng chọn khoản vay để thanh lý 3. Hệ thống hiển thị form nhập thông tin 4. Người dùng điền thông tin và xác nhận 5. Người dùng xác nhận giao dịch trên ví 6. Contract nhận tài sản trả nợ, gửi lại tài sản thế chấp và thông báo thành công	
Luồng sự kiện ngoại lệ	4a. Nếu nhập dữ liệu sai, hiển thị lỗi trên form 5a. Người dùng từ chối ký xác nhận, hệ thống hiển thị thông báo từ chối xác nhận	
Hậu điều kiện	Số dư tài sản trả nợ trong ví của người dùng giảm đi, số dư tài sản thế chấp tăng lên	

Hình 2.9: Đặc tả usecase Thanh lý khoản vay



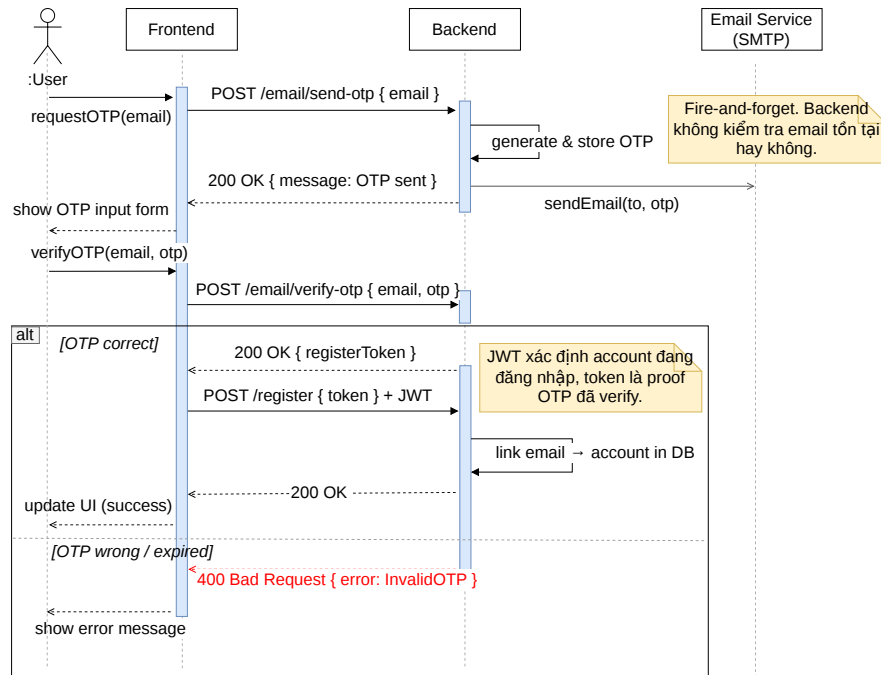
Hình 2.10: Sơ đồ tuần tự Thanh lý khoản vay

2.3.5 Đăng ký email

Đặc tả usecase Đăng ký email được thể hiện bằng hình 2.11, cùng với sơ đồ tuần tự ở hình 2.12

Tên ca sử dụng: Đăng ký email		ID: 12
Tác nhân chính	Người dùng	
Tác nhân phụ	Không có	
Mô tả ngắn gọn	Ca sử dụng này miêu tả quá trình người dùng đăng ký email cá nhân	
Tiền điều kiện	Người dùng đã xác nhận địa chỉ ví Người dùng chưa đăng ký email	
Luồng sự kiện thông thường	1. Người dùng nhập địa chỉ email vào form 2. Người dùng chọn gửi mã OTP 3. Hệ thống gửi mã OTP đến email của người dùng 4. Người dùng nhập mã OTP để xác nhận email 5. Hệ thống xác nhận OTP và tiến hành đăng ký email cho người dùng	
Luồng sự kiện ngoại lệ	1a. Email không hợp lệ, hệ thống hiển thị lỗi 2a. Người dùng chọn gửi OTP quá nhiều lần, hệ thống thông báo giới hạn 3a. Người dùng nhập OTP sai, hệ thống báo lỗi, nếu nhập sai quá số lần cho phép, hệ thống khóa và yêu cầu gửi lại OTP mới	
Hậu điều kiện	Email được đăng ký thành công, người dùng có thể nhận được email khi có hành động admin	

Hình 2.11: Đặc tả usecase Đăng ký email



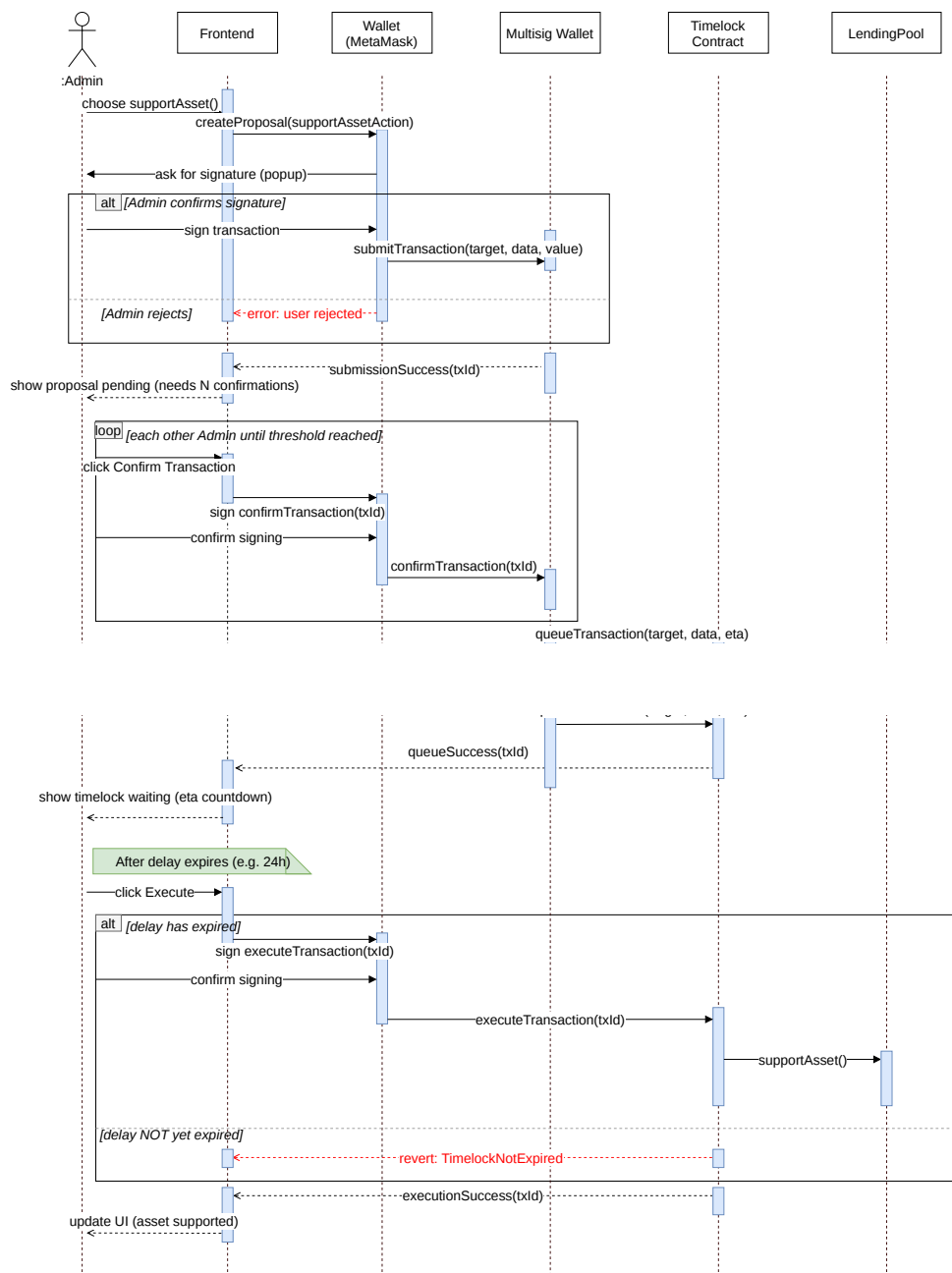
Hình 2.12: Sơ đồ tuần tự Đăng ký email

2.3.6 Hỗ trợ tài sản mới

Đặc tả usecase hỗ trợ tài sản mới được thể hiện bằng hình 2.13, cùng với sơ đồ tuần tự trong hình 2.14

Tên ca sử dụng: Hỗ trợ tài sản mới		ID: 17
Tác nhân chính	Admin	
Tác nhân phụ	Các admin khác	
Mô tả ngắn gọn	Ca sử dụng này miêu tả quá trình admin thêm tài sản vào giao thức	
Tiền điều kiện	Admin đã kết nối ví và được xác thực là admin trong hệ thống	
Luồng sự kiện thông thường	1. Trong admin panel, admin chọn tab quản lý tài sản 2. Admin điền thông tin vào form thêm tài sản 3. Admin xác nhận trên ví điện tử. 4. Hệ thống ghi nhận đề nghị, thông báo đề nghị đã được tạo thành công. 5. Các admin khác ký đề nghị và xác nhận trên ví điện tử đến khi đủ chữ ký. 6. Khi chữ ký đủ, admin nhấn chọn thực thi đề nghị 7. Admin xác nhận trên ví điện tử 8. Hệ thống xác nhận, hiển thị hàng chờ timelock 9. Sau thời gian chờ, admin nhấn chọn thực thi hành động và xác nhận trên ví điện tử 10. Hệ thống xác nhận, thông báo thành công và load lại giao diện.	
Luồng sự kiện ngoại lệ	3a, 7a, 9a. Admin từ chối xác nhận, hệ thống thông báo lỗi 9b. Timelock chưa hết hạn, hệ thống từ chối thực thi	
Hậu điều kiện	Tài sản mới được thêm vào giao thức và hiện ra ở trang chủ và các trang vay nạp	

Hình 2.13: Đặc tả usecase Hỗ trợ tài sản mới



Hình 2.14: Sơ đồ tuần tự Hỗ trợ tài sản mới

2.4 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng các yêu cầu phi chức năng sau: (i) Tính an toàn (Security): Giao thức phải đảm bảo an toàn trong việc

quản lý tài sản của người dùng, ngăn chặn các truy cập trái phép, tấn công reentrancy, thao túng giá và các hành vi gây tổn hại đến hệ thống, (ii) Tính minh bạch (Transparency): Toàn bộ logic nghiệp vụ quan trọng được triển khai on-chain thông qua smart contract, cho phép bất kỳ ai cũng có thể kiểm tra và xác minh. (iii) Hiệu năng (Performance): Hệ thống cần đảm bảo khả năng hoạt động ổn định dưới số lượng giao dịch lớn, trong đó backend được sử dụng để index và truy vấn dữ liệu sự kiện. (iv) Khả năng mở rộng (Scalability): Thiết kế hệ thống cho phép dễ dàng mở rộng thêm tài sản mới, thay đổi mô hình lãi suất và nâng cấp giao thức trong tương lai.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này trình bày các khái niệm lý thuyết và công nghệ nền tảng làm cơ sở cho toàn bộ hệ thống được xây dựng trong đề án. Mỗi mục tập trung vào nguyên lý hoạt động và lý do lựa chọn công nghệ, không đi vào chi tiết cài đặt cụ thể; phần đó được trình bày trong Chương 4 (Thiết kế) và Chương 6 (Cài đặt).

3.1 Blockchain và Ethereum

3.1.1 Mô hình blockchain và cơ chế đồng thuận

Blockchain là một cấu trúc dữ liệu phân tán, trong đó dữ liệu được tổ chức thành các khối (block) liên kết với nhau theo chuỗi thông qua hàm băm mật mã. Mỗi block chứa tập hợp các giao dịch (transaction), hash của block trước đó, và một số định danh duy nhất. Cấu trúc này đảm bảo tính bất biến: thay đổi bất kỳ block nào sẽ làm thay đổi hash của toàn bộ chuỗi phía sau, và mạng lưới sẽ từ chối nhánh bị thay đổi đó [1], [2].

Ethereum ra mắt năm 2015, là blockchain thế hệ hai với khả năng thực thi hợp đồng thông minh, cho phép lưu trữ logic nghiệp vụ tùy ý trên chuỗi, không chỉ giới hạn ở giao dịch chuyển token như Bitcoin. Từ tháng 9/2022, sau sự kiện "The Merge", Ethereum chuyển hoàn toàn sang cơ chế đồng thuận Proof of Stake (PoS), thay thế Proof of Work (PoW) trước đó [2].

Trong PoS, mạng lưới được bảo vệ bởi các validator, tức là những node đặt cọc (stake) tối thiểu 32 ETH để tham gia xác nhận giao dịch. Thời gian trên PoS Ethereum được tổ chức thành hai đơn vị: (i) Slot (12 giây): Tại mỗi slot, một validator được chọn ngẫu nhiên bởi giao thức để đề xuất (propose) một block mới. Đồng thời, một ủy ban (committee) gồm các validator khác được chỉ định để bỏ phiếu xác nhận (attestation) tính hợp lệ của block đó. Nếu validator được chọn không đề xuất block kịp thời (do offline hoặc lỗi mạng), slot đó bị bỏ trống (missed slot). (ii) Epoch (32 slot = 6,4 phút): Đơn vị thời gian để đánh giá và xác nhận trạng thái chuỗi. Sau mỗi epoch, các block được xác nhận sơ bộ thông qua cơ chế bỏ phiếu của toàn bộ tập validator. Khi 2 epoch liên tiếp đều được justified, epoch đầu tiên trở thành xác nhận vĩnh viễn; các block trong epoch đó không thể bị thay đổi trừ khi ít nhất 1/3 tổng ETH đặt cọc bị tiêu hủy (slashed). Cơ chế PoS có hai ưu điểm quan trọng so với PoW: giảm tiêu thụ năng lượng hơn 99%, và cung cấp cơ chế finality xác định (deterministic finality), nghĩa là sau khi finalize, block là bất biến, khác với PoW nơi xác nhận chỉ mang tính xác suất.

3.1.2 Ethereum Virtual Machine (EVM) và Gas

EVM là máy tính ảo thực thi smart contract trên Ethereum. Mỗi node trong mạng chạy một bản sao EVM và thực thi các lệnh theo cùng thứ tự, đảm bảo kết quả đồng nhất trên toàn mạng; đây là nền tảng cho tính chất deterministic của smart contract [2].

Gas là đơn vị đo lường chi phí tính toán trên EVM. Mỗi lệnh (opcode) có chi phí gas cố định — ví dụ phép cộng (ADD) tốn 3 gas, trong khi ghi vào storage (SSTORE) tốn khoảng 20.000 gas. Người gửi transaction phải trả gas fee bằng ETH; nếu gas hết trước khi transaction hoàn thành, toàn bộ thay đổi được hoàn tác (revert) nhưng gas đã dùng vẫn bị tính. Cơ chế này ngăn chặn các vòng lặp vô hạn và tấn công từ chối dịch vụ (DoS). Chi phí SSTORE cao nhất trong các opcode chính là lý do cốt lõi tại sao thiết kế smart contract phải tránh cập nhật storage cho người dùng trong một transaction; bài toán này được giải quyết bởi cơ chế Global Interest Index trình bày tại mục 5.1.

Kể từ bản nâng cấp London (tháng 8/2021), Ethereum áp dụng cơ chế phí gas theo chuẩn EIP-1559. Phí gas được chia thành hai thành phần: base fee — mức phí tối thiểu do giao thức tự động điều chỉnh theo mức độ sử dụng block, bị đốt hoàn toàn và không trả cho validator; và priority fee (tip) — khoản phí tùy chọn trả cho validator để giao dịch được ưu tiên. Tổng phí được tính bằng công thức

$$\text{Gas fee} = \text{Gas used} \times (\text{Base fee} + \text{Priority fee}) \quad (3.1)$$

Cơ chế này giúp phí gas trở nên minh bạch và dễ dự đoán hơn so với mô hình đấu giá trước đó [2].

Đối với đề án, EIP-1559 không ảnh hưởng đến logic nghiệp vụ bên trong smart contract, nhưng ảnh hưởng trực tiếp đến chi phí mỗi lần người dùng tương tác với giao thức (deposit, borrow, repay, withdraw). Vì vậy, tối ưu gas trong thiết kế smart contract, chẳng hạn như giảm số lượng SSTORE và sử dụng Lazy Accrual thay vì Eager Accrual, có ý nghĩa thực tế quan trọng.

3.1.3 Smart contract và tính bất biến

Smart contract là chương trình được lưu trữ trên blockchain và thực thi tự động trong transaction. Khi đã deploy, bytecode của smart contract không thể thay đổi; đây là tính chất bất biến (immutability) của blockchain. Tính chất này đảm bảo người dùng có thể kiểm chứng logic của smart contract và tin rằng nó sẽ hoạt động đúng như cam kết.

Tuy nhiên, tính bất biến cũng tạo ra thách thức: khi phát hiện lỗi hỏng bảo mật hoặc cần nâng cấp tính năng, không thể sửa smart contract đã deploy. Để giải quyết, hệ thống sử dụng Proxy Pattern — một kỹ thuật tách biệt logic khỏi dữ liệu, cho phép thay thế phần logic trong khi giữ nguyên địa chỉ smart contract và dữ liệu người dùng. Chi tiết về UUPS Proxy Pattern được trình bày tại mục 3.4.

3.1.4 Chuẩn token ERC-20

ERC-20 (Ethereum Request for Comment 20) là chuẩn giao diện (interface) phổ biến nhất cho token trên Ethereum, được đề xuất bởi Fabian Vogelsteller và Vitalik Buterin vào năm 2015. Chuẩn này quy định một tập hợp tối thiểu các hàm mà mọi token contract phải triển khai, bao gồm: `transfer()` để chuyển token giữa các địa chỉ, `approve()` và `transferFrom()` để cho phép một địa chỉ khác (thường là smart contract) chi tiêu token thay mặt chủ sở hữu, cùng với `balanceOf()` để truy vấn số dư và `totalSupply()` để biết tổng cung [2].

Chuẩn ERC-20 là nền tảng kỹ thuật trực tiếp của đồ án: toàn bộ tài sản mà giao thức hỗ trợ đều là token ERC-20. Khi người dùng gửi tài sản vào LendingPool, quy trình bắt buộc gồm hai bước: (1) người dùng gọi `approve(lendingPoolAddress, amount)` trên token contract để cấp quyền, (2) LendingPool gọi `transferFrom(user, address(this), amount)` để chuyển token vào pool. Cơ chế approve-transferFrom này đảm bảo chỉ khi người dùng chủ động cấp quyền, smart contract mới có thể di chuyển token, từ đó ngăn chặn truy cập trái phép.

Một vấn đề thực tế quan trọng là không phải mọi token ERC-20 đều tuân thủ chuẩn một cách đầy đủ: một số token không trả về giá trị `bool` khi gọi `transfer()`, một số khác trả về `false` thay vì `revert` khi thất bại. Để xử lý các trường hợp bất thường này, LendingPool sử dụng thư viện `SafeERC20` của OpenZeppelin — bọc lại các lời gọi transfer và revert nếu token báo lỗi, đảm bảo an toàn cho mọi loại ERC-20 (xem tầng 1 bảo mật tại mục 4.2.1.d) [10].

Ngoài ra, mỗi token ERC-20 có thuộc tính `decimals` xác định số chữ số thập phân: ví dụ WETH có 18 decimals trong khi USDC chỉ có 6 decimals. Sự khác biệt này đòi hỏi giao thức phải chuẩn hóa về cùng một không gian (18 decimals) khi tính toán giá trị USD, đặc biệt trong quá trình thanh lý (xem mục 4.2.3).

3.2 DeFi Lending — Mô hình và cơ chế kinh tế

3.2.1 Mô hình pool thanh khoản chung

Tài chính phi tập trung (Decentralized Finance, viết tắt là DeFi) là hệ sinh thái ứng dụng tài chính vận hành trên blockchain, không cần trung gian tập trung. Trong mô hình lending phi tập trung, người gửi tiền (depositor) và người vay (borrower)

không giao dịch trực tiếp với nhau mà thông qua một pool thanh khoản chung được quản lý bởi smart contract [3].

Cách hoạt động của pool: người gửi đưa tài sản vào pool và nhận lãi suất từ người vay; người vay rút tài sản từ pool với điều kiện thế chấp tài sản khác có giá trị cao hơn khoản vay. Smart contract tự động tính toán lãi suất, theo dõi số dư và thực thi thanh lý khi cần; vì thế, không cần nhân viên ngân hàng, hợp đồng giấy hay thủ tục phê duyệt.

Ưu điểm của mô hình pool so với mô hình peer-to-peer truyền thống: người gửi không cần chờ được khớp với người vay cụ thể; người vay có thể vay ngay lập tức nếu pool có đủ thanh khoản; lãi suất được điều chỉnh tự động theo cung cầu thị trường.

3.2.2 Overcollateralization và lý do tại sao

Khác với ngân hàng truyền thống có thể cho vay tín chấp (dựa trên lịch sử tín dụng và thu nhập), DeFi lending yêu cầu người vay thế chấp tài sản có giá trị lớn hơn khoản vay — gọi là overcollateralization (thế chấp quá mức) [3].

Lý do bắt buộc phải overcollateralize: trên blockchain, không có cơ chế pháp lý để truy đòi con nợ như trong thế giới thực. Nếu người vay không trả nợ và biến mất, giao thức không thể kiện hay thu hồi tài sản. Do đó, giao thức phải nắm giữ tài sản thế chấp có giá trị đủ lớn để tự bảo vệ — ngay cả khi giá tài sản thế chấp giảm một phần, giao thức vẫn có thể bán ra để thu hồi khoản nợ.

Collateral Factor (hệ số thế chấp) là tỷ lệ tối đa mà người vay có thể vay so với giá trị tài sản thế chấp. Ví dụ, collateral factor = 0,75 nghĩa là với 100 USD tài sản thế chấp, tối đa có thể vay 75 USD. Hệ số này có thể được đặt khác nhau cho từng tài sản dựa trên mức độ rủi ro: tài sản ổn định như USDC có thể có collateral factor cao hơn tài sản biến động như các token nhỏ.

3.2.3 Health Factor và cơ chế thanh lý

Health Factor (hệ số sức khỏe) là chỉ số đo lường mức độ an toàn của một vị thế vay. Công thức tổng quát:

$$\text{Health Factor} = \frac{\text{Tổng giá trị tài sản thế chấp} \times \text{liquidationThreshold}}{\text{Tổng giá trị nợ}} \quad (3.2)$$

Khi Health Factor ≥ 1 , vị thế an toàn. Khi Health Factor < 1 , vị thế có thể bị thanh lý — người thanh lý (liquidator) sẽ trả một phần nợ thay cho người vay và nhận lại tài sản thế chấp tương ứng với giá chiết khấu (liquidation incentive). Cơ

chế này tạo ra động lực kinh tế để bên thứ ba chủ động duy trì sức khỏe của giao thức.

Liquidation Threshold khác với Collateral Factor: Collateral Factor xác định mức tối đa có thể vay ban đầu trong khi Liquidation Threshold xác định ngưỡng mà tại đó vị thế sẽ bị thanh lý. Thông thường Liquidation Threshold sẽ lớn hơn Collateral Factor, tạo ra một vùng đệm an toàn để người vay có thời gian phản ứng khi giá tài sản giảm.

3.2.4 Utilization Rate và cơ chế điều chỉnh lãi suất

Utilization Rate (tỷ lệ sử dụng vốn) $U = \text{TotalBorrows} / \text{TotalDeposits}$ là chỉ số trung tâm điều phối cân bằng cung cầu trong giao thức.

Khi U cao (gần 100%), pool sắp cạn kiệt thanh khoản: người gửi không thể rút tiền, rủi ro hệ thống tăng. Lúc này, lãi suất vay và gửi phải tăng cao để: thu hút thêm người gửi tiền và buộc người vay trả nợ sớm từ đó tái cân bằng pool. Khi U thấp, tiền trong pool nhàn rỗi, lãi suất giảm để khuyến khích vay nhiều hơn, tối đa hóa lợi suất cho giao thức.

Mối liên hệ giữa utilization rate và lãi suất là nền tảng lý thuyết của mô hình Two-Slope được trình bày chi tiết ở mục 3.3 bên dưới.

3.3 Mô hình lãi suất Two-Slope (Kinked Rate Model)

3.3.1 Lý thuyết kinh tế của mô hình

Mô hình Two-Slope, còn gọi là Kinked Rate Model, là mô hình lãi suất chuẩn được sử dụng trong hầu hết các giao thức DeFi lending lớn (Aave, Compound) [4], [5]. Ý tưởng cốt lõi: lãi suất không tăng đều tuyến tính theo utilization rate, mà tăng theo hai giai đoạn với tốc độ khác nhau, tạo thành dạng đường gãy (kinked).

Giai đoạn 1 — Vùng bình thường ($U \leq U_{\text{optimal}}$): Lãi suất tăng chậm và tuyến tính. Mục tiêu là duy trì chi phí vay hợp lý để khuyến khích sử dụng vốn hiệu quả, trong khi vẫn đủ hấp dẫn để thu hút người gửi.

Giai đoạn 2 — Vùng quá tải ($U > U_{\text{optimal}}$): Lãi suất tăng rất nhanh với hệ số dốc lớn hơn nhiều ($\text{slope2} \gg \text{slope1}$). Tại đây, giao thức đang tiến đến nguy cơ mất cân bằng thanh khoản, lãi suất cao đột ngột tạo áp lực kinh tế cấp bách buộc người vay trả nợ, đồng thời thu hút dòng tiền mới từ người gửi.

U_{optimal} (thường 70-80%) là điểm cân bằng tối ưu về lý thuyết: pool sử dụng vốn hiệu quả nhưng vẫn giữ đủ thanh khoản cho người gửi muốn rút. Đây là tham số quan trọng nhất trong việc hiệu chỉnh hành vi kinh tế của giao thức.

3.3.2 Reserve Factor — Cơ chế doanh thu giao thức

Reserve Factor là tỷ lệ phần trăm lãi từ người vay được giữ lại cho ngân quỹ giao thức, thay vì trả toàn bộ cho người gửi. Về mặt kinh tế, đây là cơ chế tạo doanh thu bền vững: giao thức tích lũy nguồn dự phòng từ hoạt động thực tế, không phụ thuộc vào token inflation hay trợ cấp bên ngoài. Nguồn này có thể dùng để bù đắp tổn thất khi thanh lý không kịp, trả tiền cho nhóm phát triển, hoặc phát triển hệ sinh thái.

3.4 UUPS Upgradeable Proxy Pattern

3.4.1 Vấn đề và nhu cầu proxy

Như đã đề cập ở mục 3.1, smart contract sau khi deploy là bất biến. Với một giao thức tài chính thực tế, điều này tạo ra rủi ro nghiêm trọng: nếu phát hiện lỗi hỏng bảo mật, không thể vá mà không mất toàn bộ dữ liệu người dùng và TVL. Người dùng cũng sẽ phải migrate tài sản sang smart contract mới — một quy trình phức tạp, tốn kém và rủi ro.

Proxy Pattern giải quyết vấn đề này bằng cách tách một smart contract thành hai phần: Proxy Contract lưu toàn bộ state (số dư người dùng, tham số giao thức,...) tại địa chỉ cố định; Implementation Contract chứa logic nghiệp vụ tại địa chỉ có thể thay thế. Người dùng luôn tương tác với địa chỉ proxy. Khi cần nâng cấp, chỉ deploy implementation mới và trỏ proxy sang đó, từ đó state được giữ nguyên mà logic thực thi trên state đó thì thay đổi [9], [10].

3.4.2 Cơ chế delegatecall

Delegatecall là opcode EVM cho phép smart contract A gọi code của smart contract B nhưng thực thi trong context (storage, msg.sender, msg.value) của smart contract A. Đây là nền tảng của mọi proxy pattern: khi người dùng gọi hàm deposit() trên proxy, proxy dùng delegatecall để chạy code deposit() từ implementation, nhưng đọc và ghi vào storage của chính proxy — không phải của implementation. Người dùng nhìn thấy proxy như một smart contract đơn lẻ bình thường.

3.4.3 So sánh ba loại proxy phổ biến

Tiêu chí	Transparent Proxy	UUPS	Beacon Proxy
Logic upgrade	Trong Proxy Contract	Trong Implementation Contract	Trong Beacon Contract
Chi phí gas mỗi call	Cao hơn (kiểm tra admin)	Thấp nhất	Trung bình
Chi phí upgrade	Trung bình	Cao hơn một chút (deploy impl mới)	Thấp (1 thay đổi, nhiều proxy cập nhật)
Độ phức tạp	Thấp	Trung bình	Cao
Phù hợp khi	Contract đơn lẻ, upgrade thường xuyên	Contract cốt lõi, tối ưu gas	Nhiều proxy cùng implementation

Bảng 3.1: So sánh 3 loại proxy phổ biến

Sau khi so sánh cùng với các giải pháp thay thế trong bảng 3.1, UUPS được lựa chọn cho giao thức vì hai lý do chính. Thứ nhất, chi phí gas mỗi lần call thấp nhất trong ba loại proxy do không có logic kiểm tra admin tại mỗi lần dispatch. Thứ hai, logic upgrade nằm trong implementation contract, cho phép kiểm soát điều kiện nâng cấp ngay tại tầng logic; ví dụ, chỉ controller mới có quyền upgrade, thay vì phải lưu logic phân quyền riêng trong proxy. Tuy nhiên, thiết kế này đi kèm một rủi ro đặc thù cần lưu ý: nếu implementation mới được deploy thiếu hàm `_authorizeUpgrade`, proxy sẽ mất khả năng upgrade vĩnh viễn vì không còn entry point nào để thực hiện. Đây là điểm khác biệt so với Transparent Proxy, nơi logic upgrade nằm trong proxy nên không bị ảnh hưởng bởi nội dung của implementation. Rủi ro này được kiểm soát trong đồ án bằng hai cơ chế: toàn bộ test case upgrade trong `ProtocolController.test.ts` đều xác minh hàm `_authorizeUpgrade` tồn tại và hoạt động đúng trên implementation mới; và việc upgrade bắt buộc phải đi qua `ProtocolController` với `timelock delay`, tạo cửa sổ thời gian để phát hiện lỗi trước khi lệnh được thực thi.

3.4.4 Storage Layout và rủi ro xung đột slot

Một vấn đề đáng chú ý trong UUPS: khi upgrade implementation mới, thứ tự khai báo các biến state phải giữ nguyên hoàn toàn so với implementation cũ. Nếu thêm biến mới vào giữa hoặc xóa biến cũ, các biến tiếp theo sẽ bị dịch chuyển sang slot storage khác, dẫn đến đọc nhầm dữ liệu — một loại lỗi cực kỳ nguy hiểm vì không gây revert mà âm thầm trả về giá trị sai.

Giải pháp chuẩn là khai báo một mảng `__gap[ ]` ở cuối storage contract, dự trữ một số lượng slot nhất định cho các biến tương lai. Khi cần thêm biến mới, chỉ cần

thu nhỏ __gap[] tương ứng, khi đó thứ tự các biến hiện tại không bị thay đổi.

3.5 Oracle và vấn đề giá tài sản on-chain

3.5.1 Oracle Problem trong blockchain

Smart contract hoạt động trong một môi trường khép kín: EVM chỉ có thể đọc dữ liệu đã tồn tại trên blockchain, không thể tự lấy dữ liệu từ thế giới bên ngoài (giá cổ phiếu, tỷ giá, kết quả thể thao...). Đây là Oracle Problem, tức là một thách thức cơ bản của blockchain.

Với DeFi lending, việc tính health factor và giá trị tài sản thế chấp đòi hỏi phải biết giá thị trường của từng tài sản tại thời điểm hiện tại. Nếu giá nguồn cấp không chính xác hoặc bị thao túng, kẻ tấn công có thể thao túng thanh lý để thu lợi bất chính, hoặc vay nhiều hơn mức được phép. Do đó, nguồn cấp giá là một trong những điểm tấn công quan trọng nhất của bất kỳ lending protocol nào.

3.5.2 Chainlink Data Feed — Giải pháp oracle phi tập trung

Chainlink là mạng lưới oracle phi tập trung lớn nhất, cung cấp dữ liệu giá cho hầu hết các giao thức DeFi lớn bao gồm Aave và Compound. Chainlink Data Feed hoạt động theo mô hình aggregation: nhiều node độc lập lấy giá từ nhiều sàn giao dịch khác nhau và báo cáo vào một Aggregator contract trên chain. Giá cuối cùng là tổng hợp của tất cả báo cáo, loại bỏ ảnh hưởng của nguồn đơn lẻ bị lỗi hoặc bị tấn công [13].

Chainlink Data Feed có hai cơ chế cập nhật: Heartbeat — cập nhật định kỳ sau một khoảng thời gian nhất định (ví dụ 24 giờ) ngay cả khi giá không thay đổi nhiều; và Deviation Threshold — cập nhật ngay khi giá thay đổi quá một ngưỡng nhất định (ví dụ 0,5%). Kết hợp hai cơ chế này đảm bảo giá on-chain luôn gần với giá thực tế thị trường.

3.5.3 Stale Price — Rủi ro giá cũ

Stale Price xảy ra khi Chainlink node ngừng cập nhật, có thể là do sự cố mạng, lỗi node, hoặc thị trường bị gián đoạn. Smart contract không có cơ chế tự động phát hiện điều này; nó chỉ đọc giá mới nhất trên chain, có thể là giá đã lỗi thời hàng giờ đồng hồ. Rủi ro: nếu giá thực tế đã giảm mạnh nhưng oracle vẫn báo giá cao, người dùng có thể vay vượt mức an toàn thực tế. Ngược lại, giá thực tế tăng nhưng oracle vẫn báo giá thấp, người dùng có thể bị thanh lý oan.

3.6 Message Queue và kiến trúc Event-Driven

3.6.1 Message Broker và Producer-Consumer Pattern

Message broker là thành phần trung gian nhận message từ producer (bên gửi) và chuyển đến consumer (bên nhận). Khác với gọi hàm trực tiếp, message broker

tách biệt hoàn toàn producer và consumer về mặt thời gian và không gian: producer gửi message vào queue và tiếp tục công việc ngay lập tức mà không cần chờ consumer xử lý xong; consumer xử lý message theo tốc độ của mình, có thể chậm hơn producer [14].

Kiến trúc Event-Driven xây dựng hệ thống xung quanh luồng sự kiện: mỗi thay đổi trạng thái quan trọng được biểu diễn dưới dạng event và truyền đi qua message broker. Các thành phần quan tâm đến event đó sẽ subscribe và xử lý độc lập. Kiến trúc này phù hợp tự nhiên với blockchain, nơi mọi thay đổi trạng thái đều được phát ra dưới dạng event (Deposit, Borrow, Repay, ...).

3.6.2 Các đảm bảo delivery và trade-off

Mức đảm bảo	Ý nghĩa	Rủi ro	Phù hợp với
At-most-once	Message được xử lý tối đa 1 lần	Mất message nếu consumer crash	Log không quan trọng, metrics
At-least-once	Message được xử lý ít nhất 1 lần	Duplicate nếu crash sau xử lý nhưng trước ack	Dữ liệu tài chính quan trọng (với idempotency)
Exactly-once	Message được xử lý đúng 1 lần	Chi phí cao, phức tạp, throughput thấp	Giao dịch ngân hàng nghiêm ngặt

Bảng 3.2: So sánh các đảm bảo delivery

Bảng 3.2 đã đặt lên bàn cân 3 giải pháp delivery, trong đó, At-least-once được lựa chọn cho hệ thống backend của đồ án vì mất event blockchain là không thể chấp nhận (dữ liệu sẽ lệch với on-chain), trong khi duplicate có thể xử lý bằng idempotency ở tầng ứng dụng với chi phí thấp hơn nhiều so với exactly-once.

3.6.3 Idempotency — Xử lý hệ quả của at-least-once

Idempotency là tính chất của một thao tác mà thực hiện nhiều lần cho ra kết quả giống hệt thực hiện một lần. Với at-least-once delivery, một event có thể được xử lý nhiều lần (ví dụ: consumer crash sau khi lưu database nhưng trước khi ack). Nếu handler không idempotent, database sẽ có bản ghi trùng lặp.

Cách thiết kế idempotent handler: sử dụng unique constraint trên trường định danh tự nhiên của event (ví dụ: transactionHash trên Ethereum là duy nhất toàn cầu). Khi event được xử lý lần thứ hai, lệnh INSERT vi phạm constraint và bị bỏ qua — thay vì tạo bản ghi trùng hay ném lỗi.

3.7 Sign-In with Ethereum (EIP-4361)

3.7.1 Vấn đề xác thực trong ứng dụng phi tập trung

Ứng dụng Web2 truyền thống xác thực người dùng bằng username/password hoặc OAuth với bên thứ ba (Google, Facebook). Cả hai cách đều giả định tồn tại

một cơ sở dữ liệu tập trung lưu thông tin tài khoản — mâu thuẫn với triết lý phi tập trung của DeFi.

Trong Web3, mỗi người dùng đã có một danh tính duy nhất: địa chỉ ví Ethereum, được tạo ra từ cặp public/private key. Nếu người dùng có thể chứng minh họ sở hữu private key của một địa chỉ mà không lộ key đó, đây chính là cơ chế xác thực hoàn hảo: không cần đăng ký, không có password để bị lộ, không phụ thuộc bên thứ ba.

3.7.2 Chuẩn EIP-4361 và cơ chế hoạt động

EIP-4361 (Sign-In with Ethereum) chuẩn hóa quy trình xác thực bằng ví Ethereum. Quy trình cốt lõi: backend tạo một message plaintext có cấu trúc cố định (địa chỉ ví, nonce ngẫu nhiên, chainId, thời gian hết hạn); người dùng ký message này bằng private key của ví; backend xác minh chữ ký bằng địa chỉ ví tương ứng [11].

Tính bảo mật đến từ hai yếu tố: (i) chữ ký ECDSA là bằng chứng toán học về việc sở hữu private key, không thể giả mạo; (ii) nonce ngẫu nhiên trong message ngăn replay attack, vì mỗi session có một nonce khác nhau nên chữ ký cũ không thể dùng lại.

Sơ đồ tuần tự chi tiết của luồng xác thực được trình bày bằng hình 2.6, cài đặt cụ thể trong hệ thống được nói đến tại mục 6.2.3.c.

3.8 Lý do lựa chọn công nghệ

Bảng 3.3 dưới đây tổng hợp lý do lựa chọn công nghệ cụ thể cho từng tầng, bổ sung cho định hướng kiến trúc tổng thể đã trình bày ở mục 1.3. Các lựa chọn được tham khảo từ tài liệu chính thức của các nền tảng và thư viện liên quan [10], [12], [14], [15].

Tầng	Công nghệ được chọn	Lý do chính	Lựa chọn thay thế đã xem xét
Smart contract	Solidity + Hardhat	Ngôn ngữ chuẩn cho Ethereum, hệ sinh thái phong phú, OpenZeppelin library	Vyper (ít tài liệu hơn), Foundry (tốt cho testing nhưng Hardhat linh hoạt hơn cho deploy)
Smart contract	UUPS Proxy	Gas thấp nhất cho mỗi call, logic upgrade trong implementation tăng bảo mật	Transparent Proxy (gas cao hơn), Beacon Proxy (phức tạp không cần thiết)
Backend	Node.js + TypeScript	Cùng hệ sinh thái với ethers.js, async/event-driven phù hợp với blockchain	Python (ít library blockchain hơn), Go (mạnh hơn nhưng ít library DeFi)
Backend	RabbitMQ	Đủ throughput cho quy mô đồ án, cài đặt đơn giản, hỗ trợ tốt acknowledgment	Kafka (overkill cho quy mô này), Redis Streams (thiếu một số tính năng)
Backend	PostgreSQL	Hỗ trợ DECIMAL(78,0) cho uint256, mature và đáng tin cậy	MySQL (tương tự), MongoDB (schema linh hoạt hơn nhưng kém transaction)
Frontend	Next.js + ethers.js v6	SSR giúp tải trang nhanh, ethers.js là library chuẩn cho Ethereum Web3	Vite/React thuần (không có SSR), web3.js (ít được maintain hơn ethers.js)

Bảng 3.3: Các công nghệ được sử dụng

CHƯƠNG 4. THIẾT KẾ HỆ THỐNG

4.1 Tổng quan kiến trúc hệ thống

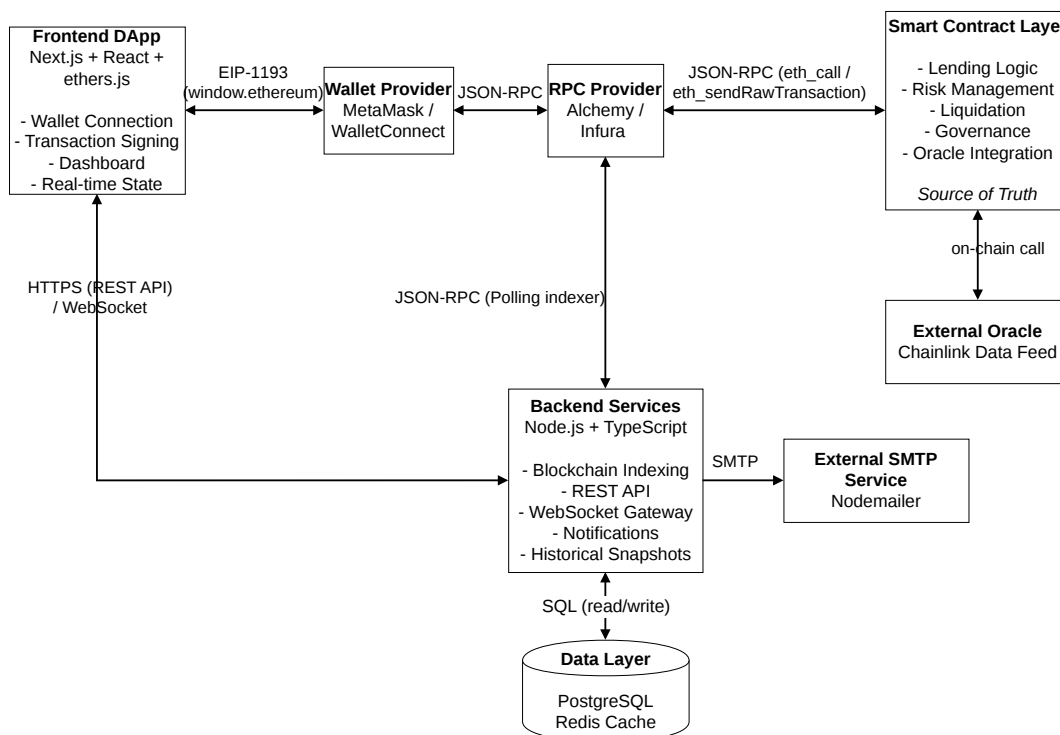
Khác với các ứng dụng web truyền thống, hệ thống ứng dụng vay và cho vay phi tập trung này được xây dựng theo mô hình ba thành phần chính: frontend, smart contract và backend, được thể hiện chi tiết bằng hình 4.1. Mỗi thành phần đảm nhiệm một vai trò riêng biệt, trong đó smart contract đóng vai trò trung tâm thay vì backend như trong mô hình Web2 thông thường.

Đầu tiên là frontend, đây là thành phần giao diện người dùng, chịu trách nhiệm hiển thị dữ liệu và tiếp nhận tương tác từ người dùng. Frontend không kết nối trực tiếp với blockchain mà thông qua hai kênh trung gian: Thứ nhất, thông qua Wallet Provider (MetaMask) bằng giao thức EIP-1193 (window.ethereum), khi người dùng cần ký và gửi giao dịch, frontend gọi MetaMask để yêu cầu chữ ký, MetaMask ký giao dịch và broadcast lên mạng thông qua RPC Provider (Alchemy / Infura) bằng JSON-RPC; Thứ hai, frontend đọc dữ liệu trực tiếp từ blockchain thông qua RPC Provider bằng `eth_call` để lấy trạng thái real-time của các smart contracts mà không cần ký giao dịch. Ngoài ra, frontend giao tiếp với backend qua HTTPS (REST API) và nhận thông báo real-time qua WebSocket.

Tiếp theo là smart contracts, smart contract là trái tim của toàn hệ thống, đóng vai trò tương đương với backend trong mô hình Web2 truyền thống. Toàn bộ trạng thái của hệ thống bao gồm sổ dư tài sản, khoản vay, tỉ lệ lãi suất, và các tham số rủi ro đều được lưu trữ trên blockchain thông qua các smart contracts. Đây cũng là tầng chứa và có nhiệm vụ thực thi toàn bộ logic nghiệp vụ của hệ thống. Ngoài ra, mỗi khi có tương tác với giao thức, smart contracts sẽ phát ra các sự kiện (events) kèm theo thông tin chi tiết về hành động đó, phục vụ cho việc index dữ liệu ở tầng backend.

Cuối cùng là tầng backend, khác với mô hình Web2 nơi backend là nơi xử lý logic nghiệp vụ, backend trong hệ thống này không có quyền thay đổi trạng thái giao thức. Mục đích chính của backend đóng vai trò indexer trung gian, lưu trữ dữ liệu không thể query on-chain như lịch sử sự kiện hay dữ liệu tổng hợp, giúp người dùng theo dõi quá trình tương tác với giao thức. Backend thực hiện điều này bằng cách liên tục polling RPC Provider thông qua `eth_getLogs` để lắng nghe các sự kiện mà smart contracts phát ra, sau đó xử lý dữ liệu từ sự kiện và lưu vào PostgreSQL. Bên cạnh đó, backend thực hiện snapshot hàng ngày các thông tin về tài sản giao thức và vị thế người dùng phục vụ cho việc theo dõi lịch sử. Để phối hợp giữa các tiến trình nội bộ (indexer, snapshot, notification), backend sử dụng RabbitMQ

làm message broker [14] và Redis để đảm nhiệm hai vai trò: lưu trữ các dữ liệu tồn tại ngắn hạn mà không có nhu cầu phải lưu trữ database, và pub/sub để hỗ trợ WebSocket real-time đẩy thông báo tới frontend. Ngoài ra, backend cho phép người dùng đăng ký email và gửi thông báo về các thay đổi quan trọng của giao thức thông qua SMTP Service.



Hình 4.1: Sơ đồ kiến trúc tổng quan hệ thống

4.2 Thiết kế hợp đồng thông minh (Smart contract)

Giao thức mà đề án này xây dựng gồm có tổng cộng 5 smart contract logic chính. Thông thường, khi đã triển khai một smart contract on-chain, smart contract đó là bất biến, các hàm logic không thể thay đổi, tuy nhiên, trong giao thức của đề án, có 2 smart contract được thiết kế theo hướng có thể cập nhật mới logic, kỹ thuật được lựa chọn sử dụng ở đây chính là UUPS (Universal Upgradeable Proxy Standard) [9], [10], chia một contract thành hai phần: một implementation contract (logic) và một proxy contract (storage). Sau đây là sơ bộ tổng quan về các smart contract được triển khai: (i) LendingPool / LendingPoolStorage: smart contract quan trọng nhất của giao thức, đảm nhiệm đa số các chức năng chính (gửi/ vay/ rút/ trả tài sản), thực hiện lazy accrual (tích lũy lãi suất chỉ khi người dùng tương tác); chứa logic cho phép admin quản lý tài sản và ngân quỹ; cung cấp các hàm tiện ích để phục vụ cho UI, UX, đồng thời lưu trữ toàn bộ các thông tin về các sổ dư tài sản, người dùng trong giao thức. (ii) Liquidation: đảm nhận chức năng thanh lý vị

thể của giao thức, chịu trách nhiệm cho các tham số rủi ro, các hàm phục vụ cho việc phát hiện vị thế rơi vào ngưỡng thanh lý và thực hiện quá trình thanh lý. (iii) InterestRateModel: đảm nhận xử lý mô hình lãi suất của giao thức, tính lãi gửi, lãi vay của từng tài sản được hỗ trợ trong giao thức, smart contract này là một smart contract độc lập, không phụ thuộc vào các smart contract khác trong hệ thống, chỉ có chức năng tính và cung cấp lãi suất vay và gửi tương ứng. (iv) PriceRouter/PriceRouterStorage: hoạt động giống như một bộ định tuyến quyết định xem sử dụng giá từ MyOracle hay từ Oracle thật (Chainlink), từ đó cung cấp giá cho các loại tài sản khác nhau trong giao thức [13]. (v) MyOracle: để tự cung cấp giá cho các tài sản trong giao thức.

Bên cạnh các smart contract phục vụ cho nghiệp vụ logic, giao thức còn có thêm 2 smart contract nhằm mục đích quản trị giao thức, đó là ProtocolController, làm entrypoint cho admin để thực thi quyền quản trị và cài đặt đối với các smart contract khác trong giao thức và TimelockController, dùng để áp đặt timelock (khóa thời gian) cho các động thái của admin tác động đến giao thức nhằm đảm bảo tính minh bạch cho giao thức. Ngoài ra còn có 1 smart contract nữa là MockERC20 - token giả định để có thể kiểm thử. Khi đã xác định xong bộ khung của giao thức, tiến hành xây dựng từng smart contract một, bao gồm các biến (state), các cấu trúc dữ liệu, các hàm logic nghiệp vụ chính, xác định một smart contract sẽ cần phải tương tác với các smart contract nào (ở đây, đồ án lựa chọn hướng thiết kế đó là thay vì chỉ lưu địa chỉ smart contract ở ProtocolController thì lưu cả địa chỉ các smart contract ở nội bộ smart contract, tuy đánh đổi bởi việc phải lưu ở nhiều chỗ hơn, khi cập nhật địa chỉ smart contract mới thì cần phải ghi nhiều hơn, nhưng khi đọc thì các smart contract có thể tự lấy địa chỉ nhanh chóng mà không cần một RPC call sang ProtocolController).

Trước khi đi vào chi tiết từng smart contract, cần nắm được bức tranh tổng thể về cách các smart contract tương tác với nhau trong giao thức (được biểu diễn bằng hình 4.2). Mỗi smart contract đảm nhận một vai trò rõ ràng và chỉ giao tiếp với các smart contract khác thông qua các interface đã được định nghĩa trước.

ProtocolController là điểm vào duy nhất (single entry point) cho mọi hành động quản trị. Tất cả các lệnh cấu hình, cập nhật tham số hay nâng cấp đều phải đi qua smart contract này trước khi được chuyển tiếp đến các smart contract khác như LendingPool, Liquidation, PriceRouter và MyOracle.

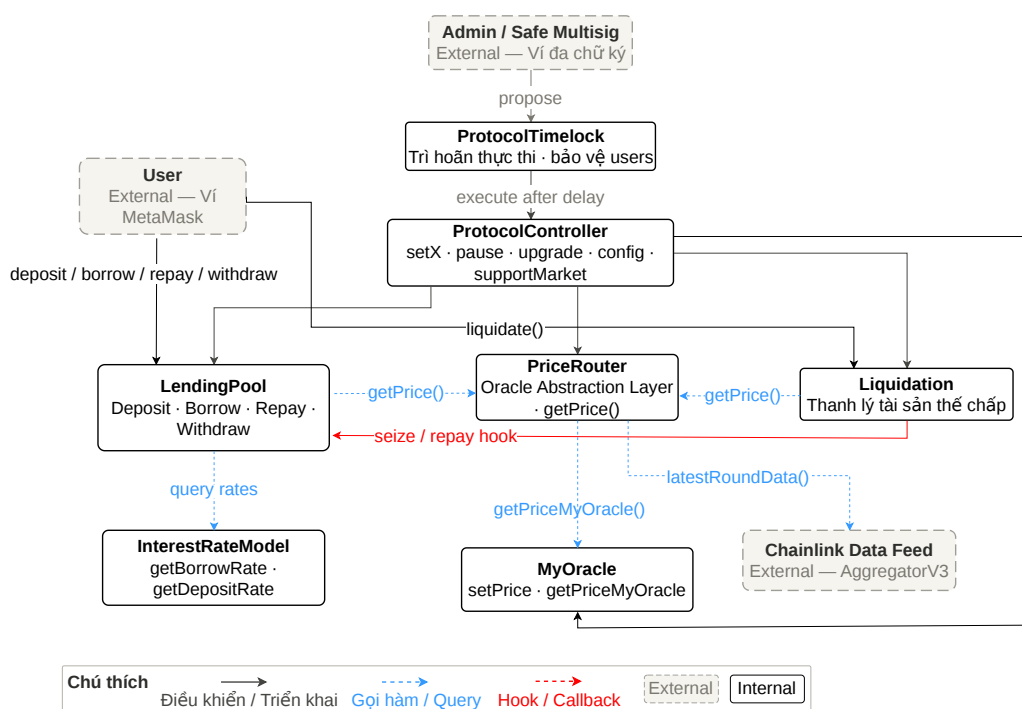
LendingPool là trung tâm của giao thức, lưu trữ toàn bộ trạng thái tài sản và vị thế người dùng. Contract này chỉ chấp nhận lệnh từ hai nguồn: ProtocolController (các hàm admin) và Liquidation (hai hook thanh lý seizeCollateral và re-

payFromLiquidation). Khi cần tính toán liên quan đến giá hoặc lãi suất, LendingPool gọi ra ngoài đến PriceRouter và InterestRateModel tương ứng.

Liquidation đọc trạng thái vị thế người dùng từ LendingPool, lấy giá từ PriceRouter, rồi gọi ngược lại hai hook của LendingPool để cập nhật số dư trong quá trình thanh lý. Contract này không tự lưu trữ bất kỳ số dư nào của người dùng.

PriceRouter là lớp trừu tượng hóa nguồn giá (Oracle Abstraction Layer), được gọi bởi cả LendingPool và Liquidation. Không smart contract nào gọi trực tiếp Chainlink hay MyOracle — mọi truy vấn giá đều đi qua PriceRouter.getPrice().

InterestRateModel là contract thuần tính toán (stateless), chỉ được gọi bởi LendingPool khi cần lấy lãi suất vay và gửi. Không phụ thuộc vào bất kỳ smart contract nào khác trong hệ thống.



Hình 4.2: Tổng quan kiến trúc smart contract

4.2.1 LendingPool / LendingPoolStorage

a, Mô hình dữ liệu (Data Model)

LendingPool là smart contract nền tảng của toàn bộ giao thức, lưu trữ tài sản của giao thức và vị thế của người dùng, smart contract này được thiết kế với hai struct chính, đầu tiên chính là Struct Market, được trình bày trong bảng 4.1, đây là trạng thái của một tài sản trong giao thức:

Trường	Ý nghĩa
isSupported	Tài sản có đang được kích hoạt không
totalDeposits	Tổng gốc + lãi tích lũy đã gửi (tăng theo thời gian)
totalBorrows	Tổng gốc + lãi tích lũy đang vay (tăng theo thời gian)
borrowIndex	Chỉ số tích lũy lãi vay (bắt đầu = 1e18)
depositIndex	Chỉ số tích lũy lãi gửi (bắt đầu = 1e18)
lastUpdateTimestamp	Thời điểm cập nhật lãi suất cuối cùng
interestRateModel	Địa chỉ smart contract tính lãi suất

Bảng 4.1: Struct Market của LendingPool

Tiếp theo là Struct Balance, được trình bày ở bảng 4.2, là vị thế của một người dùng đối với 1 tài sản

Trường	Ý nghĩa
deposited	Số dư gốc tại thời điểm snapshot
borrowed	Số nợ gốc tại thời điểm snapshot
borrowIndexSnapShot	Giá trị borrowIndex tại lần tương tác cuối
depositIndexSnapShot	Giá trị depositIndex tại lần tương tác cuối

Bảng 4.2: Struct Balance của LendingPool

Trạng thái của giao thức được lưu trữ thông qua các map và array, tiêu biểu có thể kể đến như các cấu trúc dữ liệu trong bảng 4.3.

State	Cấu trúc dữ liệu	Ý nghĩa
markets	mapping(address => Market)	Theo dõi trạng thái của từng tài sản (asset => Market)
userBalances	mapping(address => mapping(address => Balance))	Theo dõi trạng thái của từng người dùng với từng tài sản (user => asset => Balance)
userMarkets	mapping(address => address[])	Lưu trữ địa chỉ toàn bộ tài sản mà người dùng tương tác (user => list of assets)
treasuryBalances	mapping(address => uint256)	Lưu trữ ngân quỹ của từng tài sản (asset => amount)
userMarketExists	mapping(address => mapping(address => bool))	Mapping để tránh push duplicate vào userMarkets array

Bảng 4.3: Cấu trúc dữ liệu tiêu biểu

Quan trọng, LendingPoolStorage khai báo `__gap [39]` để dự trữ slot cho UUPS upgrade — đây là best practice bắt buộc khi dùng upgradeable pattern. Lý thuyết về rủi ro xung đột slot đã trình bày tại mục 3.4.4; chi tiết phân bổ slot cụ thể tại mục 6.1.3.b.

b, Cơ chế tích lũy lãi suất

Cơ chế tích lũy lãi suất theo thời gian là điểm nổi bật và quan trọng nhất của giao thức, là trái tim của contract. Hai tính chất — không kỳ hạn và lãi suất thả nổi — tạo ra bài toán tích lũy lãi liên tục: tại mỗi thời điểm, số dư thực tế của người dùng bằng số tiền gốc cộng với toàn bộ lãi đã tích lũy từ lần tương tác cuối đến hiện tại, với lãi suất có thể đã thay đổi nhiều lần trong khoảng đó. Giao thức giải quyết bài toán này thông qua cơ chế Lazy Accrual (tích lũy lãi trễ) cùng với Global Interest Index (chỉ số lãi toàn cục). Cơ chế này hoạt động dựa vào 2 struct Market (borrowIndex, depositIndex) và struct Balance (borrowIndexSnapShot, depositIndexSnapShot), Lazy Accrual đảm bảo chỉ khi có người dùng tương tác với một loại tài sản, hàm accrueInterest(address asset) mới được thực thi và tiến hành tích lũy lãi suất cho tài sản đó. Cách thực hoạt động: accrueInterest sẽ tiến hành thu thập lãi suất và tiến hành tích lũy cho số dư thực tế và chỉ số index cho một loại tài sản, bao gồm cả phía vay và phía gửi, đồng thời phần dư ra sẽ được đưa vào ngân quỹ của tài sản:

$$\text{interestAccrued} = \text{totalBorrows} \times r_{\text{borrow}} \times \frac{\Delta t}{\text{SECONDS_PER_YEAR} \times S} \quad (4.1)$$

$$\text{borrowIndex} += \text{borrowIndex} \times r_{\text{borrow}} \times \frac{\Delta t}{\text{SECONDS_PER_YEAR} \times S} \quad (4.2)$$

$$\text{depositInterest} = \text{totalDeposits} \times r_{\text{deposit}} \times \frac{\Delta t}{\text{SECONDS_PER_YEAR} \times S} \quad (4.3)$$

$$\text{depositIndex} += \text{depositIndex} \times r_{\text{deposit}} \times \frac{\Delta t}{\text{SECONDS_PER_YEAR} \times S} \quad (4.4)$$

$$\text{toTreasury} = \text{interestAccrued} - \text{depositInterest} \quad (4.5)$$

Trong đó, $S = 10^{18}$ là hằng số SCALE, Δt là số giây đã trôi qua kể từ lần cập nhật cuối, và toTreasury là phần spread giữ lại cho ngân quỹ giao thức.

Đối với vị thế của người dùng được lưu trong Balance, giao thức sẽ tiến hành tích lũy một phía (nghĩa là nếu người dùng gửi thêm hoặc rút thì tích lũy Deposit, nếu vay hoặc trả nợ thì tích lũy Borrow):

$$\text{currentUserDeposit} = \text{deposited} \times \frac{\text{depositIndex}}{\text{depositIndexSnapShot}} \quad (4.6)$$

$$\text{currentUserBorrow} = \text{borrowed} \times \frac{\text{borrowIndex}}{\text{borrowIndexSnapShot}} \quad (4.7)$$

Lý do lựa chọn Lazy Accrual thay vì Eager Accrual, cùng với phân tích so sánh Global Interest Index với các cách tiếp cận thay thế, được trình bày chi tiết tại mục 5.1. Kỹ thuật số học fixed-point phục vụ cài đặt các công thức trên được trình

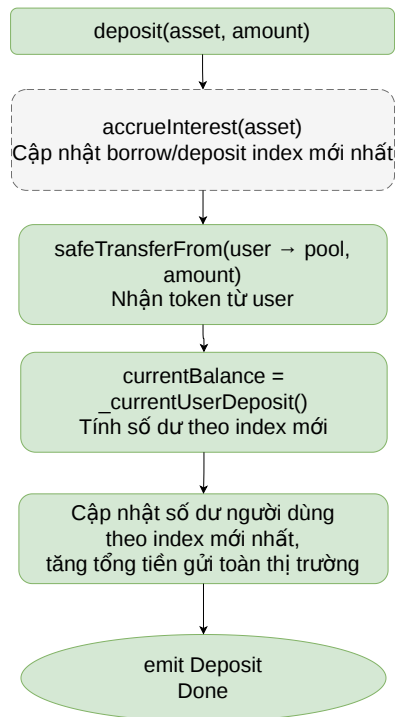
bày tại mục 6.1.3.a.

c, Các chức năng nghiệp vụ chính (Core Functions)

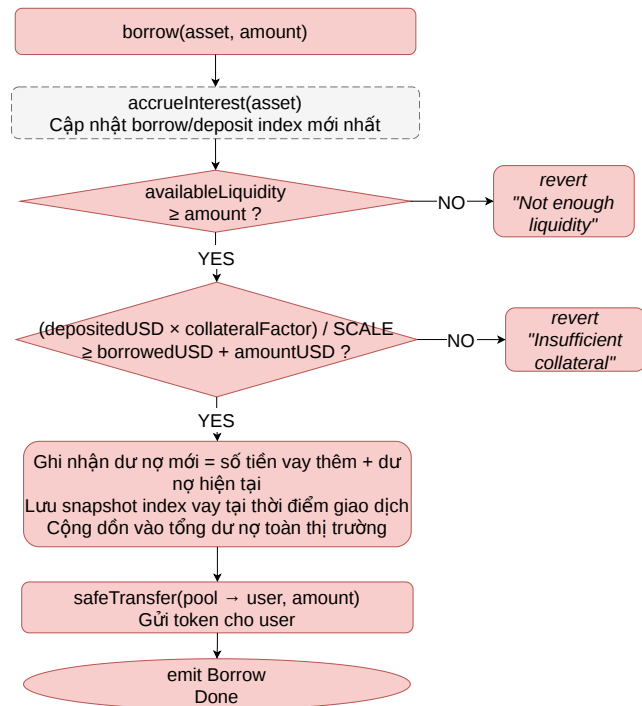
Giao thức có tổng cộng là 4 hàm nghiệp vụ chính, tương đương với 4 chức năng chính của giao thức, được liệt trong bảng 4.4, luồng hoạt động của các chức được thể hiện lần lượt ở các hình 4.3, 4.4, 4.5 và 4.6.

Hàm	Chức năng
deposit(address asset, uint amount)	Gửi tài sản vào giao thức để nhận lãi
borrow(address asset, uint amount)	Vay tài sản dựa trên tài sản thế chấp
repay(address asset, uint amount)	Trả nợ vay
withdraw(address asset, uint amount)	Rút tài sản đã gửi

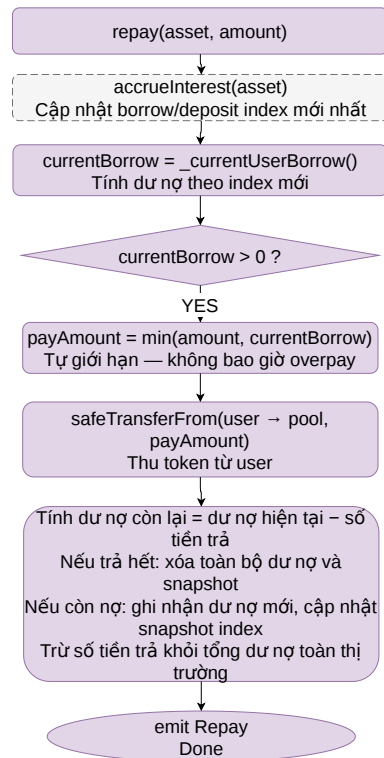
Bảng 4.4: Các hàm nghiệp vụ chính



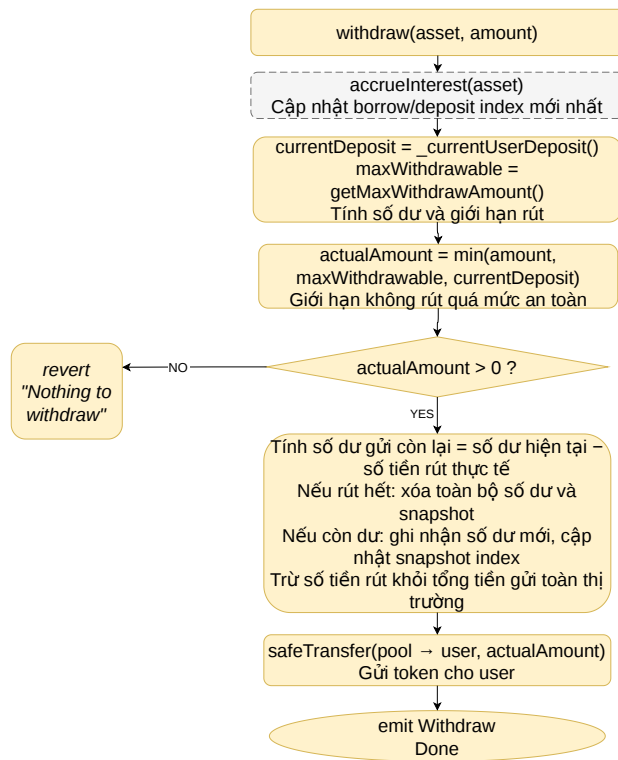
Hình 4.3: Sơ đồ luồng gửi tài sản



Hình 4.4: Sơ đồ luồng vay tài sản



Hình 4.5: Sơ đồ luồng trả nợ



Hình 4.6: Sơ đồ luồng rút tài sản

d, Cơ chế bảo mật (Security Mechanism)

Bề mặt tấn công của một lending protocol bao gồm nhiều lớp rủi ro khác nhau: reentrancy attack ở tầng EVM, chiếm quyền khởi tạo proxy, truy cập trái phép hàm admin, lỗi logic nghiệp vụ, và rút cạn thanh khoản treasury. Mỗi loại rủi ro này đòi hỏi một cơ chế phòng vệ riêng biệt — không có cơ chế nào bảo vệ được toàn bộ. Do đó, LendingPool được thiết kế bảo mật theo 5 tầng độc lập, mỗi tầng phòng vệ một lớp rủi ro cụ thể.

Tầng 1: Bảo vệ cấp contract (Contract-level), dùng các thư viện đã được audit, kiểm chứng của Openzeppelin, đầu tiên là chống lại tấn công reentrancy sử dụng ReentrancyGuardTransient, áp dụng nonReentrant cho toàn bộ các hàm state-changing (deposit, borrow, repay, withdraw, seizeCollateral, repayFromLiquidation); SafeERC20 cho tất cả lệnh transfer token đều qua safeTransferFrom / safeTransfer thay vì gọi trực tiếp transfer. Điều này xử lý các ERC20 không chuẩn: một số token không trả về bool hoặc trả về false thay vì revert khi thất bại, SafeERC20 sẽ wrap lại và revert nếu token báo lỗi; PausableUpgradeable hết hợp whenNotPaused: tất cả hàm user-facing và accrueInterest đều có guard whenNotPaused; khi admin kích hoạt pause, toàn bộ giao thức đóng băng ngay lập tức, không thể deposit, borrow, repay, withdraw, hay tích lũy lãi.

Tầng 2: Bảo vệ khởi tạo (Initialization Safety), _disableInitializers() trong con-

structor làm cho implementation contract (không phải proxy) bị khóa cứng, không thể gọi initialize(). Nếu không có dòng này, kẻ tấn công có thể khởi tạo implementation trực tiếp và chiếm quyền controller. Modifier initializer sẽ đảm bảo initialize() chỉ được gọi đúng một lần trên proxy, các lần gọi tiếp theo đều bị revert.

Tầng 3: Phân quyền truy cập (Access Control), tầng này sử dụng các modifier để định nghĩa quyền truy cập và ngăn chặn các truy cập trái phép, các modifier được thể hiện trong bảng 4.5.

Modifier	Áp dụng cho	Ý nghĩa bảo vệ
onlyController	Toàn bộ hàm admin	Chỉ ProtocolController (và sau đó là Timelock) mới được thay đổi cấu hình
onlyLiquidation	seizeCollateral, repayFromLiquidation	Chỉ Liquidation contract mới được gọi hook — không ai có thể tùy ý xóa collateral của người khác
_authorizeUpgrade override với onlyController	UUPS upgrade	Chỉ controller mới được nâng cấp implementation, không ai khác

Bảng 4.5: Các modifier phân quyền truy cập

Lưu ý thiết kế: hai hook thanh lý được bảo vệ bởi onlyLiquidation chứ không phải onlyController, vì trong runtime, controller không phải người gọi các hàm này, Liquidation contract mới là người gọi trực tiếp.

Tầng 4: Kiểm soát nghiệp vụ (Business Logic Guards), được cài đặt trực tiếp bên trong các hàm tương tác với giao thức, sử dụng để đảm bảo nghiệp vụ của giao thức được thực hiện một cách chính xác mà không có lỗi gì có thể khai thác

Trong borrow():

- Kiểm tra thanh khoản thị trường: $\text{amount} \leq \text{totalDeposits} + \text{treasury} - \text{totalBorrows}$ — không được vay quá số token thực có trong pool.
- Kiểm tra đủ tài sản thế chấp: $\text{totalDepositedUSD} \times \text{collateralFactor} / \text{SCALE} \geq \text{totalBorrowedUSD} + \text{amountUSD}$ — collateral phải đủ cover khoản vay mới.

Trong repay():

- Auto-cap số tiền trả: Nếu $\text{amount} > \text{currentBorrow}$ thì $\text{payAmount} = \text{currentBorrow}$ — người dùng không bao giờ trả nhiều hơn số nợ thực tế, tránh mất tiền do nhập nhầm số.

Trong withdraw():

- Giới hạn rút bởi collateral: $\text{actualAmount} = \min(\text{amount}, \text{getMaxWithdrawAmount}())$ — không cho phép rút phần collateral đang cover nợ, đảm bảo vị thế luôn trên ngưỡng an toàn.
- Giới hạn rút bởi số dư thực: $\text{actualAmount} = \min(\text{actualAmount}, \text{currentDeposit})$ — không rút quá số dư hiện có.

Trong `seizeCollateral()` (liquidation hook):

- $\text{currentBorrowerDeposit} \geq \text{seizeAmount}$ — không thể seize nhiều hơn số collateral borrower đang có.

Trong `repayFromLiquidation()` (liquidation hook):

- $\text{repayAmount} \leq \text{currentBorrow}$ — không thể repay nhiều hơn số nợ thực (Liquidation contract đã cap trước với $\text{maxRepayAmount}(\text{currentBorrow} \cdot \text{closeFactor})$, nhưng LendingPool vẫn kiểm tra độ lập — defense in depth).

Trong `accrueInterest()`:

- $\text{Guard timeElapsed} == 0$ — nếu đã tích lũy lãi trong cùng block thì return sớm, tránh tích lũy hai lần.

Tầng 5: Bảo vệ ngân quỹ giao thức (Protocol Treasury Safety)

`withdrawTreasury()` — trước khi rút tiền từ treasury, kiểm tra bất biến:

$$\text{totalDeposits} + \text{treasuryBalances} - \text{amount} \geq \text{totalBorrows} \quad (4.8)$$

Nghĩa là dù treasury bị rút, tổng tài sản trong pool vẫn phải đủ cover toàn bộ nợ đang được vay. Admin không thể rút cạn thanh khoản và để lại người gửi không rút được.

`rescueToken()` — chỉ cho phép rút phần surplus — token dư ra không được hạch toán trong bất kỳ vị thế nào:

$$\text{surplus} = \text{balanceOf}(\text{contract}) - (\text{totalDeposits} + \text{treasuryBalances}) \quad (4.9)$$

Token của người dùng và treasury không bao giờ bị động đến. Hàm này xử lý trường hợp ai đó gửi nhầm token vào địa chỉ contract.

Ngoài các hàm quản lý ngân quỹ dành riêng cho admin, LendingPool còn cung cấp hàm `donate(asset, amount)` cho phép bất kỳ ai đóng góp token vào treasuryBalances của một tài sản đang được hỗ trợ, mà không làm thay đổi totalDeposits hay totalBorrows, từ đó không pha loãng utilization rate của giao thức, tránh tác động

đến lãi suất của giao thức. Ứng dụng thực tế của hàm này bao gồm: nhóm phát triển bơm thêm thanh khoản dự phòng trong giai đoạn đầu khi lượng người dùng còn ít, hoặc cộng đồng bổ sung nguồn lãi để kích thích người gửi tài sản. Mặc dù mở với tất cả mọi người, hàm donate vẫn được bảo vệ bởi `nonReentrant`, `whenNotPaused` và `onlySupportedMarket`, đảm bảo không thể donate vào asset không hợp lệ hoặc khi giao thức đang tạm dừng khẩn cấp.

Phân tích lý do chọn thiết kế phân tầng thay vì validate tập trung, cùng với các quyết định thiết kế phi hiển nhiên trong cơ chế bảo mật, được trình bày tại mục 5.2.

e, Liquidation Hooks & Mô hình phân tách trách nhiệm

Hai hàm `seizeCollateral()` và `repayFromLiquidation()` chỉ được gọi bởi Liquidation contract (`onlyLiquidation`). Đây là thiết kế separation of concerns:

- LendingPool quản lý balance/state.
- Liquidation chứa toàn bộ logic thanh lý.
- LendingPool chỉ expose hook để Liquidation gọi vào cập nhật state.

f, View/Preview Functions — phục vụ UX

Nhóm hàm này không thay đổi state, dùng để frontend hiển thị real-time, thể hiện dưới bảng 4.6.

Hàm	Mục đích
<code>getAccountLiquidity()</code>	Tính tổng collateral và debt theo USD, dùng <code>_preview-MarketAfterAccrual</code> (simulate accrue không write)
<code>getMaxWithdrawAmount()</code>	Tính tối đa có thể rút mà không bị liquidation
<code>previewDeposit/Borrow/Withdraw/Repay</code>	Simulate tác động của một hành động lên vị thế của một người dùng trước khi ký giao dịch

Bảng 4.6: Các hàm preview của giao thức

g, Hệ thống sự kiện (Events) — Cầu nối on-chain và off-chain

Mỗi hành động thay đổi trạng thái trong LendingPool đều kèm theo việc phát ra một event tương ứng. Đây không chỉ là yêu cầu về tính minh bạch on-chain mà còn là cơ chế chủ động để backend index dữ liệu mà không cần polling RPC liên tục.

Các event chính được định nghĩa trong LendingPool bao gồm:

- `Deposit(user, asset, amount)`: phát ra khi người dùng gửi tài sản thành công.
- `Borrow(user, asset, amount)`: phát ra khi người dùng vay tài sản.
- `Repay(user, asset, amount)`: phát ra khi người dùng trả nợ, trong

đó amount là số tiền thực tế sau khi áp dụng auto-cap.

- `Withdraw(user, asset, amount)`: phát ra khi người dùng rút tài sản.
- `Accrue(asset, interestAccrued, toDepositors, toTreasury, ...)`: phát ra sau mỗi lần tích lũy lãi suất, mang toàn bộ thông tin về sự thay đổi của index và số dư.
- `CollateralSeized(borrower, collateralAsset, seizeAmount)`: phát ra khi tài sản thế chấp bị tịch thu trong quá trình thanh lý.
- `RepayFromLiquidation(borrower, repayAsset, repayAmount)`: phát ra khi khoản nợ được trả thay bởi liquidator.
- `TreasuryWithdrawn(asset, to, amount)`: phát ra khi admin rút tiền từ ngân quỹ.
- `Donated(donor, asset, amount)`: phát ra khi có ai đó đóng góp vào ngân quỹ giao thức.

Đáng chú ý nhất là event `Accrue`, vì nó mang theo đầy đủ thông tin sau mỗi lần tích lũy lãi — bao gồm tổng vay mới, chỉ số index mới của cả hai phía, và lượng lãi chuyển vào treasury. Nhờ đó, backend có thể tái tạo lại hoàn toàn trạng thái giao thức chỉ từ chuỗi event mà không cần truy vấn thêm bất kỳ dữ liệu nào. Đây là thiết kế hướng đến khả năng full re-sync — toàn bộ dữ liệu trong database có thể được tái tạo bằng cách replay lại toàn bộ event từ block đầu tiên mà giao thức được triển khai.

4.2.2 InterestRateModel

Đây là một smart contract có vai trò hoàn toàn độc lập trong hệ thống, không phụ thuộc bất kỳ smart contract nào, chỉ có nhiệm vụ cung cấp lãi suất vay và gửi cho `LendingPool`. Điểm nổi bật trong thiết kế đối với smart contract này chính là có thể có nhiều bản sao (instance) của `InterestRateModel` được triển khai và sử dụng, mỗi bản lại có thể có một mô hình lãi suất và các tham số lãi suất khác biệt hoàn toàn; đồng thời, trong `LendingPool`, mỗi asset có thể được gán một `InterestRateModel` riêng, cho phép tùy chỉnh chính sách lãi suất theo từng loại tài sản, điều này cho phép giao thức có thể dễ dàng mở rộng và thay đổi khi có nhiều tài sản hơn.

Hiện tại, giao thức đang sử dụng một mô hình lãi suất duy nhất, đó là mô hình Two-Slope (Kinked Rate Model). Mô hình này bao gồm 5 tham số cấu hình được biểu diễn trong bảng 4.7.

Tham số	Ý nghĩa	Ví dụ
baseRate	Lãi suất vay tối thiểu	2% (0.02e18)
rateSlope1	Độ dốc lãi suất vùng bình thường (< optimal)	8% (0.08e18)
rateSlope2	Độ dốc lãi suất vùng quá tải (> optimal) — rất cao để kích thích trả nợ	100% (1.00e18)
optimalUtilization	Ngưỡng sử dụng tối ưu	80% (0.8e18)
reserveFactor	Tỷ lệ spread giữ lại cho treasury	10% (0.1e18)

Bảng 4.7: Các tham số mô hình lãi suất

Tất cả 5 tham số đều khai báo immutable — được set 1 lần trong constructor và không bao giờ thay đổi. Để thay đổi mô hình lãi suất, admin phải deploy smart contract mới và gọi setInterestRateModel trên LendingPool — đây là thiết kế rõ ràng về governance: thay đổi IRM phải thông qua quá trình timelock, không thể thay đổi ngầm.

Công thức tính lãi suất vay (borrowRate) được biểu diễn như sau:

$$r_{\text{borrow}} = \begin{cases} r_{\text{base}} + \frac{U \times s_1}{S} & \text{nếu } U \leq U_{\text{opt}} \\ r_{\text{base}} + \frac{U_{\text{opt}} \times s_1}{S} + \frac{s_2 \times (U - U_{\text{opt}})}{S} & \text{nếu } U > U_{\text{opt}} \end{cases} \quad (4.10)$$

Trong đó $U = \text{totalBorrows} / \text{totalDeposits}$ (utilization rate), $S = 10^{18}$ (SCALE), s_1 và s_2 lần lượt là rateSlope1 và rateSlope2.

Thiết kế mấu chốt chính là khi $U > U_{\text{optimal}}$, lãi suất tăng rất nhanh (slope2 $\gg$ slope1), tạo áp lực kinh tế buộc người vay phải trả nợ sớm, tái cân bằng thanh khoản giao thức. Khi pool hoàn toàn trống và chưa có ai vay hoặc toàn bộ nợ đã được trả, tỷ lệ sử dụng vốn $U = 0$. Trong trường hợp này, hàm getBorrowRate(0) trả về 0 thay vì baseRate. Đây là thiết kế có chủ ý: khi không có ai vay, không có rủi ro thanh khoản nào cần được bù đắp, do đó lãi suất bằng 0 là hợp lý về mặt kinh tế. Hệ quả trực tiếp là depositRate cũng bằng 0, người gửi không nhận lãi nếu vốn của họ không được ai sử dụng, phản ánh đúng nguyên lý hoạt động của giao thức.

Công thức tính lãi suất gửi (depositRate)

$$r_{\text{deposit}} = \frac{r_{\text{borrow}} \times U \times (S - \text{reserveFactor})}{S^2} \quad (4.11)$$

Cách tính lãi suất của hệ thống được sử dụng nhằm 3 ý nghĩa: (i) Người gửi nhận phần lãi từ người vay, tỷ lệ với mức độ sử dụng vốn của họ; (ii) reserveFactor là phần lãi giữ lại cho treasury giao thức, đây là nguồn doanh thu của protocol; (iii)

Luôn đảm bảo $\text{depositRate} < \text{borrowRate}$ (spread dương cho protocol) .

4.2.3 Liquidation

Contract Liquidation chứa toàn bộ logic thanh lý vị thế, được tách hoàn toàn ra khỏi LendingPool, điều này cho phép logic thanh lý có thể được nâng cấp một cách độc lập (Liquidation là non-upgradeable smart contract — để thay đổi logic, admin deploy smart contract mới và cập nhật địa chỉ qua ProtocolController), và giúp LendingPool không cần biết gì về logic thanh lý, chỉ expose 2 hook seizeCollateral và repayFromLiquidation để cập nhật theo dõi trạng thái của tài sản và người dùng trong quá trình thanh lý, đây là thiết kế tách biệt trách nhiệm giúp giao thức có thể được mở rộng, bảo trì dễ dàng.

Liquidation gồm có 3 tham số rủi ro, phục vụ cho quá trình thanh lý vị thế, lần lượt được biểu diễn trong bảng 4.8.

Tham số	Ý nghĩa	Ví dụ
liquidationThreshold	Ngưỡng debt/collateral để vị thế bị liquidate	90% (0,9e18)
closeFactor	Tỷ lệ tối đa debt của borrower được phép thanh lý trong 1 lần	50% (0,5e18)
liquidationIncentive	Bonus % collateral mà liquidator nhận thêm	5% (0,05e18)

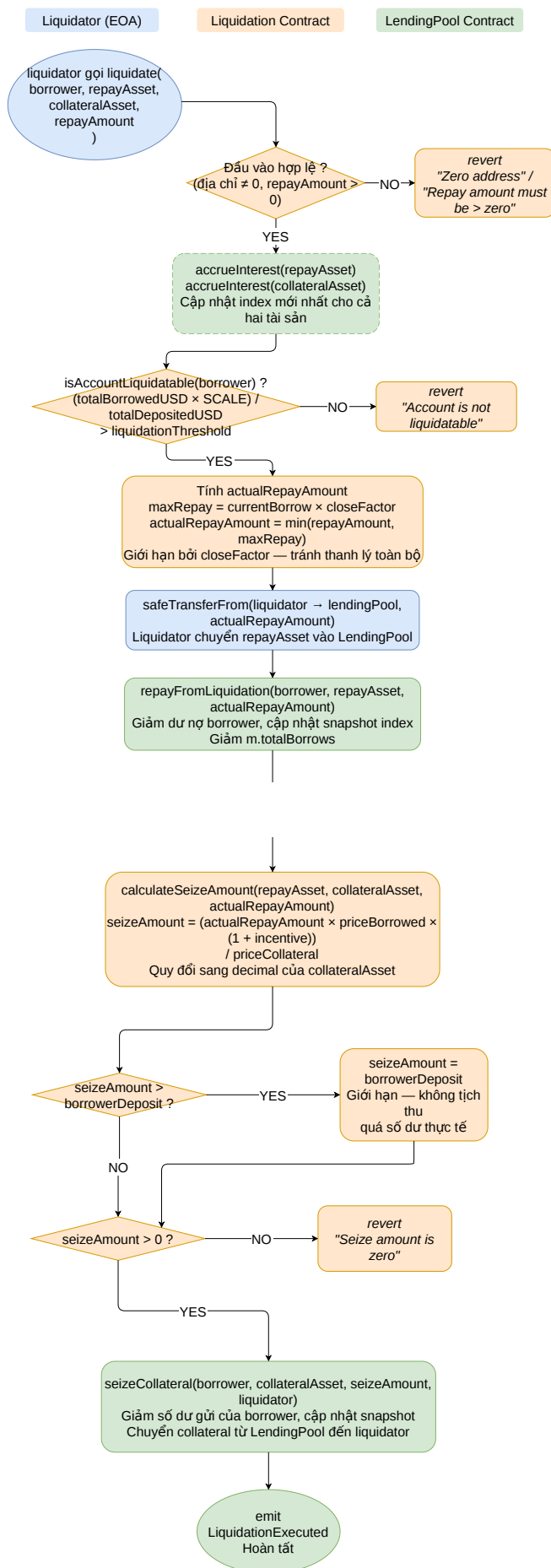
Bảng 4.8: Các tham số rủi ro

Liquidation được thiết kế để có kiểm tra xem một vị thế có bị rơi vào điều kiện để kích hoạt thanh lý hay không, vị thế có thể bị thanh lý một khi tỉ lệ debt/collateral lớn hơn liquidationThreshold, tương đương với health factor < 1 , cụ thể:

$$\text{isLiquidatable} \Leftrightarrow \frac{\text{totalBorrowedUSD} \times S}{\text{totalDepositedUSD}} > \text{liquidationThreshold} \quad (4.12)$$

$$\text{healthFactor} = \frac{\text{totalDepositedUSD} \times \text{liquidationThreshold}}{\text{totalBorrowedUSD} \times S} \quad (4.13)$$

Luồng liquidate được thể hiện thông qua sơ đồ trong hình 4.7.



Công thức tính seize amount khi liquidate sử dụng liquidationIncentive, có ý nghĩa kinh tế khi Liquidator trả X USD tiền nợ, nhận lại $X \times (1 + \text{incentive})$ USD tài sản thế chấp, đây là lợi nhuận của liquidator, đồng thời là động lực để duy trì sức khỏe giao thức.

Giao thức xử lý cẩn thận trường hợp tài sản có số decimals khác nhau (ví dụ USDC = 6, WETH = 18), bằng cách chuẩn hóa tất cả về 18 decimals để tính toán rồi sau đó convert ngược về decimals gốc của tài sản khi trả kết quả.

$$\text{repayAmount}_{18} = \text{repayAmount} \times 10^{(18-d_{\text{repay}})} \quad (\text{chuẩn hóa lên 18 decimals}) \quad (4.14)$$

$$\text{seizeAmount}_{18} = \text{repayAmount}_{18} \times (1 + \text{incentive}) \times \frac{p_{\text{borrowed}}}{p_{\text{collateral}}} \quad (4.15)$$

$$\text{seizeAmount} = \frac{\text{seizeAmount}_{18}}{10^{(18-d_{\text{collateral}})}} \quad (\text{convert về collateral decimals}) \quad (4.16)$$

4.2.4 PriceRouter / PriceRouterStorage

Contract này đóng vai trò là một tầng Oracle Abstraction Layer, đóng vai trò trung gian giữa giao thức và nguồn giá, tất cả các smart contract cần giá tài sản (LendingPool, Liquidation) đều gọi qua PriceRouter.getPrice(), không bao giờ gọi trực tiếp oracle. Thiết kế này mang đến tính linh hoạt cho giao thức: Có thể thêm nguồn giá mới mà không cần sửa LendingPool/Liquidation, đồng thời PriceRouter cũng được cài đặt UUPS, có thể upgrade logic mà không đổi địa chỉ.

Mô hình dữ liệu của PriceRouter gồm có

```
enum Source { CHAINLINK, MYORACLE, NONE }

struct FeedInfo {
    Source source;           // nguồn giá
    address feedOrToken;     // Chainlink feed address hoặc
    token address (cho MyOracle)
}

mapping(address => FeedInfo) feeds; // asset => nguồn giá
__gap[47] // du trừ storage slot cho UUPS upgrade, tổng 50
slot (3 biến + 47 gap).
```

PriceRouter của giao thức được sử dụng để lưu trữ nguồn giá của một loại asset, sử dụng Chainlink hoặc nguồn giá từ MyOracle, trong môi trường Production, cần

phải sử dụng Chainlink hoặc các dịch vụ Oracle on-chain tương tự.

```
feeds[asset].source == NONE          -> revert "No price feed
    available"
feeds[asset].source == CHAINLINK -> gọi
    AggregatorV3Interface.latestRoundData()
feeds[asset].source == MYORACLE  -> gọi MyOracle.
    getPriceMyOracle(asset)
```

Đặc biệt, đối với Chainlink Feed, giao thức có cơ chế bảo vệ khi xảy ra Stale Price, giá on-chain bị chậm và không còn được cập nhật kịp thời, smart contract PriceRouter thực hiện 3 lớp validation đối với giá từ Chainlink:

```
(
    uint80 roundId,
    int chainlinkPrice,
    ,
    uint updatedAt,
    uint80 answeredInRound
) = priceFeed.latestRoundData();

require(updatedAt > 0, "Round not complete");
require(answeredInRound >= roundId, "Stale price");
require(chainlinkPrice > 0, "Invalid price from Chainlink
    ");
```

Mặc dù PriceRouter đã triển khai ba lớp kiểm tra cơ bản đối với giá từ Chainlink, giao thức hiện chưa xác minh khoảng thời gian tối đa kể từ lần cập nhật giá gần nhất (staleness timeout). Ba kiểm tra hiện có đảm bảo giá hợp lệ về mặt cấu trúc — round hoàn tất, round không bị lỗi, giá dương — nhưng không ngăn được trường hợp node Chainlink ngừng cập nhật trong nhiều giờ liên tiếp. Khi đó, giá cũ vẫn được chấp nhận, dẫn đến rủi ro định giá sai tài sản và có thể gây ra thanh lý không chính xác. Kiểm tra còn thiếu là: `require(block.timestamp - updatedAt ≤ MAX_STALENESS)`. Đây là giới hạn đã được nhận diện trong phạm vi đề án; trong môi trường production, cần bổ sung tham số MAX_STALENESS phù hợp với heartbeat của từng Chainlink feed, thường dao động từ 1 đến 24 giờ tùy loại tài sản theo tài liệu chính thức của Chainlink.

Cuối cùng, nguồn giá được quản lý bằng các hàm admin của giao thức (được biểu diễn trong bảng 4.9).

Hàm	Mục đích
setChainlinkFeed(asset, feed)	Gán Chainlink AggregatorV3 feed cho asset
setMyOracleFeed(asset)	Cài đặt asset dùng MyOracle
removeFeed(asset)	Xóa nguồn giá (asset sẽ không thể giao dịch)

Bảng 4.9: Các hàm quản lý nguồn giá

4.2.5 Lớp quản trị: ProtocolController + ProtocolTimelock

Quản trị là điểm tập trung rủi ro lớn nhất trong một DeFi protocol: nếu quyền admin bị compromise, kẻ tấn công có thể thay đổi toàn bộ tham số, rút treasury, hoặc upgrade smart contract sang implementation độc hại trong một giao dịch duy nhất. Phân tích các rủi ro cụ thể của mô hình single admin và lý do lựa chọn kiến trúc quản trị 2 tầng (Multisig + Timelock) được trình bày tại mục 5.5. Phần này mô tả thiết kế kỹ thuật của từng thành phần.

Toàn bộ giao thức được thiết kế theo kiến trúc quản trị 2 tầng, mọi hành động của admin, trước khi được thực thi đều phải đi qua Multisig Wallet và Timelock [10], [16]

```
[Admin / Multisig Wallet]
  | proposal
[ProtocolTimelock]          <- ap dat do tre thoi gian (
    minDelay)
  | execute (sau minDelay)
[ProtocolController]        <- single entry point quan tri
  | goi
[LendingPool / Liquidation / PriceRouter / MyOracle]
```

a, ProtocolController — Single Entry Point Pattern

ProtocolController là điểm vào duy nhất cho admin, kế thừa AccessControl của OpenZeppelin [10]. Các hàm quản trị hệ thống trong ProtocolController được biểu diễn trong bảng 4.10.

Nhóm	Ví dụ hàm	Tác động
Quản lý địa chỉ smart contract	setLendingPool, setPriceRouter, setLiquidation	Cập nhật địa chỉ đồng bộ nhiều smart contract
Quản lý tài sản	supportMarket, unsupportMarket	Thêm/xóa tài sản khỏi giao thức
Cấu hình giá	setChainlinkFeed, setMyOracleFeed, setPrice	Quản lý nguồn giá tài sản
Cấu hình rủi ro	setCollateralParams, setLiquidationParams	Điều chỉnh tham số an toàn
Khẩn cấp	pauseLendingPool, unpauseLendingPool	Dừng/tiếp tục giao thức
Tài chính	withdrawTreasury, rescueToken	Quản lý ngân quỹ
Nâng cấp	upgradeLendingPool, upgradePriceRouter	Nâng cấp UUPS implementation
Di chuyển	migrateController	Chuyển quyền điều khiển sang Controller mới

Bảng 4.10: Các hàm quản trị admin

ProtocolController có một cơ chế thiết kế đặc biệt, đó là Atomic Multi-Contract Update, cập nhật đồng thời nhiều smart contract cùng lúc, và đảm bảo tất cả đều được cập nhật giống nhau, ví dụ trong việc cập nhật địa chỉ smart contract PriceRouter:

```
setPriceRouter(_priceRouter) :
    -> ILiquidation.setPriceRouter(_priceRouter) // 1 call
    -> ILendingPool.setPriceRouter(_priceRouter) // 1 call
    -> priceRouter = _priceRouter // cập nhật local
```

Việc này đảm bảo không bao giờ có trạng thái "một smart contract dùng PriceRouter mới, smart contract kia vẫn dùng cũ". Nhiều hành động được đảm bảo thực hiện cùng nhau hoặc đều fail, ví dụ khi thêm một tài sản mới, đồng thời gọi supportMarket của LendingPool và setChainlinkFeed của PriceRouter:

```
supportMarketWithChainlinkFeed(asset, irm, feed) :
    -> LendingPool.supportMarket(asset, irm) // thêm market
    -> PriceRouter.setChainlinkFeed(asset, feed) // đăng ký nguồn giá
```

Đảm bảo này có căn cứ kỹ thuật từ tính nguyên tử (atomicity) của transaction trên Ethereum: toàn bộ các lệnh trong một transaction đều được thực thi hoặc đều bị revert cùng nhau, không có trạng thái trung gian nào được lưu lại trên blockchain nếu transaction thất bại. Do đó, nếu một lệnh cập nhật tại một smart contract nào

đó revert vì bất kỳ lý do gì, toàn bộ transaction revert — mọi smart contract đều giữ nguyên trạng thái cũ.

Quyết định thiết kế Atomic Multi-Contract Update và tầm quan trọng của nó trong bối cảnh quản trị phi tập trung được phân tích tại mục 5.5.

b, ProtocolTimelock

Đây là một smart contract rất đặc biệt, smart contract này không thêm bất kỳ một logic mới nào, hoàn toàn kế thừa TimelockController của OpenZeppelin [10]:

```
contract ProtocolTimelock is TimelockController {
    constructor(
        uint256 minDelay,
        address[] memory proposers,
        address[] memory executors,
        address admin
    ) TimelockController(minDelay, proposers, executors,
        admin) {}
}
```

Mục đích là để có địa chỉ smart contract riêng trên blockchain để track on-chain, đặt tên rõ ràng (ProtocolTimelock) để Etherscan verify và cộng đồng nhận biết, đồng thời cấu hình vai trò qua constructor: proposers (ai được đề xuất), executors (ai được thực thi). Khi deploy smart contract, timelock được cài đặt là admin của ProtocolController, tất cả các hàm của ProtocolController đều phải đi qua time-lock. Luồng hoạt động của timelock rất đơn giản, dùng để chắc chắn không hành động nào của ProtocolController được thực thi ngay lập tức, mà phải chờ đợi qua minDelay:

1. Proposer gọi schedule(target, value, data, salt, delay) -> đang ký hành động cho
2. Cho minDelay (ví dụ: 48 giờ)
3. Executor gọi execute(target, value, data, predecessor, salt) -> thực thi hành động

4.3 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu PostgreSQL được sử dụng với mục đích chính là để sao chép và tổ chức lại dữ liệu từ blockchain nhằm phục vụ truy vấn nhanh và hiển thị trên giao diện người dùng. Database không được xem là nguồn dữ liệu gốc (source of truth), toàn bộ dữ liệu đều có thể được tái tạo lại bằng cách replay lại các sự kiện đã phát sinh trên blockchain từ đầu. Hệ thống bao gồm tổng cộng 15 bảng, được chia thành 4 nhóm theo chức năng, sơ đồ ERD được thể hiện bởi hình 4.8.

4.3.1 Nhóm 1: Dữ liệu nghiệp vụ giao thức (Protocol Business Data)

Bảng `assets` lưu thông tin về các tài sản từng hỗ trợ trong giao thức. Mỗi tài sản được xác định duy nhất bởi cặp (`assetAddress`, `chainId`), cùng với ký hiệu (`symbol`), tên (`name`), số chữ số thập phân (`decimals`), trạng thái hỗ trợ hiện tại (`isSupported`), tổng số đã gửi (`totalDeposited`), tổng số đã vay (`totalBorrowed`), tổng số đã rút (`totalWithdrawn`), tổng số đã trả (`totalRepaid`), và số dư ngân quỹ giao thức (`treasuryBalance`). Số dư được lưu dạng `DECIMAL(78,0)` để đảm bảo không mất độ chính xác với số nguyên 256-bit của Solidity.

Bảng `users` lưu thông tin về các địa chỉ ví đã từng tương tác với giao thức, gồm địa chỉ ví (`userAddress`), mạng blockchain (`chainId`), email đăng ký nhận thông báo (`nullable`), và thời điểm lần đầu tham gia giao thức (`joinedAt`).

Bảng `user_assets` lưu vị thế hiện tại của một người dùng đối với một tài sản cụ thể. Ngoài số dư đang gửi (`depositedAmount`) và số dư đang vay (`borrowedAmount`), bảng còn lưu `lastSyncedBlock` và `lastSyncedLogIndex` — block và log index của lần quét cuối cùng — để đảm bảo indexer có thể xử lý lại đúng thứ tự khi khởi động lại.

Bảng `transactions` lưu lịch sử toàn bộ giao dịch đã thực hiện trong giao thức. Mỗi bản ghi gồm hash giao dịch (duy nhất), người thực hiện (`userId`), tài sản liên quan (`assetId`), loại giao dịch (`type`: `DEPOSIT/BORROW/REPAY/WITHDRAW/LIQUIDATE`), số lượng token (`amount`), giá trị quy đổi USD tại thời điểm thực hiện (`amountUSD`) và số block (`blockNumber`).

Bảng `asset_configs` lưu lịch sử thay đổi tham số cấu hình giao thức cho từng tài sản. Mỗi bản ghi ghi lại đầy đủ 9 tham số tại một thời điểm: 5 tham số mô hình lãi suất (`baseRate`, `slope1`, `slope2`, `optimalUtilization`, `reserveFactor`) và 4 tham số rủi ro (`collateralFactor`, `closeFactor`, `liquidationIncentive`, `liquidationThreshold`). Khi admin thay đổi bất kỳ tham số nào, một bản ghi mới được tạo ra thay vì cập nhật bản ghi cũ, nhờ đó cho phép truy vết lịch sử cấu hình đầy đủ.

4.3.2 Nhóm 2: Dữ liệu log và snapshot (Historical & Time-series Data)

Bảng `accrue_logs` lưu chi tiết mỗi lần tích lũy lãi suất (event `Accrue` từ smart contract). Mỗi bản ghi gồm tài sản liên quan (`assetId`), hash giao dịch, số block, tổng lãi đã tích lũy (`interestAccrued`), phần lãi chia cho người gửi (`toDepositors`), phần lãi vào treasury (`toTreasury`), và giá trị mới của tất cả các chỉ số quan trọng (`newTotalBorrows`, `newBorrowIndex`, `newTotalDeposits`, `newDepositIndex`). Bảng này cho phép backend tái tạo lại toàn bộ lịch sử lãi suất và phục vụ hiển thị biểu đồ biến động lãi theo thời gian.

Bảng `treasury_logs` lưu lịch sử các thay đổi số dư ngân quỹ giao thức, bao gồm các sự kiện rút tiền (`TreasuryWithdrawn`) và donate vào ngân quỹ (`Donated`). Mỗi bản ghi ghi lại loại sự kiện (`eventType`), số lượng (`amount`), số dư sau thay đổi (`balanceAfter`), địa chỉ nguồn (`fromAddress`) và địa chỉ đích (`toAddress`).

Bảng `asset_snapshots` lưu trạng thái tổng hợp của từng tài sản tại một block cụ thể, được tạo ra hằng ngày bởi tác vụ cron. Mỗi bản ghi ghi lại `totalDeposited`, `totalBorrowed`, `treasuryBalance`, `utilizationRate`, `depositRate` và `borrowRate` tại thời điểm snapshot. Dữ liệu này phục vụ hiển thị biểu đồ TVL (Total Value Locked) và lịch sử lãi suất trên giao diện người dùng.

Bảng `user_snapshots` lưu trạng thái tổng hợp của từng người dùng tại một block cụ thể, cũng được tạo ra hằng ngày. Mỗi bản ghi ghi lại tổng giá trị đang gửi (`totalDepositedUSD`), tổng giá trị đang vay (`totalBorrowedUSD`), giá trị ròng (`netWorthUSD`) và hệ số sức khỏe (`healthFactor`) của vị thế người dùng tại thời điểm đó.

4.3.3 Nhóm 3: Dữ liệu quản trị (Governance Data)

Bảng `proposals` lưu thông tin về các đề xuất quản trị được tạo thông qua Safe Multisig và Timelock. Mỗi proposal ghi lại địa chỉ smart contract mục tiêu (`target`), dữ liệu hàm được gọi ở dạng raw hex (`calldata`), trạng thái xử lý (`status`: 0-proposed / 1-scheduled / 2-executed / 3-cancelled), số chữ ký hiện tại (`currentSigners`), ngưỡng chữ ký cần thiết (`multisigThreshold`), hash giao dịch Safe (`safeTxHash`), hash giao dịch timelock (`timelockExecutedTxHash`), thời điểm có thể thực thi (`eta`) và ID thao tác trên Timelock (`operationId`). Đặc biệt, trường `decodedAction` lưu JSON đã được giải mã từ `calldata`, cho phép frontend hiển thị hành động admin một cách có thể đọc được thay vì hiển thị chuỗi hex thô.

4.3.4 Nhóm 4: Dữ liệu hạ tầng hệ thống (Infrastructure State)

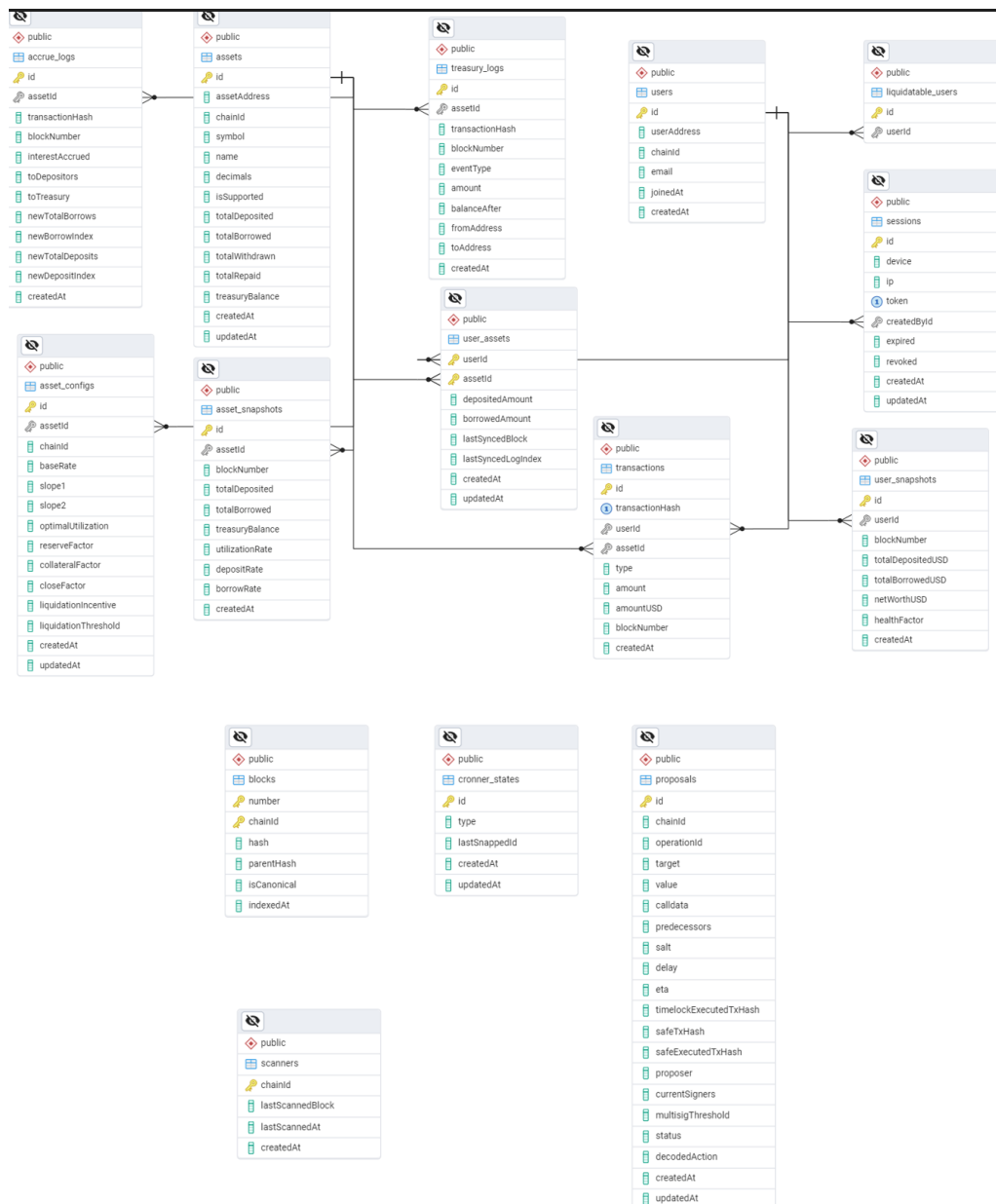
Bảng `scanners` lưu trạng thái tiến độ quét blockchain theo từng mạng (`chainId`). Mỗi bản ghi chứa `lastScannedBlock` — block cuối cùng đã xử lý thành công — và `lastScannedAt`. Khi tiến trình indexer khởi động lại sau sự cố, nó đọc bảng này để biết phải tiếp tục từ đâu, đảm bảo không bỏ sót block nào.

Bảng `cronner_states` lưu tiến trình snapshot hằng ngày của tác vụ cron. Mỗi bản ghi theo dõi loại snapshot (`type`: 0 = user snapshot, 1 = asset snapshot) và ID cuối cùng đã được snapshot (`lastSnappedId`). Cơ chế này cho phép tác vụ snapshot tiếp tục từ điểm dừng nếu bị gián đoạn giữa chừng, tránh phải xử lý lại từ đầu.

Bảng `sessions` lưu thông tin phiên đăng nhập của người dùng sau khi xác thực bằng chữ ký ví (SIWE). Mỗi session ghi lại token (duy nhất), thiết bị (`device`), địa

chỉ IP (ip), người dùng sở hữu (createdById), thời điểm hết hạn (expired) và trạng thái thu hồi (revoked). Bảng hỗ trợ đa thiết bị — một người dùng có thể có nhiều session hoạt động đồng thời trên các thiết bị khác nhau.

Bảng liquidatable_users lưu danh sách các vị thể hiện đang đủ điều kiện bị thanh lý. Đây là bảng trạng thái động, được làm mới hoàn toàn mỗi 30 giây bởi tiến trình blc-indexer sau khi tính toán health factor của tất cả người dùng đang có nợ. Thông tin này được push real-time đến frontend qua WebSocket để hiển thị cảnh báo và hỗ trợ liquidator thực hiện thanh lý kịp thời.



Hình 4.8: Sơ đồ ERD của hệ thống

4.4 Thiết kế backend

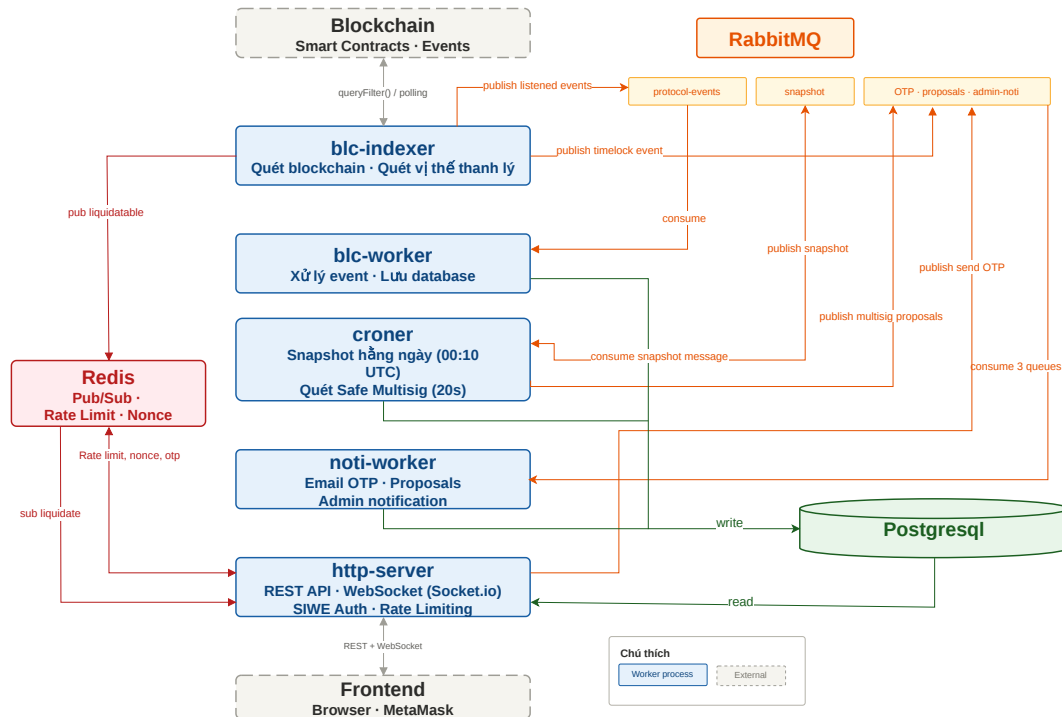
4.4.1 Tổng quan kiến trúc — Mô hình đa tiến trình Event-Driven

Trước khi mô tả kiến trúc, cần nêu rõ bài toán mà kiến trúc này giải quyết. Hệ thống backend có 5 tác vụ với tính chất hoàn toàn khác nhau: quét blockchain liên tục, xử lý event bất đồng bộ, snapshot định kỳ, gửi thông báo, và phục vụ HTTP request. Nếu chạy chung một process, một tác vụ bị block hoặc crash sẽ kéo theo toàn bộ hệ thống.

Backend không được xây dựng theo mô hình monolith truyền thống mà theo kiến trúc đa tiến trình, hướng sự kiện (multi-process event-driven). Hệ thống gồm 5 worker độc lập, mỗi worker chạy trong một process riêng và giao tiếp với nhau qua RabbitMQ message queue [14] và Redis pub/sub. Không worker nào gọi trực tiếp vào tiến trình khác, toàn bộ phối hợp đều thông qua message. Cách các worker giao tiếp với nhau và với redis cùng message queue được trình bày bằng hình 4.9.

Ưu điểm của kiến trúc này là mỗi tiến trình có thể được scale, khởi động lại hoặc nâng cấp độc lập mà không ảnh hưởng đến các tiến trình khác; kết hợp việc sử dụng RabbitMQ có khả năng hồi phục, đảm bảo không bị mất mát dữ liệu.

Phân tích lý do lựa chọn kiến trúc đa tiến trình thay vì monolith, cùng với lý do sử dụng RabbitMQ thay vì gọi trực tiếp, được trình bày tại mục 5.3.



Hình 4.9: Sơ đồ tổng quan hệ thống backend

4.4.2 Blockchain indexer (blc-indexer)

Đây là worker cốt lõi nhất của hệ thống backend, hoạt động như một "máy quét blockchain" liên tục, được cài đặt với mô hình mỗi chain tương ứng một cron job gọi `scanChain()` để xử lý các block mới của chain đó. Chu kỳ của cron job được cấu hình riêng theo block time của từng chain, ví dụ 12 giây với Ethereum Mainnet/Sepolia và 3,5 giây với BSC, tránh quét vô nghĩa khi chưa có block mới, đồng thời tạo nền tảng để mở rộng sang multi-chain mà không cần thay đổi logic lõi.

Trước khi mô tả luồng xử lý, cần biết trước một quyết định thiết kế nền tảng: indexer sử dụng mô hình polling (`queryFilter`) thay vì subscribe event theo thời gian thực. Phân tích chi tiết và so sánh hai hướng tiếp cận này được trình bày tại mục 5.4. Ngắn gọn, lý do cốt lõi là polling cung cấp khả năng phục hồi trạng thái hoàn toàn sau downtime trong khi subscribe event không đảm bảo được tính chất này.

Luồng xử lý chính của `scanChain()` gồm 4 bước tuần tự:

Bước 1 — Đọc điểm bắt đầu: Khi worker khởi động lần đầu, nếu bảng scanners chưa được khởi tạo cho chain đó, hệ thống đặt `lastScannedBlock = currentBlock -`

10 làm điểm xuất phát. Ở các lần chạy tiếp theo, worker truy vấn bảng scanners để lấy lastScannedBlock, từ đó tính $\text{fromBlock} = \text{lastScannedBlock} + 1$. Giới hạn trên toBlock được tính như sau:

```
toBlock = min(currentBlock, fromBlock + MAX_BLOCK_RANGE - 1);
```

Giới hạn MAX_BLOCK_RANGE xử lý trường hợp worker bị downtime dài — khi đó khoảng cách giữa fromBlock và currentBlock có thể lên đến hàng nghìn block, và quét toàn bộ một lần có thể vi phạm rate limit của nhà cung cấp RPC.

Bước 2 — Quét events: Gọi đồng thời (Promise.all) 3 hàm quét trên cùng khoảng block

fromBlock, toBlock

(i) scanLendingPoolContract quét toàn bộ event của LendingPool (Deposit, Borrow, Repay, Withdraw, Accrue, CollateralSeized, ...), (ii) scanLiquidationContract quét event thanh lý và (iii) scanTimelockContract quét event quản trị từ ProtocolTimelock (CallScheduled, CallExecuted, Cancelled), Mỗi hàm sử dụng `contract.queryFilter("*", fromBlock, toBlock)` để lấy toàn bộ event trong khoảng, sau đó sắp xếp theo blockNumber và logIndex tăng dần để đảm bảo xử lý đúng thứ tự thời gian.

Bước 3 — Dispatch event: Mỗi event được tra cứu trong bảng protocolEventHandlers - một map từ tên event sang handler function tương ứng. Nếu tìm thấy handler, event được đẩy vào message queue RabbitMQ để blc-worker xử lý bất đồng bộ; nếu không tìm thấy, event bị bỏ qua và ghi log cảnh báo. Thiết kế tách biệt giữa việc indexer chỉ phát hiện và dispatch, worker mới xử lý nghiệp vụ.

Bước 4 — Lưu block headers và cập nhật scanner: Toàn bộ thao tác ghi trong bước này được bao trong một Sequelize database transaction duy nhất: lưu block headers (number, hash, parentHash) vào bảng blocks và cập nhật lastScannedBlock trong bảng scanners. Tính atomicity đảm bảo không xảy ra trạng thái không nhất quán, hoặc cả hai thao tác đều thành công, hoặc đều rollback.

4.4.3 Cơ chế quét và phát hiện vị thế thanh lý

Mỗi 30 giây, blc-indexer chạy thêm một cron job để phát hiện các vị thế có thể bị thanh lý. Thuật toán sử dụng cursor-based pagination thay vì load toàn bộ người dùng vào bộ nhớ một lần — an toàn khi số lượng người dùng lớn:

Bước 1: Truy vấn database theo từng batch 100 người dùng có borrowedAmount > 0 (đang có nợ), sắp xếp theo userId tăng dần, dùng userId cuối của batch làm

cursor cho lần tiếp theo.

Bước 2: Với mỗi địa chỉ ví trong batch, gọi `isAccountLiquidatable(address)` trực tiếp trên blockchain để kiểm tra health factor.

Bước 3: Nếu ví thể có thể bị thanh lý, upsert vào bảng `liquidatable_users`. Nếu không còn đủ điều kiện, xóa khỏi bảng.

Bước 4: Sau khi quét xong toàn bộ, publish sự kiện `LiquidatableUsersUpdated` vào Redis pub/sub, kèm danh sách địa chỉ và block number hiện tại. HTTP server nhận event này qua Redis subscription và push ngay đến frontend qua WebSocket.

4.4.4 Tiến trình Blockchain Worker (blc-worker)

Tiến trình này lắng nghe queue RabbitMQ và xử lý các message do blc-indexer publish sau khi phát hiện event. Kiến trúc consumer-handler tách biệt hai trách nhiệm: `BLCConsumerService` chịu trách nhiệm lấy message từ queue và acknowledge; `BLCWorkerHandler` chịu trách nhiệm xử lý nghiệp vụ thực tế (parse event, lưu database).

Thiết kế này cho phép tăng số lượng blc-worker instance để xử lý song song khi lượng event lớn, mà không cần thay đổi blc-indexer.

4.4.5 Tiến trình Cron (croner) và Snapshot hằng ngày

Tiến trình croner đảm nhận 2 tác vụ định kỳ:

Snapshot hằng ngày (00:10 UTC): Tiến trình publish message vào RabbitMQ cho blc-worker xử lý — snapshot trạng thái hiện tại của từng tài sản (lãi suất, TVL, utilization rate) và từng người dùng (totalDepositedUSD, totalBorrowedUSD, healthFactor) vào bảng `asset_snapshots` và `user_snapshots`. Trạng thái tiền độ được lưu vào bảng `cronner_states` — nếu bị gián đoạn giữa chừng, lần chạy tiếp theo tiếp tục từ ID cuối cùng đã snapshot thay vì chạy lại từ đầu.

Quét Safe Multisig (mỗi 20 giây): Gọi Safe API để lấy danh sách giao dịch đang chờ ký từ ví đa chữ ký của admin, cập nhật số chữ ký hiện tại và trạng thái vào bảng `proposals`. Nhờ đó, frontend có thể hiển thị lịch sử hành động admin một cách minh bạch mà không cần kết nối trực tiếp đến Safe API.

4.4.6 Tiến trình Notification Worker (noti-worker)

Tiến trình này lắng nghe đồng thời 3 loại message từ RabbitMQ:

Email consumer: Xử lý yêu cầu gửi email xác thực OTP khi người dùng đăng ký email nhận thông báo. Sử dụng Nodemailer với Email SMTP.

Proposal consumer: Nhận event về proposal mới từ blc-indexer, cập nhật trạng thái proposal vào database và thông báo cho admin.

Admin notification consumer: Theo dõi các thay đổi quan trọng của giao thức (thay đổi tham số, tạm dừng giao thức, thêm/xóa tài sản...) và gửi email thông báo đến tất cả người dùng đã đăng ký. Đây là lý do tại sao bảng users có trường email nullable — chỉ người dùng đã đăng ký mới nhận được thông báo.

4.4.7 HTTP Server — REST API và WebSocket

HTTP server phục vụ frontend theo hai nhóm API có vai trò khác nhau. Nhóm thứ nhất là API dữ liệu giao thức (protocol data APIs), dùng để đọc dữ liệu mà backend đã index từ blockchain và xử lý như thông tin tài sản, lịch sử giao dịch, snapshot và proposal quản trị. Với nhóm này, blockchain vẫn là nguồn dữ liệu gốc duy nhất (source of truth), còn backend chỉ đóng vai trò lớp truy vấn và tăng tốc hiển thị; vì vậy các endpoint thuộc nhóm dữ liệu giao thức chỉ cung cấp thao tác đọc, không có endpoint tạo, sửa hay xóa trạng thái giao thức. Nhóm thứ hai là API hỗ trợ ngoài chuỗi (off-chain support APIs), phục vụ các nhu cầu hạ tầng của ứng dụng như xác thực SIWE, quản lý session, gửi/ kiểm tra OTP và đăng ký email nhận thông báo. Các API này có thể sử dụng POST vì chúng thao tác lên dữ liệu phụ trợ ngoài chuỗi (nonce, session, OTP, email registration), không làm thay đổi trạng thái tài chính của giao thức trên blockchain. Các nhóm endpoint chính của HTTP server được trình bày trong bảng 4.11.

Nhóm	Prefix	Mô tả
Tài sản	/api/assets	Danh sách tài sản, thông số thị trường
Giao dịch	/api/transactions	Lịch sử giao dịch, cần xác thực
Người dùng	/api/users	Vị thế cá nhân, cần xác thực
Snapshot	/api/snapshots	Dữ liệu lịch sử cho biểu đồ
Proposals	/api/proposals	Danh sách hành động admin
Logs	/api/logs	Lịch sử tích lũy lãi suất
Xác thực	/api/auth	SIWE, refresh/revoke session
Email	/api/email	Gửi OTP, xác thực OTP, đăng ký email

Bảng 4.11: Các nhóm endpoint của hệ thống

Mỗi request HTTP đi qua một pipeline middleware theo thứ tự: (1) IP Rate Limiter kiểm tra số request/phút từ IP (Redis sliding window) → (2) CORS kiểm tra origin → (3) Auth Middleware (chỉ với endpoint yêu cầu xác thực) tra cứu session token trong database, xác nhận chưa hết hạn và chưa bị thu hồi → (4) Route Handler xử lý nghiệp vụ và trả response. Server áp dụng IP rate limiting toàn cục 60 request/phút/IP (sliding window, lưu trong Redis) để bảo vệ API. Endpoint OTP có rate limit riêng nghiêm ngặt hơn để chống brute force, cụ thể 60s 1 lần gửi OTP và tối đa 5 lần thử OTP.

Các endpoint thuộc nhóm dữ liệu giao thức (assets, snapshots, proposals, logs)

không yêu cầu xác thực — bất kỳ ai cũng có thể đọc dữ liệu công khai của giao thức. Các endpoint thuộc nhóm người dùng (transactions cá nhân, ví thể cá nhân) và off-chain support (email, session) yêu cầu token hợp lệ trong Authorization header.

WebSocket server (Socket.io) nhận event từ Redis pub/sub do bkc-indexer publish và forward đến frontend đang kết nối — kiến trúc hoàn toàn stateless, cho phép chạy nhiều instance HTTP server song song mà WebSocket vẫn hoạt động đúng nhờ Redis làm trung gian.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

5.1 Cơ chế tích lũy lãi Lazy Accrual + Global Interest Index

Cơ chế tích lũy lãi suất là thành phần quan trọng bậc nhất của giao thức vay và cho vay phi tập trung. Mục 4.2.1.b đã mô tả chi tiết cơ chế hoạt động của Lazy Accrual kết hợp Global Interest Index, bao gồm công thức tính `accrueInterest` và cách cập nhật vị thế người dùng. Cách tiếp cận chỉ số lãi toàn cục được tham chiếu từ các giao thức lending phổ biến như Compound v2 [5]. Mục này tập trung phân tích lý do lựa chọn cơ chế Lazy Accrual thay vì các giải pháp thay thế, cùng với các tính chất mà Global Interest Index đảm bảo. Bài toán cốt lõi là mỗi lần cập nhật trạng thái on-chain đều tốn chi phí gas (đặc biệt SSTORE tốn xấp xỉ 20.000 gas — xem mục 3.1.2), vậy làm thế nào để tích lũy lãi cho n người dùng mà vẫn tối ưu chi phí.

Đây chính là lý do hệ thống lựa chọn giải pháp Lazy Accrual thay cho Eager Accrual. Với cách tiếp cận Eager Accrual, mỗi khi có một giao dịch làm thay đổi trạng thái của market, giao thức phải cập nhật lại số dư đã tích lũy lãi của toàn bộ người dùng đang tham gia market đó, chứ không chỉ của người vừa tương tác. Điều này có nghĩa là chi phí xử lý của một lần thay đổi lãi suất sẽ tăng theo số lượng người dùng đang hoạt động trong market. Nếu ký hiệu U là số người dùng và T là số lần market phát sinh thay đổi cần cập nhật lãi suất trong một giai đoạn, thì tổng khối lượng cập nhật trạng thái sẽ tỷ lệ với $U \times T$. Nói cách khác, càng nhiều người dùng tham gia và càng nhiều giao dịch xảy ra, lượng storage write mà giao thức phải thực hiện càng tăng nhanh. Trong môi trường blockchain, nơi mỗi lần ghi storage đều tốn gas đáng kể, cách tiếp cận này không còn phù hợp ở quy mô lớn. Vì vậy, giao thức sử dụng Lazy Accrual: chỉ khi một người dùng trực tiếp tương tác, hệ thống mới tính và cập nhật phần lãi liên quan đến người đó, thay vì cập nhật đồng loạt cho toàn bộ market.

Tuy đã giải quyết được bài toán về chi phí cho hệ thống, nhưng Lazy Accrual lại mang đến một vấn đề khó khăn hơn khi nghiệp vụ hệ thống yêu cầu 2 ràng buộc liên quan đến lãi suất. Ràng buộc thứ nhất là không kỳ hạn: người dùng có thể gửi hoặc rút tại bất kỳ block nào, không thể biết trước thời điểm tương tác. Ràng buộc thứ hai là lãi suất thả nổi: lãi suất thay đổi mỗi khi tỷ lệ sử dụng vốn thay đổi, tức là thay đổi mỗi khi có bất kỳ giao dịch nào của bất kỳ người dùng nào xảy ra trên cùng một tài sản. Hệ quả là trong khoảng thời gian giữa 2 lần tương tác của người dùng A, lãi suất có thể đã thay đổi hàng chục lần do các giao dịch của người dùng B, C, D. Vậy câu hỏi đặt ra là: làm thế nào để đảm bảo tính công bằng, tính đúng

số tiền lãi mà người dùng A được nhận mà không cần lưu lịch sử toàn bộ các lần thay đổi lãi suất, đồng thời tối ưu chi phí gas mà người dùng phải chịu mỗi lần thực hiện Lazy Accrual.

Đối với bài toán này có thể nghĩ đến 3 cách tiếp cận như sau: Cách tiếp cận thứ nhất là cập nhật số dư của tất cả người dùng mỗi khi lãi suất thay đổi. Cách này cho kết quả chính xác nhưng chi phí $O(n)$ storage write mỗi giao dịch, với n người dùng, mỗi giao dịch deposit của bất kỳ ai cũng phải cập nhật số dư của tất cả người còn lại, hoàn toàn không khả thi trên blockchain nơi mỗi storage write tốn gas. Cách tiếp cận thứ hai là lưu toàn bộ lịch sử thay đổi lãi suất theo thời gian và tính tổng tích phân khi người dùng rút tiền. Cách này tránh được $O(n)$ write nhưng chi phí đọc $O(k)$ với k là số lần thay đổi lãi suất có thể lên đến hàng nghìn lần trong vài ngày, và gas chi phí đọc lịch sử tăng tuyến tính theo thời gian người dùng gửi tiền. Cách tiếp cận thứ ba, cũng là giải pháp được giao thức lựa chọn là Global Interest Index: thay vì lưu lịch sử, giao thức duy trì một chỉ số tích lũy tăng dần liên tục. Mỗi khi lãi suất thay đổi, chỉ số này được cập nhật một lần duy nhất ($O(1)$). Người dùng chỉ cần lưu snapshot của chỉ số tại thời điểm gửi tiền; số lãi tích lũy được tính bằng tỷ lệ giữa chỉ số hiện tại và snapshot, bất kể lãi suất đã thay đổi bao nhiêu lần ở giữa.

Thiết kế này đảm bảo được 3 tính chất đồng thời: (i) chính xác: người dùng nhận đúng lãi tương ứng với thời gian gửi thực tế tính đến giây, bất kể lãi suất thả nổi biến động như thế nào; (ii) công bằng: không có sự khác biệt giữa người gửi sớm và muộn, mỗi người nhận lãi đúng theo phần vốn và thời gian của mình; (iii) tiết kiệm gas: chi phí tích lũy lãi không phụ thuộc vào số người dùng trong hệ thống, mỗi giao dịch chỉ cập nhật 2 giá trị index toàn cục thay vì n số dư người dùng, từ đó giảm tải đáng kể chi phí gas mà người dùng phải chi trả.

5.2 Kiến trúc bảo mật 5 tầng của LendingPool

Mục 4.2.1.d đã mô tả chi tiết kiến trúc bảo mật 5 tầng của LendingPool. Mục này tập trung phân tích lý do thiết kế và ba quyết định phi hiển nhiên.

Một cách tiếp cận phổ biến khi bảo mật smart contract là tập trung toàn bộ kiểm tra vào một modifier tổng hợp hoặc một hàm validate duy nhất. Cách này đơn giản khi cài đặt nhưng tạo ra single point of failure: nếu logic validate có lỗi hoặc bị bypass, toàn bộ hệ thống mất bảo vệ. LendingPool áp dụng nguyên tắc defense in depth - phòng vệ theo chiều sâu bằng cách phân chia bảo mật thành 5 tầng hoàn toàn độc lập. Mỗi tầng phòng vệ một loại rủi ro cụ thể, và sự thất bại của một tầng không làm suy yếu các tầng còn lại. Kẻ tấn công phải vượt qua đồng thời tất cả 5 tầng để có thể gây hại cho giao thức.

Quyết định thiết kế phi hiển nhiên thứ nhất là sử dụng `onlyLiquidation` thay vì `onlyController`. Hai hook thanh lý `seizeCollateral()` và `repayFromLiquidation()` được bảo vệ bởi modifier `onlyLiquidation` thay vì `onlyController` như toàn bộ các hàm admin khác. Lý do là trong runtime, `ProtocolController` không phải người gọi các hàm này, `Liquidation contract` mới là người gọi trực tiếp trong quá trình thanh lý. Nếu dùng `onlyController`, `ProtocolController` sẽ phải làm trung gian cho mọi lần thanh lý, tăng chi phí gas không cần thiết và vi phạm nguyên tắc *separation of concerns*: `ProtocolController` chỉ nên là entry point cho hành động quản trị, không phải cho hành động nghiệp vụ thời gian thực. Hệ quả của thiết kế này là `LendingPool` biết chính xác ai được phép gọi từng hàm: admin đi qua `Controller`, liquidation đi qua `Liquidation contract`, hai luồng hoàn toàn tách biệt, không thể nhầm lẫn hay lạm dụng.

Quyết định thiết kế thứ hai nằm ở việc kiểm tra trùng lặp có chủ ý: Trong tầng logic nghiệp vụ, cả `LendingPool` lẫn `Liquidation contract` đều kiểm tra `seizeAmount ≤ currentBorrowerDeposit`, dù `Liquidation` đã cap giá trị này trước khi gọi hook. Tương tự, cả hai đều kiểm tra `repayAmount ≤ currentBorrow`. Đây không phải lỗi thiết kế hay code thừa, đây là defense in depth có chủ ý ở tầng business logic. Lý do là nếu trong tương lai `Liquidation contract` được thay thế bằng một implementation mới có lỗi logic hoặc bị upgrade sai, `LendingPool` vẫn có lớp bảo vệ độc lập. Hai smart contract phòng vệ độc lập nhau đảm bảo ngay cả khi một bên bị compromise, bên còn lại vẫn giữ được tính toàn vẹn của trạng thái.

Quyết định thiết kế phi hiển nhiên cuối cùng nằm ở tầng khởi tạo contract khi sử dụng “`_disableInitializers()`” trong constructor. Với UUPS proxy pattern, tồn tại một attack vector ít được biết đến: kẻ tấn công gọi `initialize()` trực tiếp trên implementation contract (không phải proxy), chiếm quyền controller của implementation, sau đó dùng quyền đó để upgrade proxy sang một implementation độc hại. Lỗ hổng này không hiển nhiên vì trong điều kiện bình thường, người dùng chỉ tương tác qua Proxy, implementation trông có vẻ vô hại. Tuy nhiên vì implementation cũng là một smart contract on-chain với địa chỉ công khai, bất kỳ ai cũng có thể gọi hàm của nó. Giải pháp là gọi `_disableInitializers()` trong constructor của implementation. Lệnh này khóa cứng mọi khả năng khởi tạo trên implementation contract ngay khi deploy [10].

5.3 Kiến trúc backend đa tiến trình event-driven

Mục 4.4.1 đã mô tả kiến trúc đa tiến trình event-driven với 5 worker độc lập (`blc-indexer`, `blc-worker`, `croner`, `noti-worker`, `http-server`) và cách chúng phối hợp qua RabbitMQ và Redis (Hình 4.10). Mục này phân tích lý do kiến trúc tại sao

không sử dụng monolith thuần túy.

Xây dựng backend dưới dạng một ứng dụng monolith duy nhất là cách tiếp cận đơn giản với một process vừa quét blockchain, vừa xử lý event, vừa chạy cron job, vừa phục vụ HTTP request. Tuy nhiên, cách này tạo ra 3 vấn đề nghiêm trọng. Thứ nhất là tính sẵn sàng: nếu blockchain node RPC bị chậm khiến indexer block, toàn bộ HTTP server cũng bị ảnh hưởng, người dùng không truy cập được giao diện chỉ vì một vấn đề với RPC provider. Thứ hai là khả năng mở rộng: khi lượng event tăng cao, không thể scale riêng phần xử lý event mà không scale cả HTTP server đang chạy tốt. Kiến trúc nhiều worker này tăng tối đa khả năng mở rộng theo chiều dọc của hệ thống, nếu một worker là không đủ chịu tải, trong khi server còn nhiều tài nguyên dư thừa, ta có thể dễ dàng thêm nhiều instance của worker đó, từ đó tăng khả năng xử lý của một nhiệm vụ. Thứ ba là độ phức tạp: code xử lý blockchain, code nghiệp vụ database, và code phục vụ HTTP request trộn lẫn nhau, khó bảo trì và dễ gây side effect không lường trước.

Tại sao lại cần message queue (RabbitMQ) thay vì xử lý trực tiếp? Khi indexer phát hiện một event từ blockchain, cách đơn giản nhất là gọi thẳng hàm xử lý trong cùng process. Cách này đồng bộ và dễ debug, nhưng tạo ra vấn đề mất dữ liệu: nếu worker crash ngay sau khi indexer đã xử lý event nhưng trước khi lưu vào database, event đó biến mất hoàn toàn. RabbitMQ giải quyết vấn đề này thông qua cơ chế acknowledgment: message chỉ bị xóa khỏi queue sau khi worker gửi ack xác nhận đã xử lý thành công. Nếu worker crash trước khi ack, RabbitMQ tự động requeue message, event sẽ được xử lý lại khi worker khởi động trở lại [14]. Đây là đảm bảo at-least-once delivery, phù hợp với dữ liệu tài chính nơi mất event là không chấp nhận được. Hệ quả trực tiếp là hệ thống có khả năng self-healing: bất kỳ worker nào crash và restart đều tự động tiếp tục từ điểm bị gián đoạn mà không cần can thiệp thủ công.

At-least-once delivery dẫn đến khả năng xử lý trùng lặp, cơ chế idempotency để xử lý hệ quả này đã được trình bày về mặt lý thuyết tại mục 3.6.3, chi tiết cài đặt với unique constraint tại mục 6.2.3.b.

5.4 Thiết kế blockchain indexer: Polling thay vì Subscribe Event

Bài toán khó của việc index blockchain để lưu trữ dữ liệu chính là việc blockchain indexer cần liên tục theo dõi các event phát sinh từ smart contract, xử lý và lưu trữ vào database. Câu hỏi thiết kế cốt lõi là: dùng cơ chế nào để nhận event từ blockchain.

Đây là quyết định có ảnh hưởng lớn đến tính tin cậy của toàn bộ hệ thống backend, vì một event bị bỏ sót đồng nghĩa với dữ liệu trong database lệch khỏi trạng

thái thực tế on-chain, sai lệch không thể tự phát hiện và sửa chữa nếu không có cơ chế phục hồi. Ban đầu có hai giải pháp có thể được nghĩ tới, một là thiết kế indexer theo hướng Subscribe Event (`eth_subscribe` / `WebSocket Listener`) và hai là theo hướng Polling theo khoảng block với `queryFilter`. Hai giải pháp này được so sánh ngắn gọn trong bảng 5.1.

Đối với giải pháp đầu tiên, thực hiện kết nối `WebSocket` đến `RPC node` và đăng ký lắng nghe event theo thời gian thực thông qua `eth_subscribe`. Khi smart contract phát ra event, `RPC node` đẩy ngay về client mà không cần client hỏi. Ưu điểm của giải pháp này chính là độ trễ cực thấp, gần như tức thì, event được nhận gần như ngay lập tức sau khi block được mint. Nhưng với ưu điểm không có nhiều hấp dẫn, nhược điểm của nó lại cực kỳ nghiêm trọng: Đầu tiên là mất event khi kết nối đứt do kết nối `WebSocket` không ổn định, bị ngắt do mạng, timeout, worker bị chết hoặc `RPC node` restart. Trong khoảng thời gian kết nối bị đứt và kết nối lại, toàn bộ event phát sinh đều bị mất vĩnh viễn. Thư viện `ethers.js` có cơ chế reconnect tự động nhưng không lưu lại event đã bỏ lỡ trong khoảng downtime [12]. Thứ hai, khi đã mất event rồi, thì lại không có cơ chế hồi phục bởi consumer không biết mình đã nhận event đến block nào. Khi worker khởi động lại sau sự cố, không có cách nào xác định điểm tiếp tục, chỉ có thể bắt đầu lắng nghe từ block hiện tại, để lại khoảng trống dữ liệu không thể khôi phục.

Vì vậy, giải pháp thể hiện ưu điểm tối ưu hơn hoàn toàn, và được lựa chọn chính là giải pháp còn lại: sử dụng croner chạy định kỳ quét blockchain. Blockchain indexer sẽ định kỳ gọi `contract.queryFilter("*", fromBlock, toBlock)` để lấy toàn bộ event trong một khoảng block xác định. Trạng thái tiến độ được lưu bền vững vào database qua `lastScannedBlock`. Nhược điểm dễ nhận thấy so với giải pháp đầu đó là Latency cao hơn, event được phát hiện muộn hơn tối đa một chu kỳ cron (12 giây với Ethereum). Tuy nhiên, với ứng dụng lending protocol, độ trễ này hoàn toàn chấp nhận được: các hành động như gửi, vay, thanh lý không đòi hỏi phản ứng trong millisecond. Trong khi đó, thiết kế polling cho phép hệ thống phục hồi hoàn toàn sau downtime: Khi worker khởi động lại sau sự cố bất kể bao lâu, nó đọc `lastScannedBlock` từ database và tiếp tục chính xác từ block tiếp theo. Không có event nào bị bỏ sót, mọi block trong khoảng thời gian downtime đều được quét đầy đủ khi worker hoạt động trở lại. Bên cạnh đó, nó còn cho phép kiểm soát lưu lượng RPC với `MAX_BLOCK_RANGE` giới hạn số block mỗi lần quét, tránh vi phạm rate limit khi worker cần bắt kịp sau downtime dài đồng thời dễ debug và kiểm tra khi mà trạng thái tiến độ (`lastScannedBlock`) hiển thị rõ ràng trong database, tại bất kỳ thời điểm nào đều biết chính xác indexer đã xử lý đến block nào và đang xử lý block nào.

Tiêu chí	Subscribe Event	Polling + queryFilter
Latency nhận event	Rất thấp (< 1s)	Thấp ($\leq$ block time)
Phục hồi sau downtime	Không — mất event trong khoảng downtime	Hoàn toàn — tiếp tục từ lastScannedBlock
Ổn định kết nối	Phụ thuộc WebSocket — dễ mất kết nối	HTTP request độc lập — không có kết nối dài hạn
Kiểm soát lưu lượng RPC	Khó — event đến bất kỳ lúc nào	Dễ — MAX_BLOCK_RANGE giới hạn mỗi lần quét
Khả năng debug	Khó — luồng event bất đồng bộ	Dễ — trạng thái lưu rõ ràng trong database

Bảng 5.1: So sánh hai hướng tiếp cận index blockchain

5.5 Kiến trúc quản trị 2 tầng: Multisig và Timelock

Mục 4.2.5 đã trình bày thiết kế kỹ thuật của ProtocolController (Single Entry Point, Atomic Multi-Contract Update) và ProtocolTimelock (kế thừa OpenZeppelin TimelockController). Mục này phân tích lý do kiến trúc tại sao cần Multisig và Timelock, thay vì mô hình single admin.

Mô hình quản trị đơn giản nhất là một địa chỉ admin duy nhất có toàn quyền thay đổi giao thức, tuy nhiên mô hình này lại dính phải điểm yếu chí mạng: single point of failure. Mô hình này có 3 rủi ro nghiêm trọng: Thứ nhất là rủi ro mất private key: nếu key của admin bị mất, không ai có thể quản lý giao thức, không thể thêm tài sản mới, không thể xử lý khẩn cấp, không thể upgrade khi phát hiện lỗi hổng. Thứ hai là rủi ro bị compromise: nếu key bị lộ hoặc bị tấn công, kẻ tấn công có thể thực hiện bất kỳ hành động quản trị nào ngay lập tức, không có cơ chế nào để chặn lại. Thứ ba là thiếu minh bạch: cộng đồng không có cách nào biết trước các thay đổi sắp xảy ra, không có thời gian để phát hiện và phản ứng với các thay đổi bất thường.

Để giải quyết các rủi ro này, hệ thống quyết định triển khai sử dụng ví đa chữ ký đi kèm với timelock. Vậy làm thế nào multisig có thể giải quyết được rủi ro thứ nhất và rủi ro thứ hai. Safe Multisig yêu cầu chữ ký từ k trong n người nắm giữ key trước khi một hành động được thực thi [16]. Thiết kế này loại bỏ single point of failure theo cả hai chiều: mất một key không làm mất quyền quản trị (vì vẫn còn $k-1$ key khác), và lộ một key không đủ để kẻ tấn công thực hiện hành động (vì cần đủ k chữ ký). Điều quan trọng cần nhấn mạnh là Multisig không phải là tính năng tiện lợi, đây là yêu cầu bảo mật tối thiểu cho bất kỳ DeFi protocol nào quản lý tài sản của người dùng. Thiếu Multisig đồng nghĩa với việc toàn bộ tài sản trong giao thức phụ thuộc vào tính an toàn của một private key duy nhất.

Timelock đóng vai trò trong việc xử lý rủi ro còn lại trong quản trị. Nó giải quyết vấn đề minh bạch bằng cách áp đặt độ trễ bắt buộc giữa lúc đề xuất và lúc thực thi [10]. Trong khoảng thời gian này, toàn bộ hành động admin đều public trên blockchain, bất kỳ ai cũng có thể đọc và kiểm tra. Cộng đồng có đủ thời gian để phát hiện thay đổi bất thường và phản ứng: rút tài sản, đề xuất cancel, hoặc cảnh báo công khai. Ngoài vai trò minh bạch, Timelock còn là lớp phòng vệ thứ hai cho trường hợp Multisig bị compromise: kẻ tấn công có đủ k chữ ký vẫn phải chờ minDelay trước khi hành động có hiệu lực, tạo ra cửa sổ thời gian để phát hiện và phản ứng.

Một hệ quả quan trọng của kiến trúc này là Atomic Multi-Contract Update. Vì tất cả hành động admin đều đi qua một entry point duy nhất là ProtocolController, một hành động có thể cập nhật nhiều smart contract trong cùng một transaction on-chain. Ví dụ khi thêm tài sản mới, supportMarketWithChainlinkFeed() gọi đồng thời LendingPool.supportMarket() và PriceRouter.setChainlinkFeed(), đảm bảo không bao giờ có trạng thái trung gian: tài sản được thêm vào LendingPool nhưng chưa có nguồn giá trong PriceRouter, hoặc ngược lại. Nếu một trong hai lệnh fail, cả transaction bị revert, đây là tính atomicity của blockchain được tận dụng triệt để vào logic quản trị.

Đối với thư viện timelock on-chain, OpenZeppelin đã cung cấp sẵn TimelockController có thể dùng trực tiếp mà không cần deploy smart contract riêng. Việc tạo ProtocolTimelock kế thừa lại mà không thêm logic mới nhìn có vẻ thừa. Lý do thực tế có 2 mặt. Về mặt kỹ thuật, smart contract riêng có địa chỉ on-chain riêng, frontend, backend và các công cụ monitoring có thể theo dõi đúng smart contract này mà không nhầm với các TimelockController khác có thể tồn tại trên cùng mạng. Về mặt cộng đồng, khi smart contract được verify trên Etherscan với tên ProtocolTimelock, người dùng nhìn vào transaction history ngay lập tức hiểu đây là timelock của giao thức, tăng niềm tin và tính minh bạch.

CHƯƠNG 6. CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG

6.1 Hợp đồng thông minh

6.1.1 Công nghệ sử dụng

Các hợp đồng thông minh của hệ thống Lending Pool được phát triển trên nền tảng Ethereum, sử dụng các công nghệ và công cụ sau:

- Solidity ^0.8.28: Ngôn ngữ lập trình bậc cao dùng để xây dựng smart contract chạy trên Ethereum Virtual Machine (EVM). Phiên bản 0.8.28 bao gồm nhiều cải tiến về bảo mật so với các phiên bản cũ hơn, đặc biệt là kiểm tra overflow mặc định và cú pháp custom error tiết kiệm gas [2].
- Hardhat ^2.26.5: Framework phát triển smart contract chạy trên Node.js, hỗ trợ biên dịch, kiểm thử, deploy, verify smart contract, cũng như mô phỏng và fork blockchain phục vụ quá trình phát triển [15].
- OpenZeppelin Contracts ^5.4.0 và Contracts Upgradeable ^5.6.1: Bộ thư viện smart contract chuẩn đã qua kiểm định bảo mật, được sử dụng cho các tính năng: UUPS upgradeable proxy, ReentrancyGuardTransient, SafeERC20, PausableUpgradeable, AccessControl, và TimelockController [10].
- Chainlink Contracts ^1.5.0: Interface AggregatorV3 để đọc Chainlink Data Feed [13].
- Hardhat Gas Reporter ^2.3.0: Đo lường chi phí gas của từng function trong test.
- TypeScript^5.9.3 + Ethers.js^6.15.0: Ngôn ngữ lập trình có kiểu tĩnh và thư viện tương tác blockchain, được dùng cho toàn bộ script deploy và test suite [12].
- Chai ^4.5.0 + Mocha: Assertion framework dùng cho kiểm thử tự động unit test.

6.1.2 Cấu trúc codebase

```
.
|-- contracts
|   |-- InterestRateModel.sol
|   |-- LendingPool.sol
|   |-- LendingPoolStorage.sol
|   |-- Liquidation.sol
|   |-- MyOracle.sol
|   |-- PriceRouter.sol
|   |-- PriceRouterStorage.sol
```

```

|   |-- ProtocolController.sol
|   |-- ProtocolTimelock.sol
|   |-- interfaces
|   |   `-- Interfaces.sol
|   `-- test
|   `-- MockERC20.sol
|-- deployments
|   |-- abis.json
|   `-- addresses.json
|-- hardhat.config.ts
|-- scripts
|   |-- deploy-protocol
|   |   |-- config.ts
|   |   |-- deploy.ts
|   |   |-- phases
|   |   |   |-- index.ts
|   |   |   |-- phase1.ts
|   |   |   |-- phase2.ts
|   |   |   `-- phase3.ts
|   |   `-- utils
|   |   |-- abis.ts
|   |   |-- index.ts
|   |   `-- save.ts
|   `-- deploy-safe.ts
|-- test
|   |-- Fixture.test.ts
|   |-- InterestRateModel.test.ts
|   |-- LendingPool.test.ts
|   |-- Liquidation.test.ts
|   `-- ProtocolController.test.ts
`-- tsconfig.json

```

6.1.3 Cài đặt và các quyết định kỹ thuật quan trọng

a, Xử lý số học fixed-point với uint256

Vấn đề: Solidity không hỗ trợ số thực (floating point). Phép tính lãi suất như $\text{borrowRate} \times \text{utilization} / \text{SECONDS_PER_YEAR}$ phải được thực hiện hoàn toàn bằng số nguyên mà không mất độ chính xác. Đây là nền tảng kỹ thuật của cơ chế tích lũy lãi Lazy Accrual được phân tích tại mục 5.1.

Giải pháp: Sử dụng $\text{SCALE} = 1e18$ như một hằng số biểu diễn số 1,0 trong không gian fixed-point. Mọi tỷ lệ phần trăm, lãi suất và hệ số đều được lưu dưới

dạng số nguyên nhân với $1e18$. Quy tắc thực hiện phép tính: nhân trước, chia sau để tránh mất độ chính xác.

```
// Dung: nhan truoc, chia sau
result = (a * b) / SCALE

// Sai: chia truoc, nhan sau
result = (a / SCALE) * b
```

Ví dụ: lãi suất vay = 5% $\rightarrow$ lưu là $0.05 \times 10^{18} = 5 \times 10^{16}$. Khi tính $\text{interest} = \text{principal} \times \text{rate} / \text{SCALE}$, kết quả trả về đúng số tiền lãi tính bằng token gốc (với 18 decimals).

Đây là nền tảng kỹ thuật triển khai của mục 5.1 (Cơ chế tích lũy lãi Lazy Accrual).

b, Thiết kế storage gap tránh xung đột slot trong UUPS

Vấn đề: Khi upgrade implementation mới, nếu thêm biến state mới vào cuối smart contract LendingPool (không phải LendingPoolStorage), thứ tự slot bị dịch chuyển do cách Solidity kế thừa storage. Điều này dẫn đến đọc sai dữ liệu sau khi upgrade.

Giải pháp: Khai báo `__gap[39]` ở cuối LendingPoolStorage, dự trữ 39 slot cho các biến tương lai. Hiện tại LendingPoolStorage sử dụng 11 biến (11 slot), tổng cộng $11 + 39 = 50$ slot được dự trữ cho smart contract. PriceRouterStorage tương tự với `__gap[47]` (3 biến + 47 gap = 50 slot). Khi cần thêm biến mới, thu nhỏ `__gap[ ]` tương ứng — thứ tự các biến hiện tại không bị ảnh hưởng.

Con số 50 slot là convention được OpenZeppelin khuyến nghị trong tài liệu chính thức về Upgradeable Contracts, đủ cho hầu hết các trường hợp mở rộng trong vòng đời của một giao thức [10]. Khi cần thêm biến state mới trong một lần upgrade, chỉ cần khai báo biến mới ngay trước `__gap` và giảm kích thước `__gap` tương ứng — thứ tự các biến hiện tại không bị thay đổi và không có xung đột slot nào xảy ra.

Tham chiếu lý thuyết: Xem mục 3.4.4 về Storage Layout và rủi ro xung đột slot trong UUPS.

6.1.4 Quá trình triển khai

Sau khi hoàn tất cài đặt và kiểm thử, các smart contract được triển khai lên mạng Sepolia (Ethereum Testnet) theo quy trình 3 giai đoạn được tổ chức thành deploy script TypeScript riêng biệt, không sử dụng Hardhat Ignition. Lý do lựa chọn custom deploy script thay vì Ignition là vì Hardhat Ignition không hỗ trợ

@openzeppelin/hardhat-upgrades — plugin cần thiết để deploy UUPS proxy và kiểm tra storage layout tự động khi upgrade. Toàn bộ script được điều phối bởi deploy.ts, nhận thông tin cấu hình từ config.ts theo từng mạng (chainId), sau đó thực thi tuần tự 3 phase:

Phase 1 — Deploy: Triển khai tất cả smart contract lên blockchain theo thứ tự phụ thuộc:

1. Deploy 2 instance InterestRateModel: một cho stablecoin (USDC) và một cho tài sản biến động (WETH, WBTC) với tham số baseRate, slope1, slope2, optimalUtilization, reserveFactor
2. Deploy MyOracle — oracle tự quản lý để có thể tự đặt giá trong môi trường testnet.
3. Deploy PriceRouter dưới dạng UUPS proxy bằng upgrades.deployProxy() — plugin tự động deploy cả proxy contract lẫn implementation contract, kiểm tra storage layout, và lưu lại địa chỉ implementation để phục vụ upgrade sau này. Địa chỉ implementation được lấy qua upgrades.erc1967.getImplementationAddress().
4. Deploy LendingPool dưới dạng UUPS proxy tương tự, nhận địa chỉ PriceRouter và collateralFactor ngay trong lúc initialize().
5. Deploy Liquidation là smart contract thông thường (non-upgradeable), nhận địa chỉ PriceRouter, LendingPool và 3 tham số rủi ro: liquidationThreshold, closeFactor, liquidationIncentive.
6. Deploy ProtocolTimelock với cấu hình: proposer là địa chỉ Safe Multisig (đọc từ biến môi trường SAFE_ADDRESS), executor là address(0) (open execution — bất kỳ ai cũng có thể execute sau khi hết minDelay), và minDelay đọc từ TIMELOCK_MIN_DELAY_SECONDS [16].
7. Deploy ProtocolController nhận địa chỉ của tất cả smart contract vừa deploy và địa chỉ Timelock làm admin.

Sau khi deploy xong, địa chỉ mỗi smart contract (bao gồm cả proxy và implementation riêng biệt đối với LendingPool và PriceRouter) được lưu vào file deployments/addresses.json

Phase 2 — Wire: Kết nối các smart contract với nhau sau khi toàn bộ đã có địa chỉ. Trong phase này, LendingPool.setLiquidation() được gọi để LendingPool biết địa chỉ Liquidation — cần thiết để onlyLiquidation modifier hoạt động đúng.

Phase 3 — Migrate: Chuyển giao quyền kiểm soát từ deployer sang ProtocolController. Lần lượt gọi setController(controllerAddress) trên 4 smart contract:

MyOracle, PriceRouter, LendingPool và Liquidation. Sau bước này, deployer không còn quyền admin trực tiếp — mọi hành động quản trị đều phải đi qua ProtocolController và ProtocolTimelock (tức là qua Safe Multisig với minDelay). Đây là bước quan trọng nhất về mặt bảo mật, hoàn tất việc "chuyển giao quyền lực" cho cơ chế quản trị phi tập trung.

Sau khi 3 phase hoàn tất, exportABIs() tự động đọc artifact Hardhat đã biên dịch (từ thư mục artifacts/contracts/) và trích xuất ABI của tất cả 7 smart contract (LendingPool, PriceRouter, MyOracle, Liquidation, ProtocolTimelock, ProtocolController, InterestRateModel), lưu vào deployments/abis.json. File này được chia sẻ trực tiếp cho frontend và backend sử dụng khi khởi tạo contract instance.

6.2 Backend

6.2.1 Công nghệ sử dụng

Hệ thống backend được xây dựng trên nền tảng Node.js với TypeScript, sử dụng kiến trúc đa tiến trình event-driven. Các công nghệ và thư viện chính gồm:

- Node.js 22.19.0 + TypeScript ^5.9.3: Môi trường runtime JavaScript phía server với hệ thống kiểu tĩnh. TypeScript giúp phát hiện lỗi tại compile-time, đặc biệt quan trọng khi làm việc với BigInt (uint256 từ Solidity) và các kiểu dữ liệu phức tạp từ blockchain.
- Express.js ^5.1.0: Framework nhẹ để xây dựng RESTful API.
- Sequelize ^6.37.8 + PostgreSQL: ORM cho Node.js, hỗ trợ ánh xạ dữ liệu giữa PostgreSQL và các đối tượng TypeScript. PostgreSQL được chọn nhờ kiểu DECIMAL(78,0) có thể lưu chính xác số nguyên 256-bit từ Solidity.
- ethers.js ^6.15.0: Thư viện tương tác blockchain, dùng để kết nối với RPC node, đọc event, gọi hàm view trên smart contract và decode event log [12].
- RabbitMQ (amqplib ^0.10.9): Message broker trung gian để phối hợp giữa các tiến trình. Đảm bảo không mất event khi worker gặp sự cố [14].
- Redis (ioredis ^5.10.0): In-memory store phục vụ 2 mục đích: (1) làm trung gian pub/sub cho WebSocket, (2) Lưu trữ dữ liệu ngắn hạn (OTP, nonce).
- Zod ^4.3.6: Thư viện validate input và infer type tự động tích hợp với TypeScript.
- Socket.io ^4.8.1: Thư viện WebSocket, forward event từ Redis pub/sub đến frontend theo thời gian thực.
- Croner ^10.0.1: Thư viện cron job scheduler hỗ trợ timezone và chế độ protect.

- Nodemailer ^8.0.5: Thư viện gửi email qua SMTP.
- Jose ^6.2.3: Thư viện JWT hiện đại.
- Các thư viện khác: dayjs, dotenv,...

6.2.2 Cấu trúc codebase

Cấu trúc thư mục phân chia theo kiến trúc: src/apps/ chứa 5 entry point của 5 tiến trình độc lập, src/modules/ chứa business logic được tổ chức theo domain (blockchain, blockchain-worker, proposals, email, noti-worker), src/shared/ chứa các tiện ích dùng chung (database client, RabbitMQ helper, WebSocket publisher, config).

```
.
|-- infra
|   |-- docker-compose.local.yaml
|-- logs
|   |-- blc-indexer.log
|   |-- blc-worker.log
|   |-- croner.log
|   |-- http-server.log
|   |-- noti-worker.log
|   |-- seed.log
|-- src
|   |-- apps
|   |   |-- blc-indexer
|   |   |-- blc-worker
|   |   |-- croner
|   |   |-- http-server
|   |   |-- noti_proposal-worker
|   |   |-- seed
|   |-- models
|   |   |-- accrue_log.model.ts
|   |   |-- asset.model.ts
|   |   |-- asset_config.model.ts
|   |   |-- asset_snapshot.model.ts
|   |   |-- block.model.ts
|   |   |-- cronner_state.model.ts
|   |   |-- index.ts
|   |   |-- liquidatable_user.model.ts
|   |   |-- proposal.model.ts
|   |   |-- scanner.model.ts
|   |   |-- session.model.ts
```

```
| | |-- transaction.model.ts
| | |-- treasury_log.model.ts
| | |-- user.models.ts
| | |-- user_asset.model.ts
| | `-- user_snapshot.model.ts
| |-- modules
| | |-- assets
| | |-- auth
| | |-- blockchain
| | |-- blockchain-worker
| | |-- email
| | |-- logs-chart
| | |-- multisig
| | |-- noti-worker
| | |-- proposals
| | |-- seed
| | |-- snapshots
| | |-- transactions
| | `-- users
| `-- shared
| |-- bootstrap
| |-- config
| |-- constants
| |-- infra
| |-- types
| |-- utils
| `-- ws
|-- tsconfig.json
```

6.2.3 Cài đặt và các quyết định kỹ thuật quan trọng

a, Lưu uint256 của Solidity vào PostgreSQL

Solidity sử dụng uint256 với giá trị tối đa khoảng 1.15×10^{77} , vượt xa giới hạn lưu trữ của hai tầng phía dưới: PostgreSQL BIGINT chỉ hỗ trợ tối đa khoảng 9.2×10^{18} , trong khi kiểu number của JavaScript chỉ biểu diễn chính xác số nguyên đến $2^{53} - 1$ ($\approx 9 \times 10^{15}$).

Để tránh mất độ chính xác, hệ thống sử dụng bigint native của JavaScript/TypeScript để xử lý toàn bộ giá trị uint256 ở tầng ứng dụng. Khi lưu xuống PostgreSQL, các giá trị này được chuyển sang dạng chuỗi thập phân và lưu bằng kiểu DECIMAL(78,0) (NUMERIC(78,0)), cho phép lưu chính xác số nguyên lên đến 78 chữ

số.

Cách tiếp cận này đảm bảo dữ liệu blockchain được bảo toàn đầy đủ từ smart contract đến database mà không xảy ra overflow hoặc mất precision.

b, Đảm bảo idempotency khi xử lý event

Với at-least-once delivery của RabbitMQ, một event có thể được deliver nhiều lần nếu consumer crash sau khi xử lý nhưng trước khi ack — Phân tích chi tiết về idempotency và cách triển khai đã được trình bày tại mục 5.3. Trong quá trình cài đặt, idempotency được áp dụng nhất quán:

- Bảng `accrue_logs`, `treasury_logs`, `transactions`: unique constraint trên (`transactionHash`, `logIndex`)
- Bảng `user_assets`: cột `lastSyncedBlock` và `lastSyncedLogIndex` ngăn event cũ ghi đè state mới
- Bảng `blocks`: unique constraint trên (`number`, `chainId`) ngăn duplicate block header

Tham chiếu: Mục 5.3.

c, Xử lý SIWE và quản lý session

Vấn đề: backend cần xác thực người dùng mà không dùng username/password truyền thống. Đồng thời, cần hỗ trợ người dùng đăng nhập từ nhiều thiết bị cùng lúc và có khả năng thu hồi session cụ thể.

Giải pháp: Áp dụng EIP-4361 (Sign-In with Ethereum) — chuẩn được trình bày tại mục 3.7 [11]. Luồng cụ thể trong hệ thống:

- Frontend request nonce từ backend (GET `/api/auth/nonce?address=0x...`)
- Backend tạo nonce ngẫu nhiên, lưu vào Redis với TTL 5 phút
- Frontend tạo SIWE message chuẩn EIP-4361, yêu cầu ví ký
- Frontend gửi message + chữ ký lên backend (POST `/api/auth/verify`)
- Backend xác minh chữ ký bằng `ethers.verifyMessage()`, kiểm tra nonce khớp và chưa hết hạn
- Backend tạo access và refresh token (UUID), lưu refresh token vào bảng `sessions` với device/IP info, trả về access token
- Các request tiếp theo gửi kèm token trong Authorization header — backend lookup session trong database

Thiết kế nhiều session cho một địa chỉ: bảng `sessions` có thể chứa nhiều bản ghi

cho cùng một `userAddress`, mỗi bản ghi là một thiết bị khác nhau. Người dùng có thể xem danh sách session đang hoạt động và thu hồi session cụ thể (`set revoked = true`).

Tham chiếu lý thuyết: Mục 3.7 — Sign-In with Ethereum (EIP-4361).

6.2.4 Quá trình triển khai

Hệ thống backend được triển khai dưới dạng 5 tiến trình Node.js độc lập. Mỗi tiến trình được khởi động bằng lệnh riêng từ `package.json`:

Tiến trình	Entry point	Vai trò
blc-indexer	<code>apps/blc-indexer/index.ts</code>	Quét blockchain, publish event vào RabbitMQ
blc-worker	<code>apps/blc-worker/index.ts</code>	Nhận message từ RabbitMQ, lưu database
croner	<code>apps/croner/index.ts</code>	Snapshot hằng ngày, quét Safe Multisig
noti-worker	<code>apps/noti_proposal-worker/index.ts</code>	Xử lý email OTP, proposal, thông báo admin
http-server	<code>apps/http-server/index.ts</code>	REST API và WebSocket

Bảng 6.1: Khởi động các tiến trình

Các bước triển khai:

Bước 1 — Cấu hình môi trường: Tạo file `.env` với các biến: RPC URL của từng chain, địa chỉ các smart contract đã deploy, DSN của PostgreSQL, URL của RabbitMQ, URL của Redis, thông tin SMTP cho email.

Bước 2 — Khởi động các tiến trình: Mỗi tiến trình được khởi động độc lập. Trong môi trường production có thể dùng PM2 hoặc Docker để quản lý vòng đời tiến trình và tự động restart khi gặp lỗi.

Bước 3 — Xác nhận hoạt động: Quan sát log của `blc-indexer` để xác nhận đang quét block và publish event đúng; quan sát log `blc-worker` để xác nhận đang nhận và xử lý message, quan sát log của các tiến trình khác để xác nhận hoạt động

6.3 Frontend

6.3.1 Công nghệ sử dụng

Phần giao diện người dùng được xây dựng bằng các công nghệ sau:

- React.js + Next.js (App Router): Framework React hỗ trợ điều hướng, Server-Side Rendering (SSR) và tối ưu hiệu năng.
- ethers.js v6: Tương tác với MetaMask và các smart contract trên blockchain [12].
- siwe (Sign-In with Ethereum): Thư viện triển khai chuẩn EIP-4361 [11].

- axios: Gửi HTTP request đến backend.
- socket.io-client: Lắng nghe sự kiện WebSocket từ backend theo thời gian thực.
- Material-UI (MUI): Thư viện UI component theo Material Design.

6.3.2 Cấu trúc codebase

```
|-- app
|   |-- admin
|   |   `-- page.tsx
|   |-- borrow
|   |   `-- page.tsx
|   |-- dashboard
|   |   `-- page.tsx
|   |-- history
|   |   `-- page.tsx
|   |-- layout.tsx
|   |-- liquidation
|   |   `-- page.tsx
|   |-- page.tsx
|   |-- proposals
|   |   `-- page.tsx
|   `-- supply
|   `-- page.tsx
|-- components
|   |-- BorrowDialog.tsx
|   |-- DepositDialog.tsx
|   |-- Navbar.tsx
|   |-- RepayDialog.tsx
|   |-- WalletConnect.tsx
|   |-- WithdrawDialog.tsx
|   `-- charts
|   `-- TimeSeriesChart.tsx
|-- hooks
|   `-- useContracts.ts
|-- jsconfig.json
|-- lib
|   |-- axios.ts
|   `-- web3.ts
|-- next-env.d.ts
|-- next.config.mjs
|-- services
|   |-- SafeMultisigService.ts
```

```
| |-- assetService.ts
| |-- authService.ts
| |-- logService.ts
| |-- marketConfigService.ts
| |-- proposalService.ts
| |-- snapshotService.ts
| |-- transactionService.ts
| |-- userAssetService.ts
| |-- userService.ts
|-- tsconfig.json
|-- tsconfig.tsbuildinfo
`-- types
|-- ethereum.d.ts
`-- governance.ts
```

6.3.3 Cài đặt và các quyết định kỹ thuật quan trọng

a, Cách thức truy vấn dữ liệu: blockchain trực tiếp vs backend API

Frontend có hai nguồn dữ liệu: đọc trực tiếp từ blockchain qua ethers.js (dữ liệu real-time, luôn chính xác tuyệt đối), hoặc truy vấn từ backend API (nhanh hơn, có lịch sử).

Chiến lược phân chia được áp dụng:

- Đọc blockchain trực tiếp: số dư thực tế của người dùng (currentDeposit, currentBorrow), health factor trước khi ký transaction, preview kết quả của một giao dịch (previewDeposit, previewBorrow)
- Từ backend API: danh sách tài sản hỗ trợ và thông số, lịch sử giao dịch, biểu đồ TVL và lãi suất theo thời gian, danh sách vị thế liquidatable, danh sách proposals

Quyết định này tối ưu cả hai chiều: dữ liệu quyết định tài chính luôn lấy từ nguồn đáng tin cậy nhất (blockchain), trong khi dữ liệu hiển thị được lấy từ backend để giảm số lượng RPC call và hỗ trợ historical queries.

b, Hiển thị số dư lãi suất cập nhật mà không cần reload

Số dư lãi suất của người dùng tăng liên tục theo giây (do cơ chế Lazy Accrual của smart contract), nhưng database backend chỉ cập nhật khi có event mới. Nếu frontend chỉ lấy dữ liệu từ backend, số dư hiển thị sẽ luôn là số dư tại thời điểm event cuối cùng, không phản ánh lãi đã tích lũy từ đó đến nay. Giải pháp: frontend gọi trực tiếp hàm view trên blockchain (getPreviewUserDeposit, getPreviewUserBorrow) mỗi 15 giây — những hàm này tính toán số dư hiện tại có tính đến lãi tích lũy mà

không tốn gas (vì là view function, không ghi blockchain).

c, Xử lý sự kiện thanh lý real-time qua WebSocket

Khi một vị thế rơi vào ngưỡng có thể bị thanh lý, thông tin cần được cập nhật ngay lập tức đến giao diện người dùng mà không cần polling. Frontend kết nối WebSocket với backend và lắng nghe sự kiện `LiquidatableUsersUpdated`. Mỗi khi `blc-indexer` hoàn tất quét vị thế (mỗi 30 giây), danh sách địa chỉ bị liquidatable được publish qua Redis → http-server nhận và forward qua WebSocket → frontend cập nhật UI. Thiết kế này đảm bảo liquidator có thể phát hiện cơ hội thanh lý sớm nhất có thể.

6.3.4 Quá trình triển khai

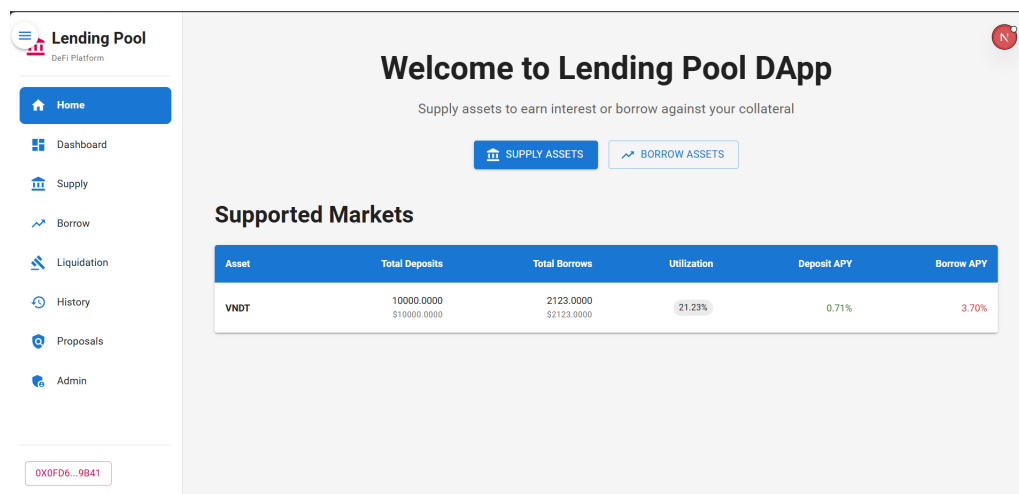
Bước 1 — Cấu hình biến môi trường: Đặt `NEXT_PUBLIC_BACKEND_URL` trỏ đến địa chỉ backend API, các biến `NEXT_PUBLIC_CONTRACT_` chứa địa chỉ từng smart contract trên Sepolia.

Bước 2 — Cấu hình ethers.js: Xây dựng thư viện tạo contract instance từ ABI và địa chỉ, bọc MetaMask bằng `BrowserProvider` để frontend có thể ký giao dịch.

Bước 3 — Triển khai production: Frontend được build và deploy lên Vercel (<https://lending-pool-prj3.vercel.app>). Vercel tự động nhận diện Next.js và cấu hình Server-Side Rendering. Mỗi lần push code lên GitHub, Vercel tự động trigger deploy lại.

6.3.5 Giao diện Demo

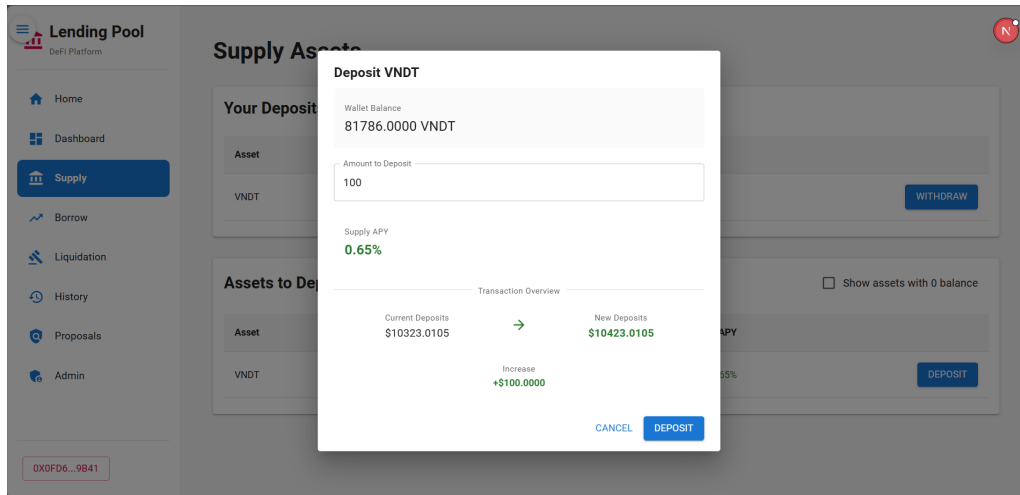
Hình 6.1 thể hiện trang chủ khi người dùng vừa vào ứng dụng, có danh sách và thông tin các tài sản đang được hỗ trợ trong giao thức.



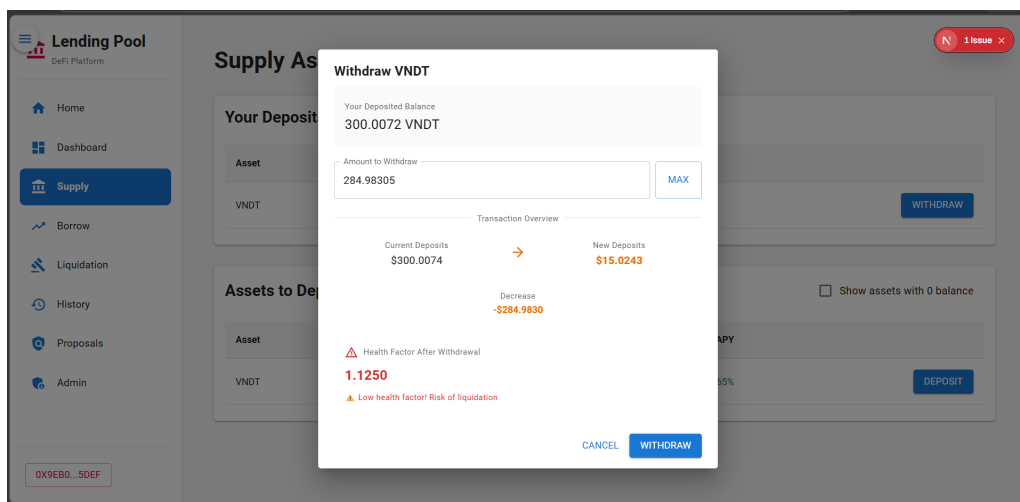
Hình 6.1: Trang chủ

Hình 6.2 biểu diễn trang gửi tài sản và form pop up cho biết số dư trong ví và

trong giao thức của một tài sản, cùng với preview sau khi gửi tài sản, trong khi hình 6.3 biểu diễn pop up form khi rút tài sản, cho biết số dư mới cùng với health factor sau khi rút tài sản.

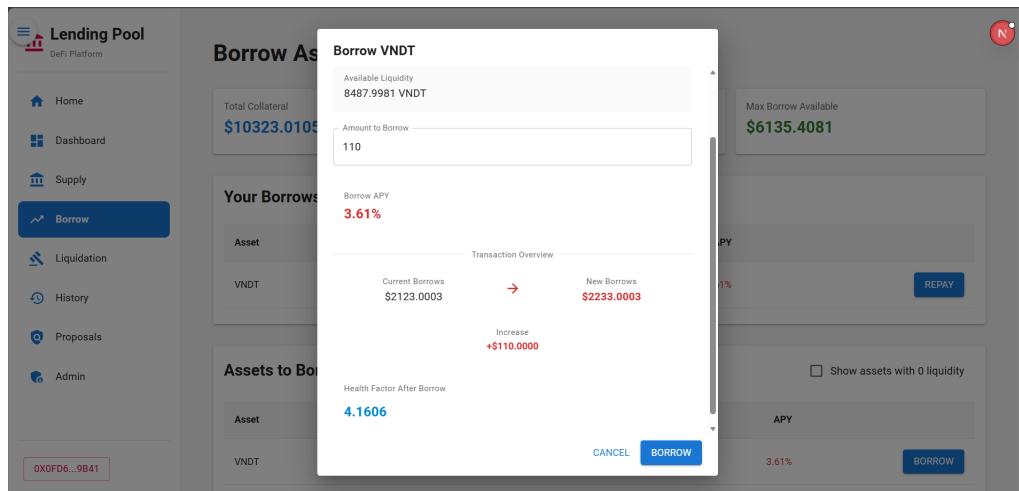


Hình 6.2: Pop up gửi tài sản

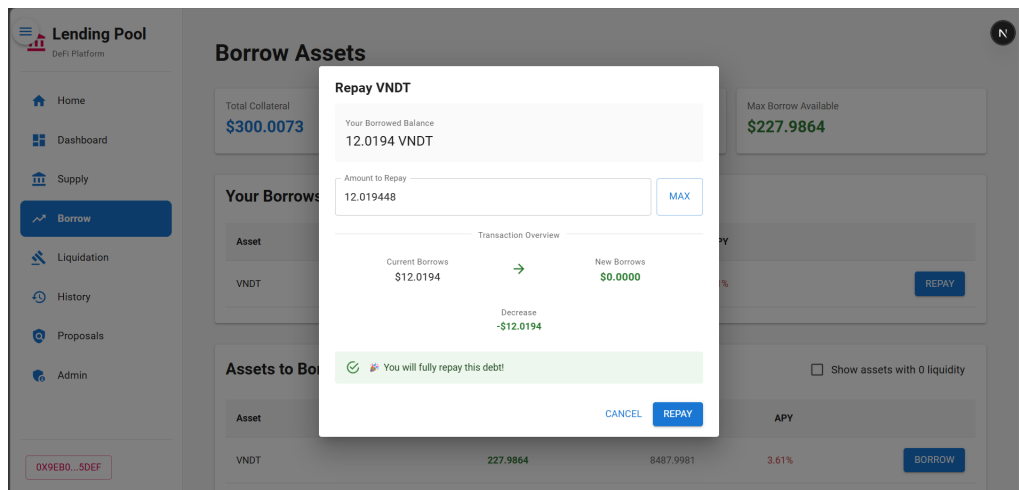


Hình 6.3: Pop up rút tài sản

Hình 6.4 biểu diễn trang vay tài sản và form pop up cho biết số tài sản có thể vay với thể chấp hiện tại, cùng với preview số dư và health factor sau khi vay tài sản, và hình 6.5 biểu diễn pop up form trả nợ tài sản cho biết số dư nợ của người dùng,

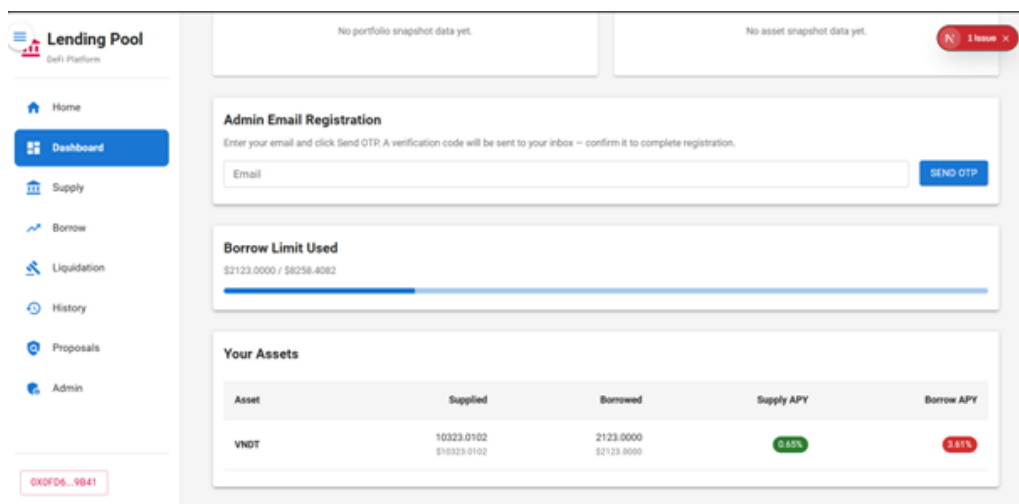


Hình 6.4: Pop up vay tài sản



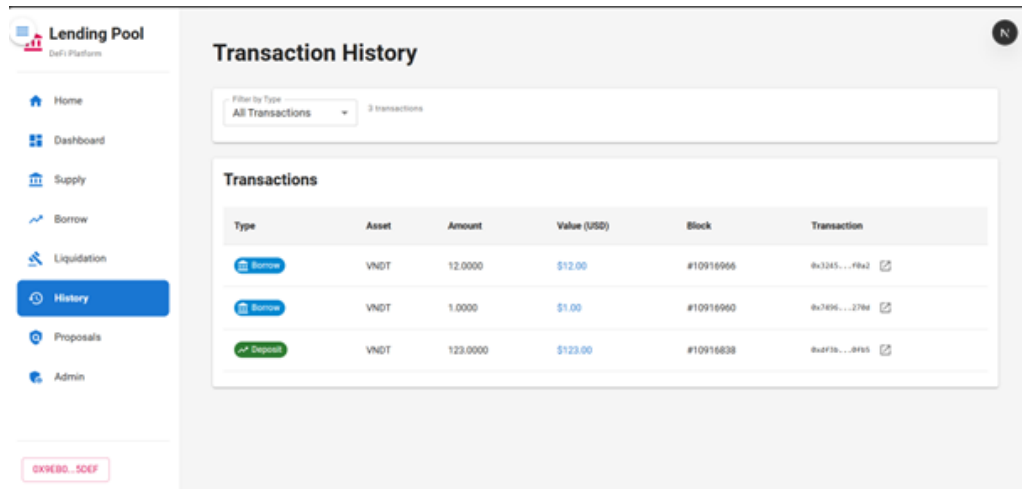
Hình 6.5: Pop up trả nợ tài sản

Hình 6.6 biểu diễn trang dashboard bao gồm các thông tin cá nhân của tài khoản ứng với địa chỉ xác định.



Hình 6.6: Trang dashboard cá nhân

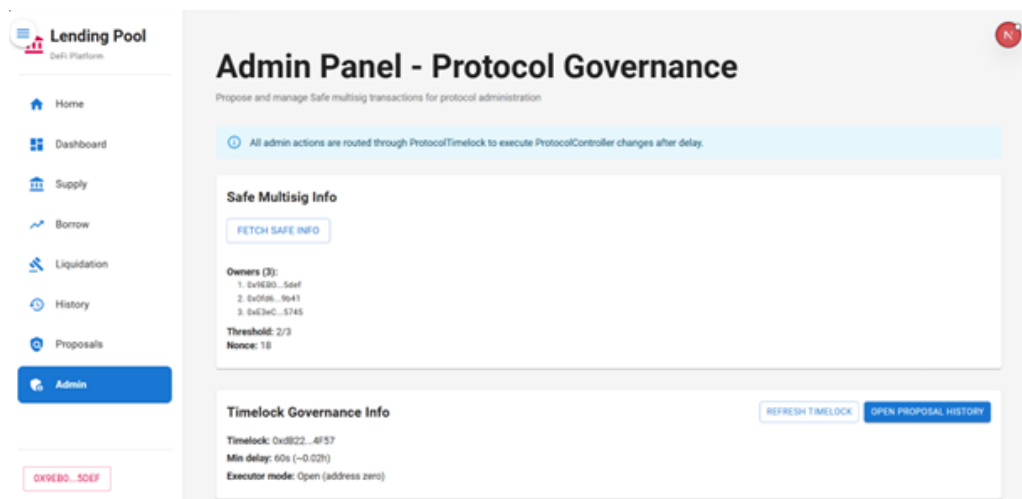
Hình 6.7 thể hiện lịch sử các giao dịch của người dùng tương tác với giao thức và các thông tin liên quan.



Type	Asset	Amount	Value (USD)	Block	Transaction
Borrow	VNDT	12.0000	\$12.00	#10916966	0x3245...f9a2
Borrow	VNDT	1.0000	\$1.00	#10916960	0x7496...2784
Deposit	VNDT	123.0000	\$123.00	#10916838	0x2f1a...4f65

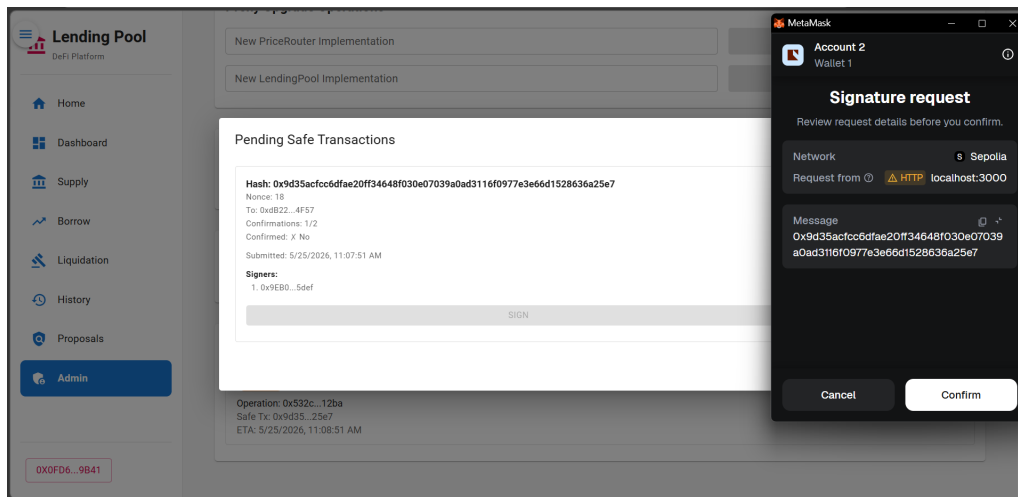
Hình 6.7: Trang lịch sử giao dịch cá nhân

Hình 6.8 biểu diễn trang admin dành cho quản trị hệ thống, chỉ có những admin được cấp quyền mới có thể truy cập trang này; hình 6.9 biểu diễn thao tác admin ký và thực thi đề xuất với ví đa chữ ký trong khi hình 6.10 thể hiện hàng đợi timelock của các đề xuất admin.

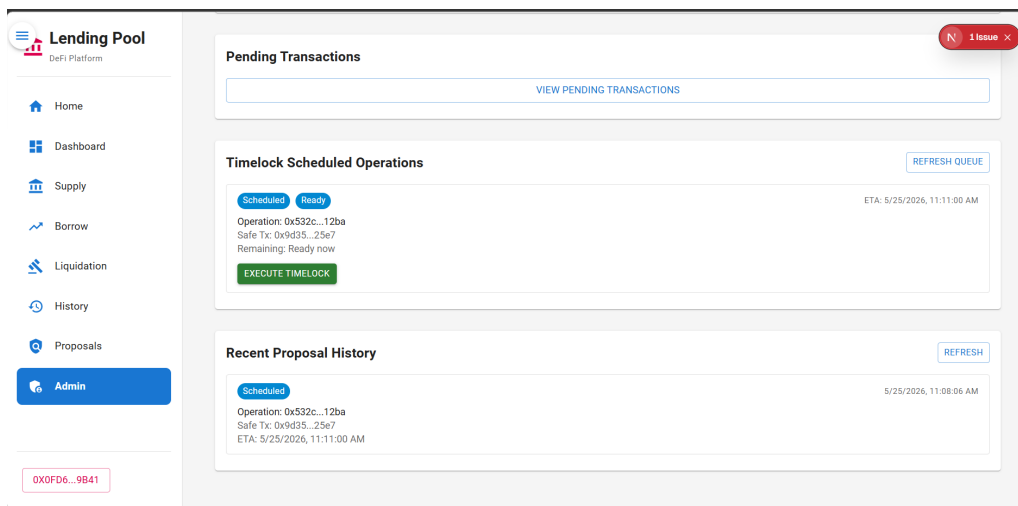


Admin Panel - Protocol Governance	
Propose and manage Safe multisig transactions for protocol administration	
All admin actions are routed through ProtocolTimelock to execute ProtocolController changes after delay.	
Safe Multisig Info	
FETCH SAFE INFO	
Owners (3): 1. 0x1680...5def 2. 0x056...9a41 3. 0x3ac...5745	
Threshold: 2/3 Nonce: 18	
Timelock Governance Info	
Timelock: 0x822...4f57 Min delay: 60s (~0.02h) Executor mode: Open (address zero)	
REFRESH TIMELOCK OPEN PROPOSAL HISTORY	

Hình 6.8: Trang admin panel

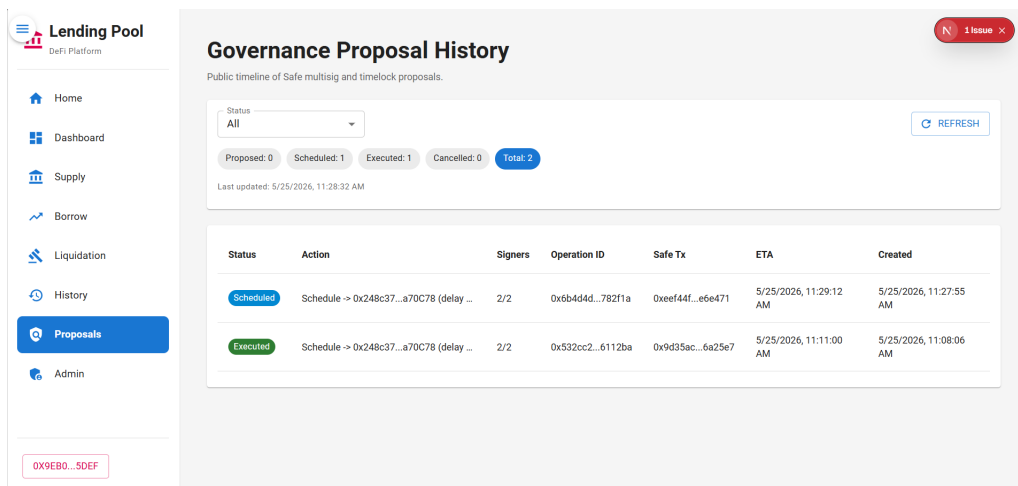


Hình 6.9: Admin tương tác với multisig



Hình 6.10: Hàng đợi timelock và danh sách đề nghị

Hình 6.11 trình bày danh sách các đề xuất đã từng xuất hiện của giao thức và các thông tin liên quan



Hình 6.11: Trang đề nghị thực thi

CHƯƠNG 7. KIỂM THỬ HỆ THỐNG

7.1 Kiểm thử smart contract

7.1.1 Công cụ và phương pháp kiểm thử

Kiểm thử smart contract được thực hiện bằng phương pháp unit test tự động, sử dụng bộ công cụ sau:

- **Hardhat:** Môi trường phát triển và kiểm thử smart contract, cung cấp mạng blockchain cục bộ (Hardhat Network) cho phép thực thi test mà không tốn gas thực tế. Hỗ trợ loadFixture (tái sử dụng snapshot trạng thái) và time.increase (mô phỏng thời gian trôi qua) [15].
- **ethers.js v6:** Thư viện tương tác với smart contract trong quá trình kiểm thử [12].
- **Mocha + Chai:** Framework tổ chức test case (describe/it) và thư viện assertion. Plugin @nomicfoundation/hardhat-chai-matchers mở rộng Chai với các matcher chuyên dụng cho smart contract như revertedWith, revertedWithCustomError, và emit.
- **solidity-coverage:** Plugin đo lường code coverage cho Solidity, phân tích mức độ bao phủ trên bốn chiều: Statements, Branches, Functions, và Lines.
- **hardhat-gas-reporter:** Plugin báo cáo lượng gas tiêu thụ của từng hàm.
- **TypeChain:** Tự động sinh type definitions từ ABI, đảm bảo type-safety khi viết test bằng TypeScript.

Mỗi test suite sử dụng loadFixture để khởi tạo trạng thái giao thức hoàn chỉnh (deploy toàn bộ smart contract, thiết lập market, cấu hình giá), đảm bảo mỗi test case thực thi độc lập trên một snapshot riêng biệt. Fixture chính deployProtocol-WithMarketsFixture tái hiện đầy đủ quy trình deploy 3 phase (Deploy → Wire → Migrate) đã trình bày tại Chương 6, bao gồm deploy toàn bộ 7 smart contract giao thức cùng 3 token MockERC20(WETH, WBTC, USDC) phục vụ kiểm thử, thiết lập ba thị trường với giá và Interest Rate Model tương ứng.

7.1.2 Phạm vi và cấu trúc kiểm thử

Tổng cộng 4 file test kiểm thử 4 nhóm smart contract chính, với tổng số 127 test case. Toàn bộ 127 test case đều pass thành công. Cấu trúc kiểm thử được tổ chức theo bảng 7.1 sau:

File test	Số test	Phạm vi kiểm thử
LendingPool.test.ts	38	initialize (3 test), deposit (5 test), withdraw (5 test), borrow (5 test), repay (3 test), accrueInterest (6 test), preview functions (6 test), initialize revert cases (5 test)
Liquidation.test.ts	31	constructor validation (10 test), isAccountLiquidatable (5 test), calculateSeizeAmount (5 test), liquidate execution (7 test), revert cases (4 test)
InterestRateModel.test.ts	8	constructor validation (3 test), getBorrowRate tại các mức utilization (2 test), getDepositRate (3 test)
ProtocolController.test.ts	50	constructor (6 test), migrateController và update addresses (5 test), price feed (3 test), liquidation params (2 test), my oracle (2 test), pause/unpause (3 test), supportMarket (2 test), unsupportMarket (1 test), setCollateralParams (3 test), setInterestRateModelBatch (3 test), donate (4 test), withdrawTreasury (5 test), rescueToken (4 test), UUPS upgrade (6 test), non-admin revert (1 test)

Bảng 7.1: Kiểm thử tự động smart contract

7.1.3 Chi tiết các nhóm kiểm thử

a, Kiểm thử LendingPool (38 test case)

Đây là test suite lớn nhất, bao phủ toàn bộ chức năng cốt lõi của giao thức:

Khởi tạo (initialize — 3 test): Kiểm tra rằng proxy được khởi tạo đúng tham số (controller, liquidation, priceRouter, collateralFactor); revert khi gọi initialize lần thứ hai (InvalidInitialization); revert khi gọi trực tiếp trên implementation contract (bảo vệ UUPS).

Initialize revert cases (5 test): Kiểm tra deploy proxy revert khi truyền controller address là zero, liquidation address là zero, price router address là zero, collateral factor bằng 0, hoặc collateral factor vượt quá 1.

Deposit (5 test): Kiểm tra cập nhật đúng userBalances.deposited, markets.totalDeposits, depositIndexSnapshot; tích lũy qua nhiều lần deposit; phát đúng event Deposit; hỗ trợ token với decimal khác nhau (USDC 6 decimals, WETH 18 decimals); revert khi market chưa được hỗ trợ, amount bằng 0, hoặc pool đang bị pause.

Withdraw (5 test): Kiểm tra giảm đúng số dư và totalDeposits; trả token về ví người dùng; cap withdrawal tại deposited amount nếu yêu cầu rút vượt quá; cap withdrawal theo collateral constraint khi có khoản vay (đảm bảo health factor

không bị vi phạm); revert tương tự deposit.

Borrow (5 test): Kiểm tra vay trong giới hạn collateral, cập nhật totalBorrows và borrowIndex; hỗ trợ multi-asset collateral; revert khi vượt quá thanh khoản (Not enough liquidity in the market); revert khi vượt quá collateral (Insufficient collateral); revert khi market không hỗ trợ, amount bằng 0, hoặc pool pause.

Repay (3 test): Kiểm tra trả nợ cập nhật đúng state và phát event Repay; cap repayment nếu trả vượt quá số nợ hiện tại và hoàn trả token dư; revert khi chưa có khoản vay (No borrow left), market không hỗ trợ, amount bằng 0, hoặc pool pause.

Accrue Interest (6 test): Kiểm tra lãi suất tích lũy đúng sau khi thời gian trôi qua — totalBorrows, borrowIndex, totalDeposits, depositIndex đều tăng; preview functions phản ánh đúng lãi tích lũy mà không thay đổi state; treasury nhận được phần reserve từ spread borrow-deposit; phát đúng event Accrue; borrower trả lãi nhiều hơn depositor nhận (đảm bảo giao thức có lãi); gọi accrueInterest hai lần trong cùng block là no-op.

Preview Functions (7 test): Kiểm tra previewDeposit, previewBorrow trả về đúng giá trị USD và health factor; previewWithdraw trả về đúng khi có khoản vay đang hoạt động; previewWithdraw trả về type(uint256).max khi không có khoản vay; previewRepay trả về đúng cho partial repay; previewRepay trả về newBorrowedUSD = 0 khi amount vượt quá total borrow; revert khi market không hỗ trợ hoặc amount bằng 0 (kiểm tra 8 assertion trong 1 test).

b, Kiểm thử Liquidation (31 test case)

Constructor validation (10 test): Kiểm tra thiết lập đúng các tham số (priceRouter, lendingPool, controller, liquidationThreshold, closeFactor, liquidationIncentive); revert khi bất kỳ địa chỉ nào là zero (priceRouter, lendingPool, controller); revert khi liquidationThreshold bằng 0 hoặc vượt SCALE; revert khi closeFactor bằng 0 hoặc vượt SCALE; revert khi liquidationIncentive bằng 0 hoặc vượt 0.2e18.

Phát hiện vị thế dưới ngưỡng (isAccountLiquidatable — 5 test): Kiểm tra trả true khi giá collateral giảm khiến vị thế dưới liquidationThreshold; trả false khi không có deposit; trả false khi có deposit nhưng không có borrow; trả false khi vị thế vẫn khỏe; trả true khi lãi suất tích lũy theo thời gian khiến vị thế mất an toàn (mô phỏng 5 năm).

Tính toán lượng tịch thu (calculateSeizeAmount — 5 test): Kiểm tra bao gồm đúng liquidationIncentive; xử lý đúng chênh lệch decimal giữa repay asset (6 decimals) và collateral asset (18 decimals) và ngược lại; tuyến tính theo repay amount; revert khi collateral không có price feed.

Thực thi thanh lý (liquidate — 11 test): Kiểm tra giảm đúng nợ và collateral của borrower; chuyển collateral cho liquidator; cap repay tại $\text{closeFactor} \times \text{currentBorrow}$; không cap repay khi nằm trong giới hạn closeFactor ; cap seize tại $\text{borrower deposit balance}$; phát event `LiquidationExecuted`; revert khi borrower là zero address; revert khi liquidator address param là zero; revert khi account không liquidatable; revert khi repay amount bằng 0; revert khi gọi `seizeCollateral` trực tiếp (chỉ cho phép qua `Liquidation` contract).

c, Kiểm thử InterestRateModel (8 test case)

Constructor (3 test): Kiểm tra constructor thiết lập đúng các tham số immutable (`baseRate`, `rateSlope1`, `rateSlope2`, `optimalUtilization`, `reserveFactor`); revert khi `optimalUtilization` vượt 100%; revert khi `reserveFactor` vượt 100%.

`getBorrowRate` (2 test): Kiểm tra trả về đúng giá trị tại 5 mức utilization (0%, 40%, 80%, 90%, 100%) theo công thức Two-Slope Kinked Rate Model; kiểm tra borrow rate tăng đơn điệu theo utilization qua 7 mức.

`getDepositRate` (3 test): Kiểm tra đúng công thức $\text{borrowRate} \times \text{utilization} \times (1 - \text{reserveFactor})$ tại các mức utilization; deposit rate tăng đơn điệu; deposit rate luôn nhỏ hơn borrow rate do `reserveFactor`.

d, Kiểm thử ProtocolController (50 test case)

Constructor (6 test): Kiểm tra thiết lập đúng owner và các tham số (`lendingPool`, `myOracle`, `priceRouter`, `liquidation`, `DEFAULT_ADMIN_ROLE`); revert khi bất kỳ địa chỉ nào là zero (`lendingPool`, `priceRouter`, `myOracle`, `liquidation`, `admin`).

Update addresses (5 test): Kiểm tra `migrateController` cập nhật controller trên tất cả smart contract liên quan; `setLendingPool`, `setPriceRouter`, `setMyOracle`, `setLiquidation` cập nhật đúng địa chỉ trên các smart contract phụ thuộc; mỗi hàm revert khi truyền zero address.

Price feed (3 test): Kiểm tra `setChainlinkFeed` thiết lập Chainlink feed; `setMyOracleFeed` thiết lập MyOracle feed; `removeFeed` xóa feed.

Liquidation params (2 test): Kiểm tra `setLiquidateParams` cập nhật đúng `liquidationThreshold`, `closeFactor`, `liquidationIncentive`; revert khi tham số không hợp lệ (5 assertion: `threshold=0`, `threshold>SCALE`, `closeFactor=0`, `closeFactor>SCALE`, `incentive` quá cao).

MyOracle (2 test): Kiểm tra `setPrice` cập nhật đúng giá và phát event `PriceUpdated`; revert khi giá bằng 0, asset là zero address, hoặc asset chưa có giá.

Lending Pool functions (26 test):

- Pause/unpause (3 test): pause thành công, non-admin bị revert, unpause thành công.
- supportMarket (2 test): hỗ trợ market mới phát event MarketSupported; cập nhật IRM cho market đã tồn tại.
- unsupportMarket (1 test): bỏ hỗ trợ market phát event MarketUnsupported.
- setCollateralParams (3 test): set thành công phát event; revert khi bằng 0; revert khi vượt SCALE.
- setInterestRateModelBatch (3 test): set IRM cho nhiều asset; revert với IRM zero address; revert với mảng rỗng.
- donate (4 test): donate thành công phát event Donated; revert khi amount = 0; revert khi market chưa hỗ trợ; revert khi pool pause.
- withdrawTreasury (5 test): rút treasury thành công; revert khi amount = 0, recipient = zero, amount vượt balance, hoặc non-admin gọi.
- rescueToken (4 test): rescue thành công; revert khi amount = 0, recipient = zero, hoặc amount vượt surplus.
- Non-admin access control (1 test): non-admin bị revert khi gọi rescueToken.

UUPS Proxy Upgrade (6 test): Kiểm tra upgradePriceRouter — revert khi zero address, revert khi non-admin, upgrade thành công phát event và thay đổi implementation address; upgradeLendingPool — tương tự 3 test.

7.1.4 Kết quả Code Coverage

Code coverage được đo bằng solidity-coverage, kết quả chi tiết chạy lệnh test thể hiện trong hình 7.1, tổng hợp trong bảng 7.2.

Contract	Statements	Branches	Functions	Lines
InterestRateModel	100%	100%	100%	100%
LendingPool	93.55%	76.61%	91.3%	91.22%
Liquidation	100%	78.85%	100%	100%
MyOracle	100%	68.75%	100%	100%
PriceRouter	60%	41.18%	100%	66.67%
ProtocolController	96.15%	75%	95.65%	96.67%
Tổng	93.14%	73.63%	95%	91.93%

Bảng 7.2: Coverage kiểm thử smart contract

127 passing (12s)

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts/	93.08	73.75	94.85	91.86	
InterestRateModel.sol	100	100	100	100	
LendingPool.sol	93.55	76.61	91.3	91.22	... 776,778,820
LendingPoolStorage.sol	100	100	100	100	
Liquidation.sol	100	78.85	100	100	
MyOracle.sol	100	68.75	100	100	
PriceRouter.sol	60	41.18	100	66.67	... 90,91,93,95
PriceRouterStorage.sol	100	100	100	100	
ProtocolController.sol	96.15	75	95.65	96.67	169,170
ProtocolTimelock.sol	100	100	100	100	
contracts/interfaces/	100	100	100	100	
Interfaces.sol	100	100	100	100	
contracts/test/	100	50	100	100	
MockERC20.sol	100	50	100	100	
All files	93.14	73.63	95	91.93	

Hình 7.1: Kết quả chạy lệnh kiểm tra coverage

Nhận xét: Tỷ lệ bao phủ Statements và Lines đạt trên 92%, Functions đạt 95%. Contract PriceRouter có coverage thấp hơn do các nhánh liên quan đến Chainlink Aggregator (yêu cầu mock phức tạp hơn) chưa được kiểm thử đầy đủ — đây là hướng cải thiện trong tương lai. Branch coverage ở mức 74% xuất phát từ hai nhóm nhánh chưa được bao phủ đầy đủ. Nhóm thứ nhất là các nhánh liên quan đến Chainlink Aggregator trong PriceRouter (40% branch coverage): để kiểm thử các nhánh `answeredInRound < roundId` và `updatedAt == 0`, cần mock AggregatorV3Interface trả về các giá trị bất thường — đây là kỹ thuật khả thi nhưng chưa được triển khai trong phạm vi đề án. Nhóm thứ hai là các nhánh bất biến phòng thủ được kiểm tra tại cả LendingPool lẫn Liquidation (defense-in-depth), vốn chỉ có thể kích hoạt nếu smart contract gọi đến bị compromise — điều không thể tái hiện trong môi trường test thông thường mà không can thiệp trực tiếp vào bytecode.

7.1.5 Kết quả Gas Report

Gas report được sinh bởi hardhat-gas-reporter, cho biết lượng gas tiêu thụ của các hàm chính, thể hiện trong bảng 7.3 và bảng 7.4 thể hiện chi phí deploy của các smart contract:

Hàm	Min (gas)	Max (gas)	Avg (gas)	Số lần gọi
deposit	80,832	168,290	136,393	41
withdraw	57,997	96,766	84,541	7
borrow	84,044	151,584	127,718	19
repay	48,764	71,752	59,217	4
accrueInterest	30,345	85,069	60,789	9
liquidate	129,430	209,447	164,849	7

Bảng 7.3: Gas report kiểm thử smart contract các hàm chính

Contract (Deploy)	Gas	% Block limit
LendingPool	3,396,978	11.3%
ProtocolController	1,474,420	4.9%
ProtocolTimelock	1,356,288	4.5%
Liquidation	1,061,830	3.5%
PriceRouter	914,408	3.0%

Bảng 7.4: Gas report kiểm thử smart contract deploy

Nhận xét: Các hàm nghiệp vụ chính có gas cost hợp lý — deposit trung bình 136k gas, liquidate trung bình 165k gas. Chi phí deploy lớn nhất là LendingPool (11.3% block gas limit) do logic phức tạp, nhưng vẫn nằm trong giới hạn cho phép và chỉ phải deploy một lần.

7.2 Kiểm thử Backend

7.2.1 Công cụ và phương pháp kiểm thử

Kiểm thử backend được thực hiện bằng phương pháp unit test tự động với chiến lược cô lập hoàn toàn (isolation testing) — mọi phụ thuộc hạ tầng (database, Redis, blockchain provider) đều được thay thế bằng mock objects. Bộ công cụ sử dụng:

- **Vitest:** Framework kiểm thử hiện đại cho TypeScript/ESM, hỗ trợ native ES modules, mock functions (vi.fn()), và tích hợp coverage qua V8 provider.
- **Mock Factory Pattern:** Hệ thống tạo mock được tổ chức trong thư mục test/shared/mocks.ts, cung cấp các factory function createMockDbClient(), createMockRedis(), createMockLogger().
- **Domain Fixtures:** File test/test-helpers/domain.ts cung cấp các factory function tạo dữ liệu mẫu (createUserRow, createAssetRow, createSessionRow, createTransactionRow) với giá trị mặc định hợp lý.
- **Error Assertion Helper:** Hàm expectAppErr(promise, errorCode) đóng gói logic kiểm tra rằng một Promise reject với đúng AppErr và mã lỗi mong đợi.

7.2.2 Phạm vi và cấu trúc kiểm thử

Tổng cộng 9 file test kiểm thử 7 module của hệ thống backend, với tổng số 71 test case (toàn bộ pass), được trình bày bằng bảng 7.5 và hình 7.2.

File test	Số test	Phạm vi kiểm thử
signature.service.test.ts	9	Tạo SIWE message, xác minh chữ ký Ethereum, quản lý nonce Redis, rate-limit nonce requests
session.service.test.ts	8	Tạo session, refresh token, revoke session, xử lý session hết hạn/bị thu hồi
token.service.test.ts	8	Sinh JWT access token, xác minh token (hợp lệ, hết hạn, sai secret, malformed, sai tokenType, thiếu claims)
asset.service.test.ts	8	Lấy danh sách/chi tiết tài sản, lấy cấu hình thị trường, xử lý lỗi DB, validate Ethereum address
blc-worker.handler.test.ts	19	Xử lý event Deposit/Withdraw/Borrow/Repay/Accrue/Liquidation/Donate/TreasuryWithdrawn/MarketUnsupported, idempotency (skip duplicate)
transaction.service.test.ts	5	Lấy lịch sử giao dịch, validate input, xử lý user không tồn tại
user.service.test.ts	7	Lấy thông tin user, dashboard detail (user + assets), lấy email, xử lý user không tồn tại, validate address
rate-limit.service.test.ts	4	ensureCooldown (SET NX), ensureAttemptLimit (INCR + TTL), xử lý khi vượt giới hạn
validate.test.ts	3	Checksum Ethereum address, reject địa chỉ không hợp lệ, reject zero address

Bảng 7.5: Kiểm thử tự động backend

```
hoangdung@DESKTOP-ELNTE38:~/lending-pool-prj3/backend$ npm run test

> backend@1.0.0 test
> vitest run

RUN v4.1.7 /home/hoangdung/lending-pool-prj3/backend

✓ test/unit/modules/blockchain-worker/blc-worker.handler.test.ts (19 tests) 147ms
✓ test/unit/modules/users/user.service.test.ts (7 tests) 49ms
✓ test/unit/modules/auth/signature.service.test.ts (9 tests) 217ms
✓ test/unit/modules/auth/session.service.test.ts (8 tests) 28ms
✓ test/unit/modules/auth/token.service.test.ts (8 tests) 20ms
✓ test/unit/modules/transactions/transaction.service.test.ts (5 tests) 21ms
✓ test/unit/shared/rate-limit.service.test.ts (4 tests) 9ms
✓ test/unit/shared/validate.test.ts (3 tests) 6ms
✓ test/unit/modules/assets/asset.service.test.ts (8 tests) 36ms

Test Files 9 passed (9)
Tests 71 passed (71)
Start at 21:43:58
Duration 2.15s (transform 958ms, setup 0ms, import 3.76s, tests 534ms, environment 1ms)
```

Hình 7.2: Kết quả chạy lệnh kiểm thử backend

7.2.3 Chi tiết các nhóm kiểm thử quan trọng

a, Kiểm thử xác thực (Authentication)

Module xác thực được kiểm thử ở cả 3 tầng: SignatureService (tạo message + verify chữ ký), SessionService (quản lý session), TokenService (sinh/xác minh JWT).

SignatureService sử dụng ethers.Wallet.createRandom() để tạo ví ngẫu nhiên trong mỗi test, ký message thực sự bằng wallet.signMessage(), sau đó gọi verifyMessage để kiểm tra luồng xác thực end-to-end. Test kiểm tra: nonce được lưu Redis và xóa sau khi verify thành công; chữ ký sai bị reject; nonce hết hạn bị reject; user được tạo tự động lần đầu đăng nhập qua findOrCreate.

TokenService kiểm tra JWT được sinh đúng format (issuer, audience, subject, expiration, custom claims); token hết hạn, sai secret, malformed, sai tokenType, hoặc thiếu claims đều bị reject với mã lỗi Unauthorized.

SessionService kiểm tra tạo session trả về đầy đủ accessToken và refreshToken; refresh token hợp lệ sinh access token mới; session không tồn tại, bị revoke, hoặc hết hạn đều bị reject với mã lỗi tương ứng (SessionNotFound, SessionRevoked, SessionExpired).

b, Kiểm thử BLC Worker Handler

Đây là test suite quan trọng nhất phía backend, kiểm thử logic xử lý blockchain events. Mỗi loại event được kiểm thử theo 3 kịch bản:

- Happy path: Event được xử lý thành công — tạo transaction record, cập nhật userAsset, cập nhật asset state.
- Idempotency: Event duplicate (cùng transactionHash + logIndex) bị skip mà không throw error — kiểm tra rằng lỗi unique constraint được bắt và xử lý đúng.

Kiểm thử Accrue event còn bao gồm: phát hiện treasury balance drift (cảnh báo khi giá trị thực tế khác giá trị kỳ vọng), và skip duplicate accrue log.

c, Kiểm thử Rate Limiting

Kiểm tra cơ chế rate limiting dựa trên Redis SET NX (cooldown) và INCR (attempt counting): lần gọi đầu tiên thành công (SET NX trả về "OK"); lần gọi thứ hai trong khoảng cooldown bị reject với RateLimitExceeded; attempt count vượt giới hạn bị reject.

7.2.4 Kết quả kiểm thử

Toàn bộ test case được chạy bằng lệnh `npm run test` (sử dụng Vitest). Kết quả: tất cả test case pass thành công. Thời gian thực thi trung bình dưới 5 giây nhờ chiến lược mock hoàn toàn, không phụ thuộc vào hạ tầng thực.

7.3 Kiểm thử Frontend

Phần frontend được kiểm thử theo phương pháp kiểm thử hộp đen (Black-box Testing), tập trung vào hành vi và kết quả hiển thị thay vì cấu trúc mã nguồn. Các tiêu chí kiểm thử:

- Tính đúng đắn của các chức năng giao diện: kết nối ví, deposit, withdraw, borrow, repay, xem lịch sử giao dịch, xem danh sách thanh lý.
- Dữ liệu hiển thị phản ánh đúng trạng thái blockchain: số dư cập nhật real-time qua view function mỗi 15 giây, danh sách liquidatable positions cập nhật qua WebSocket.
- Xử lý lỗi: hiển thị thông báo khi ví chưa kết nối, transaction bị reject, hoặc số dư không đủ.

Các công cụ kiểm thử:

- Trình duyệt web: Google Chrome, Microsoft Edge — kiểm thử trên nhiều trình duyệt để đảm bảo tương thích.
- MetaMask: Kiểm thử tương tác ví (kết nối, ký message SIWE, ký giao dịch, chuyển mạng).
- Etherscan (Sepolia): Xác minh trạng thái giao dịch on-chain, so khớp event logs với dữ liệu hiển thị trên giao diện.

Ngoài ra, kiểm thử tích hợp end-to-end được thực hiện thủ công để xác minh luồng hoàn chỉnh: event từ blockchain → blc-indexer phát hiện → RabbitMQ → blc-worker xử lý → database cập nhật → frontend hiển thị đúng.

CHƯƠNG 8. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

8.1 Kết luận

Đồ án đã hoàn thành xây dựng và triển khai một ứng dụng vay và cho vay phi tập trung hoàn chỉnh trên nền tảng Ethereum [2], bao gồm đầy đủ ba tầng kiến trúc: smart contract, backend và frontend. Bảng 8.1 dưới đây đối chiếu từng mục tiêu đã đề ra tại Chương 1 với kết quả cụ thể đạt được:

Mục tiêu đề ra (Chương I)	Kết quả đạt được	Trạng thái
Xây dựng lớp smart contract cốt lõi: gửi, vay, trả nợ, rút tài sản	7 smart contract Solidity hoàn chỉnh; 4 nghiệp vụ chính hoạt động trên Sepolia	Hoàn thành
Cơ chế tích lũy lãi suất theo mô hình Two-Slope	InterestRateModel triển khai Kinked Rate Model; lãi tích lũy $O(1)$ qua Global Interest Index	Hoàn thành
Cơ chế thanh lý vị thế dưới ngưỡng an toàn	Contract Liquidation với closeFactor + liquidationIncentive; health factor tính realtime	Hoàn thành
Lớp quản trị: ví đa chữ ký + timelock	ProtocolController + ProtocolTimelock + Safe Multisig; quyền deployer được thu hồi sau migrate	Hoàn thành
Hợp đồng có khả năng nâng cấp không mất dữ liệu	UUPS Proxy cho LendingPool và PriceRouter; storage gap 50 slot; đã kiểm thử upgrade trong test suite	Hoàn thành
Quản lý treasury và trích xuất lợi nhuận giao thức	Reserve Factor tích lũy vào treasury; hàm withdrawTreasury qua ProtocolController; kiểm thử 5 test case	Hoàn thành
Backend indexer lắng nghe dữ liệu blockchain (event-driven)	5 tiến trình độc lập: blc-indexer, blc-worker, croner, noti-worker, http-server; RabbitMQ đảm bảo at-least-once	Hoàn thành
Snapshot dữ liệu định kỳ và hệ thống thông báo	Croner chạy snapshot hằng ngày; noti-worker gửi email OTP và thông báo admin qua Nodemailer	Hoàn thành
Giao diện người dùng: kết nối ví, giao dịch, theo dõi vị thế	Next.js với 7 trang chức năng; MetaMask; cập nhật số dư mỗi 15 giây qua view function	Hoàn thành
Xác thực người dùng qua ví Ethereum	EIP-4361 (Sign-In with Ethereum); quản lý đa session; có thể thu hồi session theo thiết bị	Hoàn thành
Kiểm thử và triển khai trên Sepolia	127 test case smart contract (100% pass); 71 test case backend (100% pass); coverage >93% statements	Hoàn thành
Flash loan	Xác định ngoài phạm vi từ đầu	Ngoài phạm vi
Đa chuỗi (multi-chain)	Xác định ngoài phạm vi từ đầu	Ngoài phạm vi
Kiểm toán bảo mật chuyên sâu	Xác định ngoài phạm vi từ đầu	Ngoài phạm vi

Bảng 8.1: Mục tiêu và kết quả của đồ án

Đối với kết quả kỹ thuật từng tầng trong hệ thống, nổi bật là tầng smart contract đạt được các chỉ số kỹ thuật sau kiểm thử, được thể hiện bằng bảng 8.2 sau đây:

Chỉ số	Giá trị
Tổng số smart contract triển khai	7 smart contract (9 deployment bao gồm proxy và implementation)
Số test case	127 (100% pass)
Statement coverage	93.14%
Function coverage	95%
Gas trung bình — deposit	136,393 gas
Gas trung bình — liquidate	164,849 gas
Gas deploy LendingPool	3,396,978 gas (11.3% block limit)
Mạng triển khai	Ethereum Sepolia Testnet

Bảng 8.2: Thông số kiểm thử smart contract

Cơ chế Lazy Accrual với Global Interest Index giải quyết được bài toán tích lũy lãi suất cho toàn bộ người dùng với chi phí $O(1)$ thay vì $O(n)$ — không cần duyệt qua tất cả tài khoản mỗi block. Kiến trúc bảo mật 5 tầng phòng thủ theo chiều sâu (AccessControl, Pausable, ReentrancyGuard, modifier phân quyền, kiểm tra tham số đầu vào) cùng với mẫu UUPS Proxy đảm bảo hệ thống vừa an toàn vừa có khả năng nâng cấp trong tương lai. Tầng backend hoàn thành kiến trúc đa tiến trình event-driven với các đặc điểm kỹ thuật nổi bật: đảm bảo idempotency ở tầng database qua unique constraint (transactionHash, logIndex); hỗ trợ cập nhật thanh lý realtime qua Redis pub/sub → WebSocket → frontend. Tổng cộng 71 test case backend đều pass với chiến lược mock hoàn toàn, không phụ thuộc hạ tầng thực khi kiểm thử. Tầng frontend triển khai chiến lược lấy dữ liệu hai nguồn: dữ liệu quyết định tài chính (số dư thực tế, health factor) được đọc trực tiếp từ blockchain qua view function, trong khi dữ liệu lịch sử và tổng hợp được lấy từ backend API. Chiến lược này cân bằng giữa độ chính xác tuyệt đối và hiệu năng hiển thị.

Bên cạnh những kết quả đạt được, hệ thống hiện còn một số hạn chế kỹ thuật cần thừa nhận:

i Về smart contract: Branch coverage đạt 74% — thấp hơn mức lý tưởng do nhiều nhánh phòng thủ (defense-in-depth) khó tái hiện trong môi trường test thông thường, đặc biệt tại smart contract PriceRouter với các nhánh liên quan đến Chainlink Aggregator (40% branch coverage). Hệ thống chưa có tích hợp kiểm tra tự động bảo mật (Slither, Mythril) trong CI pipeline.

ii Về backend: Blc-indexer hiện hoạt động theo mô hình single-instance — nếu tiến trình này gặp sự cố trong khoảng thời gian dài, sẽ có độ trễ index dữ liệu tích lũy. Chưa có cơ chế tự động phát hiện và cảnh báo khi indexer bị tụt hậu quá nhiều block so với head.

iii Về frontend: Kiểm thử frontend chỉ dừng ở mức manual black-box testing,

chưa có kiểm thử tự động end-to-end (Playwright, Cypress). Khi MetaMask từ chối giao dịch, thông báo lỗi hiển thị trực tiếp từ provider mà chưa được chuẩn hóa sang định dạng thân thiện với người dùng.

iv Về bảo mật tổng thể: Hệ thống chưa được kiểm toán bảo mật bởi bên thứ ba. Với bất kỳ giao thức DeFi thực tế nào, kiểm toán chuyên sâu là yêu cầu bắt buộc trước khi triển khai trên mainnet.

8.2 Hướng phát triển tiếp theo

Từ nền tảng hiện có, các hướng phát triển tiếp theo có thể suy nghĩ tới theo thứ tự ưu tiên: Về mặt ngắn hạn, có thể làm hệ thống dày hơn về kiểm thử tự động, bổ sung kiểm thử tự động frontend bằng Playwright; tăng branch coverage cho PriceRouter bằng mock Chainlink Aggregator; tích hợp Slither vào CI pipeline để phân tích bảo mật tự động. Trong trung hạn, nên thực hiện kiểm toán bảo mật bởi bên thứ ba; bổ sung Flash Loan; xây dựng cơ chế quản trị DAO với token voting thay thế Multisig thuần túy; thêm E-Mode cho phép hệ số thể chấp cao với các tài sản tương quan chặt (tương tự Aave v3). Đối với dài hạn, có thể mở rộng sang đa chuỗi (Polygon, Arbitrum, Base); tối ưu hóa gas cost; triển khai trên Ethereum mainnet sau khi hoàn tất kiểm toán.

TÀI LIỆU THAM KHẢO

- [1] V. Buterin. “Ethereum: A next-generation smart contract and decentralized application platform.” (), [Online]. Available: <https://ethereum.org/whitepaper/> (visited on 05/23/2026).
- [2] Ethereum Foundation. “Ethereum development documentation.” (), [Online]. Available: <https://ethereum.org/developers/docs> (visited on 05/23/2026).
- [3] L. Gudgeon, D. Perez, S. Werner, and W. J. Knottenbelt, “Defi protocols for loanable funds: Interest rates, liquidity and market efficiency,” in *ACM CCS 2020*, 2020.
- [4] Aave Team. “Aave protocol whitepaper v2.0.” (), [Online]. Available: <https://github.com/aave/protocol-v2/blob/master/aave-v2-whitepaper.pdf> (visited on 05/23/2026).
- [5] Compound. “Compound v2 documentation.” (), [Online]. Available: <https://docs.compound.finance/v2/> (visited on 05/23/2026).
- [6] R. Leshner and G. Hayes, “Compound: The money market protocol,” 2019.
- [7] Aave Team. “Aave v3 technical paper.” (), [Online]. Available: https://github.com/aave/aave-v3-core/blob/master/techpaper/Aave_V3_Technical_Paper.pdf (visited on 05/23/2026).
- [8] Compound. “Compound iii documentation.” (), [Online]. Available: <https://docs.compound.finance> (visited on 05/23/2026).
- [9] Ethereum Foundation. “Universal upgradeable proxy standard (uups).” (), [Online]. Available: <https://eips.ethereum.org/EIPS/eip-1822> (visited on 05/23/2026).
- [10] OpenZeppelin. “Openzeppelin documentation.” (), [Online]. Available: <https://docs.openzeppelin.com> (visited on 05/23/2026).
- [11] Ethereum Foundation. “Sign-in with ethereum.” (), [Online]. Available: <https://eips.ethereum.org/EIPS/eip-4361> (visited on 05/23/2026).
- [12] Nomic Foundation. “Ethers.js v6 documentation.” (), [Online]. Available: <https://docs.ethers.org/v6/> (visited on 05/23/2026).
- [13] Chainlink. “Chainlink data feeds — developer documentation.” (), [Online]. Available: <https://docs.chain.link/data-feeds> (visited on 05/23/2026).
- [14] “Rabbitmq documentation — amqp 0-9-1 model explained.” (), [Online]. Available: <https://www.rabbitmq.com/tutorials/amqp-concepts> (visited on 05/23/2026).

- [15] Hardhat Team. “Hardhat — ethereum development environment.” (), [Online]. Available: <https://hardhat.org/docs> (visited on 05/23/2026).
- [16] Safe Global. “Safe multisig — smart account documentation.” (), [Online]. Available: <https://docs.safe.global/> (visited on 05/23/2026).